

网络空间安全智能决策

第1章 人工智能和网络安全概述

人工智能概述

第四次工业革命 - 人工智能

- 21世纪，是人工智能的时代
- 从1954年第一台可编程机器人诞生，到如今人工智能被应用在多个领域行业，从现在到未来，人工智能都将具有极大的研究及应用价值



人工智能概述

国家AI相关计划

- 国务院发布的《新一代人工智能发展规划》，分析当前的战略态势，提出了到2030年“人工智能理论、技术与应用总体达到世界领先水平”的战略目标，规划了三步走的发展路径：
- 提出“人工智能研发攻关、产品应用和产业培育”
 - 大力发展人工智能新兴产业、加快推进产业智能化升级、大力发展智能企业
 - 建设安全便捷的智能社会：智能服务、社会治理智能化、提升公共安全保障能力，促进社会交往共享互信等

人工智能概述

人工智能技术在我国国民经济行业中的应用主题

行业名称及代码	所属门类及代码	相关应用主题列举
农业01	农林牧渔业 A	智能育种，农业生产管理智能化，农业专家模型，精准农业（病虫害管理、田间管理），农业传感器芯片
林业02	同上	森林精准监控与管理，精准林木育种与抚育，森林火灾智能预防
畜牧业03	同上	奶牛饲养智能化管理（第5章有例子）
渔业04	同上	渔情预报
煤炭开采06	采矿业 B	煤矿生产仪器智能化，煤矿开采方案优化，煤矿井下参数监控，煤矿救灾机器人；智慧矿山
石油和天然气开采07	同上	自动钻井系统，海底油气生产机器人
其他采矿业08、09、10	同上	自动采矿系统

人工智能概述

人工智能技术在我国国民经济行业中的应用主题

行业名称及代码	所属门类及代码	相关应用主题列举
农副产品加工13、食品制造14、酒和饮料制造15	制造业 C	基于数据挖掘的产品设计，基于计算机视觉的产品制造，产品质量监控，生产过程智能化，产品虚拟助理（根据自动收集的数据回答相关问题）
烟草制品16	同上	烟草仓库智能监控
纺织业17、服装业18、皮革业19	同上	智能装备—数字化工厂，智能穿戴，智能家居，智能产品检测，智能化工厂；智能针织产业园
木材等加工20、家具制造21、造纸22	同上	基于专家系统的智能制造，模糊控制，质量监控，智能家居
印刷业23	同上	制版机器人，智能印刷，智能印刷工厂
文体娱乐用品制造24	同上	机器人玩具，智能音箱
燃料加工25	同上	核废料智能管理；核电站智能化管理

人工智能相关应用

人类学习大概包括以下几种现象：

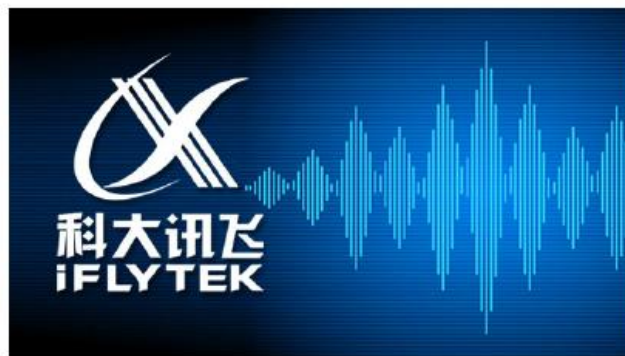
- 对语言、文字的**认知与识别**
- 对图像、场景、自然物体的**认知与识别**
- 对**规则的学习与掌握**：
 - 例如：下雨天要带伞，天冷了要穿衣等
- 对复杂事物的**推理与判断**能力：
 - 例如：事物的正误的判断

**机器学习的最终目的就是使得计算机拥有
和人类一样的学习能力**

人工智能相关应用



抖音的短视频推荐



科大讯飞的语音识别



ZAO的视频实时换脸

如何实现？➡️ 机器学习方法



AlphaGo大胜柯洁



In "Nature" 27 January 2016:

- "DeepMind's program AlphaGo beat Fan Hui, the European Go champion, five times out of five in tournament conditions..."
- "AlphaGo was not preprogrammed to play Go: rather, it learned using a general-purpose algorithm that allowed it to interpret the game's patterns."
- "...AlphaGo program applied **deep learning** in neural networks (convolutional NN) — brain-inspired programs in which connections between layers of simulated neurons are strengthened through examples and experience."

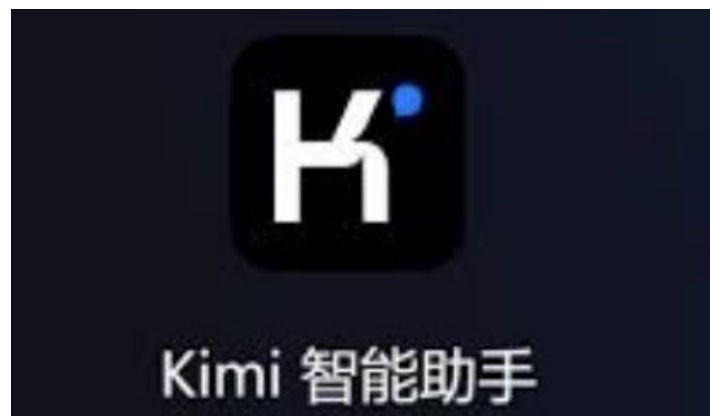
人工智能相关应用

波士顿人形机器人



人工智能相关应用

人工智能聊天机器人



人工智能相关应用

自动驾驶

2018年8月，特斯拉创始人伊隆-马斯克（Elon Musk）宣布，“具有全自动驾驶功能”的全新硬件系统Autopilot 2.0有望于2019年初正式亮相。

据悉，这一全新系统将包含8个摄像头用于覆盖360度可视范围，对周围环境的**监控距离最远可达250米**。车辆配备的12个超声波传感器完善了视觉系统，信息处理芯片由**NVIDIA Drive PX 2**支持。



网络安全概述

- 网络安全一直和**人工智能**相伴相生，从网络安全诞生的那一天起，人们就一直试图使用**自动化**的方式去解决安全问题。网络安全专家一直试图把自己对网络威胁的理解转换成机器可以理解的方式，比如黑白名单、正则表达式，然后利用机器强大的计算能力，夜以继日地从流量、日志、文件中寻找似曾相识的各类威胁。似乎这一切就是那么天经地义并无懈可击。但事情似乎又没有那么简单，机器其实并没有完全学到人的经验，网络安全专家一眼就可以识破的变形，对于机器却难以理解；更可怕的是，恶意程序数量呈指数级增长，各类新型攻击方式层出不穷，依靠极其有限的网络专家总结的经验和几个安全厂商所谓的样本交换，已经难以应付现在的网络安全威胁

网络安全问题案例

垃圾邮件识别

- 垃圾邮件作为因特网中最具有争议的副产品，对于企业邮箱用户的影响首先就在于给日常办公和邮箱管理者带来额外负担。根据不完全统计，在高效的反垃圾环境下仍然有80%的用户每周需要耗费10分钟左右的时间处理垃圾邮件，而对于中国多数企业邮件应用仍处于低效率反垃圾环境的情况下，这个比例更是呈现数十倍的增长，中国垃圾邮件的总量已经达到全球第三。对于企业邮件服务商而言，垃圾邮件的恶意投送，还会大量占用网络资源，使得邮件服务器85%的系统资源用于处理垃圾邮件的识别，不仅资源浪费极其严重，甚至可能导致网络阻塞瘫痪，影响企业正常业务邮件的沟通
- 更严重的垃圾邮件问题甚至不仅只是影响企业工作效率，甚至会祸及整个服务器。由于企业邮箱平台往往通信质量更好，更容易为国际反垃圾邮件平台白名单所接纳，因此成为垃圾邮件投送者甚至网络黑客的重点攻击目标。通过劫持这些企业邮箱，垃圾邮件投送者就可以大大提高垃圾邮件投送率，给企业邮箱用户造成无法预知的影响

网络安全问题案例

垃圾邮件识别

Enron-Spam 数据集是一个公开的电子邮件数据集，它包含了安然公司（Enron Corporation）150位高级管理人员的往来邮件。这些邮件在安然公司接受美国联邦能源监管委员会调查时被公布到网上。数据集被广泛用于机器学习领域的研究，包括文档分类、词性标注、垃圾邮件识别等任务。由于这些邮件都是在真实环境下产生的，因此具有很高的实际意义

- ✓ 大规模：数据集包含了大约50万封邮件，这为机器学习模型的训练提供了丰富的数据。
- ✓ 真实性：与其他合成或人为创建的数据集不同，Enron-Spam 数据集中的邮件是真实的，反映了真实世界的通信模式。
- ✓ 已标注：邮件已经被标注为“正常邮件”（ham）或“垃圾邮件”（spam），这使得数据集非常适合用于监督学习任务。

网络安全问题案例

负面评论识别

- 提到负面评论，我们首先要谈谈水军。顾名思义，水军是指在论坛大量灌水的人员，他们受雇于网络公关公司，以发帖回帖为主要手段，是为雇主进行网络造势的网络人员，有专职和兼职之分。一般来讲，发帖回帖造势常常需要成百上千个人共同完成，那些临时在网上征集来的发帖的人在行内被叫做“网络水军”。2009年12月央视报道网络水军这一新兴现象之后，受到社会广泛关注，不少长期在线的网虫们纷纷加入网络水军一族。水军的大行其道，严重影响了社会舆论，尤其是有组织地针对公众人物、企业的诽谤和攻击行为，造成严重的社会影响。大型的社交媒体以及知名论坛，早期都需要雇佣大量的运营支撑人员来人工鉴别处理
- 通过机器学习的技术可以达到一定程度的自动化识别负面评论

网络安全问题案例

用户行为分析与恶意行为检测

- 尽管包括数据防泄露（Data Leakage Prevention, DLP）在内的众多安全产品已被添加到企业网络安全战略中，但是确保机密数据和资产的安全仍是企业和组织面临的一大挑战。据美国波耐蒙研究所2015年的一项调查显示，目前损失最为惨重的网络犯罪案件多数是由企业内部人员监守自盗导致，其次才是分布式拒绝服务攻击（DDoS）和基于Web的攻击。

网络安全问题案例

用户行为分析与恶意行为检测

- 与企业内部人员监守自盗导致危害同样巨大的还有内部员工的异常操作。2017年1月31日23:00左右，GitLab一名系统管理员在极度疲劳的情况下，尝试删除一个空的目录，结果指令发往了另外一台服务器的命令窗口，等他回过神来的时候，27分钟过去，终止删除操作为时已晚，大约300GB左右的数据只剩下约4.5GB。GitLab.com丢失了6小时的数据库数据。
- 2017年4月5日，知名的VPS服务商DigitalOcean出现了一次删除生产数据库的事故。删库导致DigitalOcean的控制面板和API无法正常使用，时间长达4小时56分。DigitalOcean官博撰文致歉，并说明了事故前后过程：“在2017年4月5日10:24，我们开始收到公共服务功能失效的警报。在警报最初的3分钟，我们发现主数据库已经被删除了。4分钟后，我们开始从一台有延迟的数据库副本着手恢复。在接下来的4个小时中，我们复制并把数据恢复到主备副本。服务中断这么长时间，主要是因为从副本把数据恢复到在线服务器这个过程非常耗时。”此次事故的根本原因是工程师的配置错误，有个用于自动化测试的程序，错误使用了生产环境的证书。

网络安全问题案例

用户行为分析与恶意行为检测

- 我们将恶意内部人员和内部员工的异常操作统称为恶意操作。检测这种恶意操作需要使用高级技术，比如用户行为分析（User Behaviors Analysis, UBA），这种新兴技术可提供以往被遗漏的数据保护和欺诈检测功能。结合用户日常操作的系统，UBA利用一种专门的安全分析算法，不仅可以关注初始登录操作，还能跟踪用户的一举一动。UBA有两个主要功能：它有助于为用户执行的正常活动确立基线；迅速识别偏离正常行为的异常行为，以便安全分析员执行调查。某些异常行为可能乍一看不是恶意的，但是这需要安全分析员进一步调查情况，以确定它是合法行为还是恶意行为。

网络安全问题案例

WebShell检测

- WebShell就是以ASP、PHP、JSP或者CGI等网页文件形式存在的一种命令执行环境，也可以将其称为一种网页后门。黑客在入侵了一个网站后，通常会将ASP或PHP后门文件与网站服务器Web目录下正常的网页文件混在一起，然后就可以使用浏览器来访问ASP或者PHP后门，得到一个命令执行环境，以达到控制网站服务器的目的。顾名思义，“Web”的含义是需要服务器提供Web服务，“Shell”的含义是取得对服务器某种程度的操作权限。WebShell常常被入侵者利用，通过网站服务端口对网站服务器获取某种程度的操作权限。

网络安全问题案例

- 恶意程序识别
- 骚扰短信识别
- 反信用卡欺诈
- Linux后门检测
- 智能扫描器
- DGA域名检测
- 验证码识别

.....

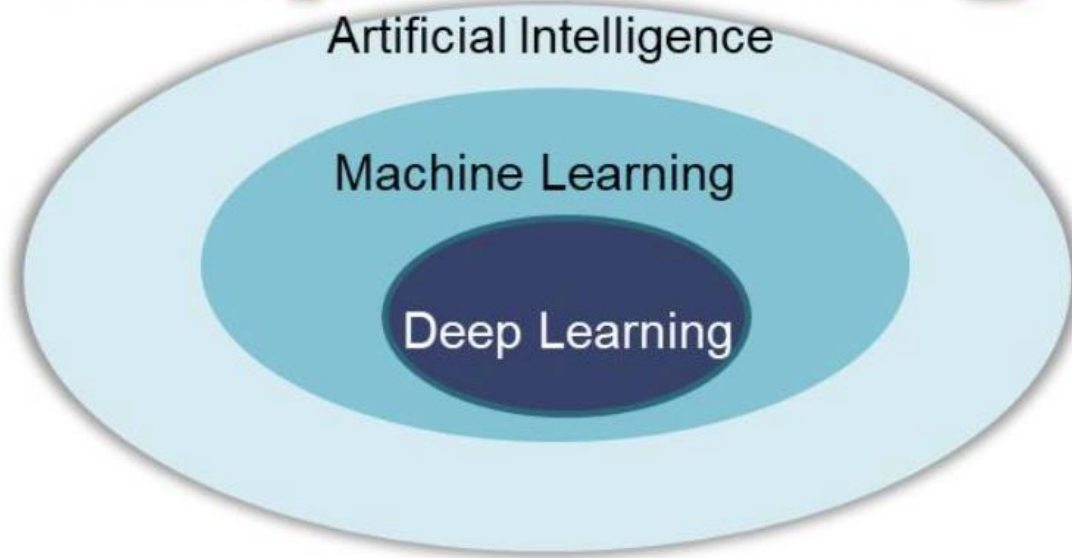
网络安全问题案例

- 如何让机器可以真正学会自动识别安全威胁？

机器学习可能是一个不错的答案

机器学习与人工智能的关系

Difference between Machine Learning & Deep Learning



机器学习与人工智能的关系

AI - 人工智能

机器学习 (ML)

- 监督学习 (Supervised Learning)
 - 分类 (Classification)
 - 回归 (Regression)
- 无监督学习 (Unsupervised Learning)
 - 聚类 (Clustering)
 - 降维 (Dimensionality Reduction)
- 强化学习 (Reinforcement Learning)
- 半监督学习 (Semi-supervised Learning)

深度学习 (Deep Learning)

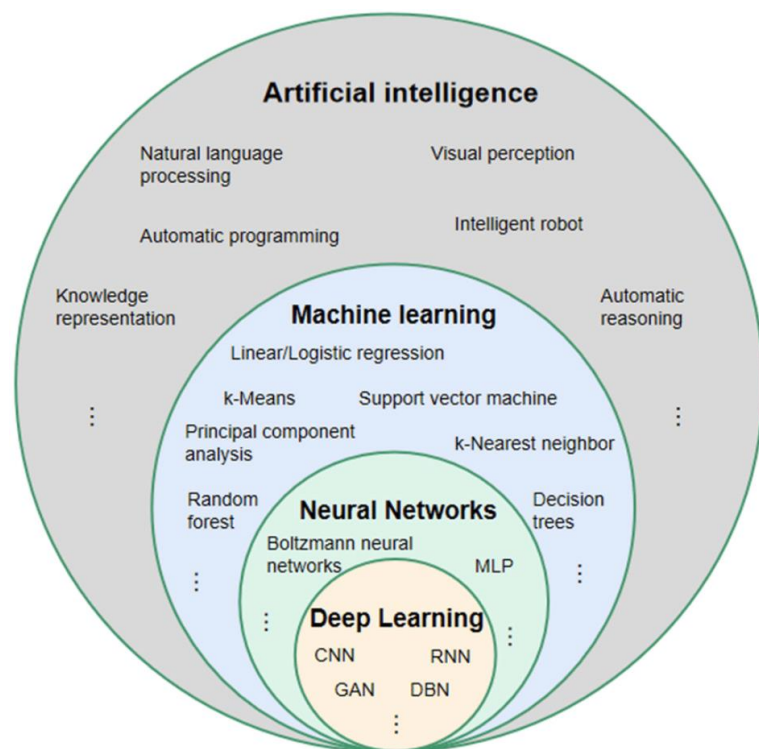
- 卷积神经网络 (CNN)
- 循环神经网络 (RNN)
- 长短期记忆网络 (LSTM)
- 变压器网络 (Transformer)

自然语言处理 (NLP)

- 语言翻译 (Translation)
- 情感分析 (Sentiment Analysis)
- 语音识别 (Speech Recognition)

计算机视觉 (Computer Vision)

- 图像识别 (Image Recognition)
- 目标检测 (Object Detection)
- 图像分割 (Image Segmentation)



机器学习

机器学习是从人工智能中产生的一个重要学科分支，是实现智能化的关键

经典定义：利用**经验**改善系统自身的性能[T.Mitchell教科书,1997]



经验 → 数据



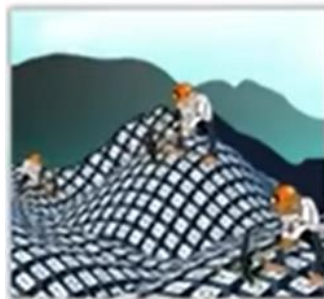
随着该领域的发展，目前主要研究**智能数据分析**的理论和方法，并已成为智能数据分析技术的源泉之一

大数据时代



大数据 ≠ 大价值

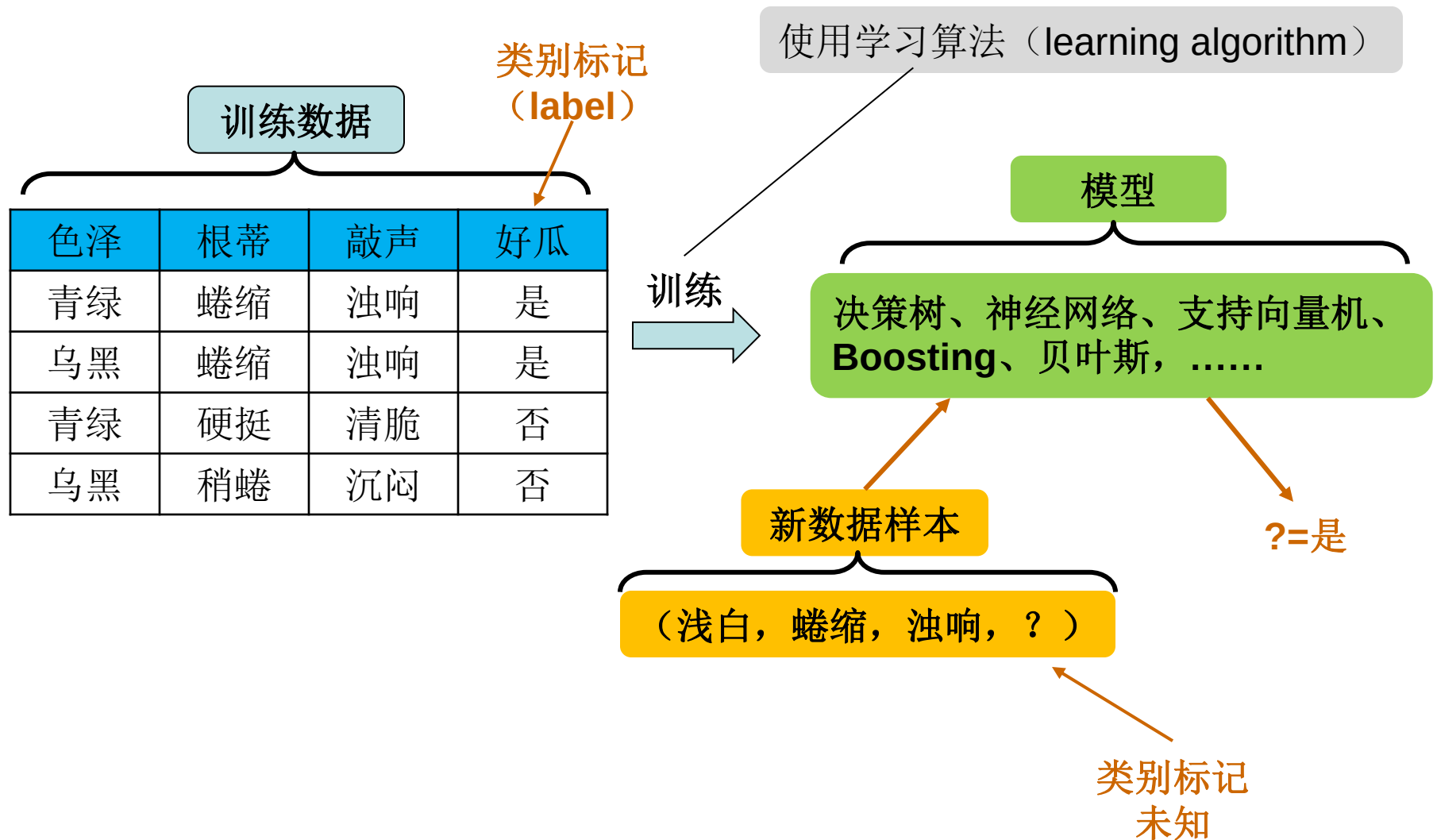
智能
数据分析



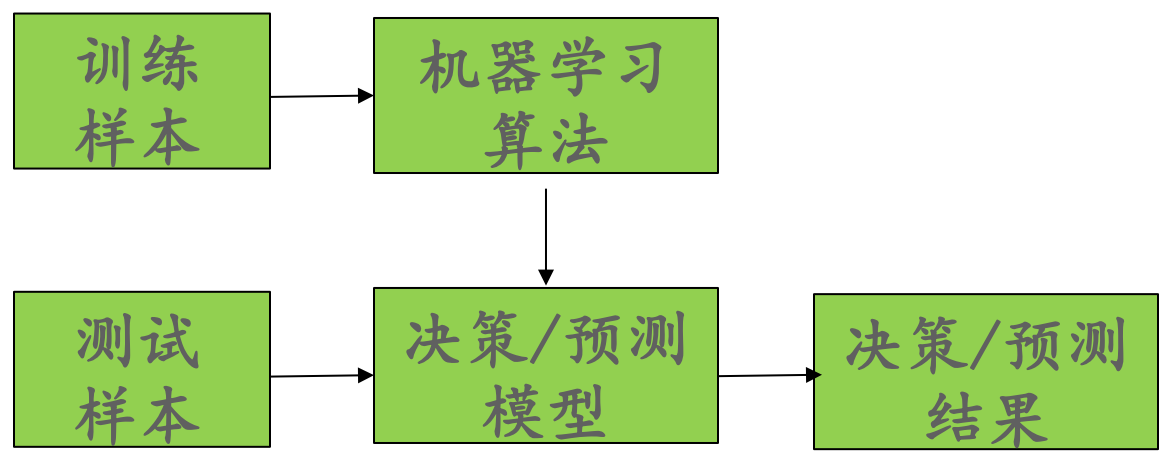
机器学习



典型的机器学习过程



典型的机器学习过程



分类

回归

未来40年人口老龄化趋势

年份	总人口数 (亿人)	60以上 (亿人)	60以上 (%)
2010年	13.48	1.73	12.83%
2020年	14.33	2.46	17.16%
2030年	14.44	3.41	23.61%
2040年	14.29	3.94	27.50%
2050年	13.73	3.78	27.50%

统计数据来源: 中国社科院

聚类

排序

名次	姓名	分数	趋势	简介
★ 1st	朱一龙	9.1038	-	演技一绝 * 与众不同
★ 2nd	杨紫	8.4625	★	全心全意, 演技有您
★ 3rd	赵丽颖	8.4100	★	相伴丽颖, 如影随行!
4	迪丽热巴	8.4025	★	吾心向往 永不分离
5	邓伦	8.3613	★	欢迎来到邓伦小站

典型的机器学习过程



经验 → 数据



- ◆ 机器学习是这样一门学科，研究如何通过计算的手段，利用经验来改善系统自身的性能
- ◆ 机器学习研究的主要内容，是通过计算机从数据中产生模型的算法，即学习算法，把经验数据提供给它，就能基于数据产生模型，面对新的样本，模型给出判断结果

机器学习理论基础



Leslie Valiant
(莱斯利·维利昂特)
(1949-)
2010年图灵奖

计算学习理论

Computational learning theory

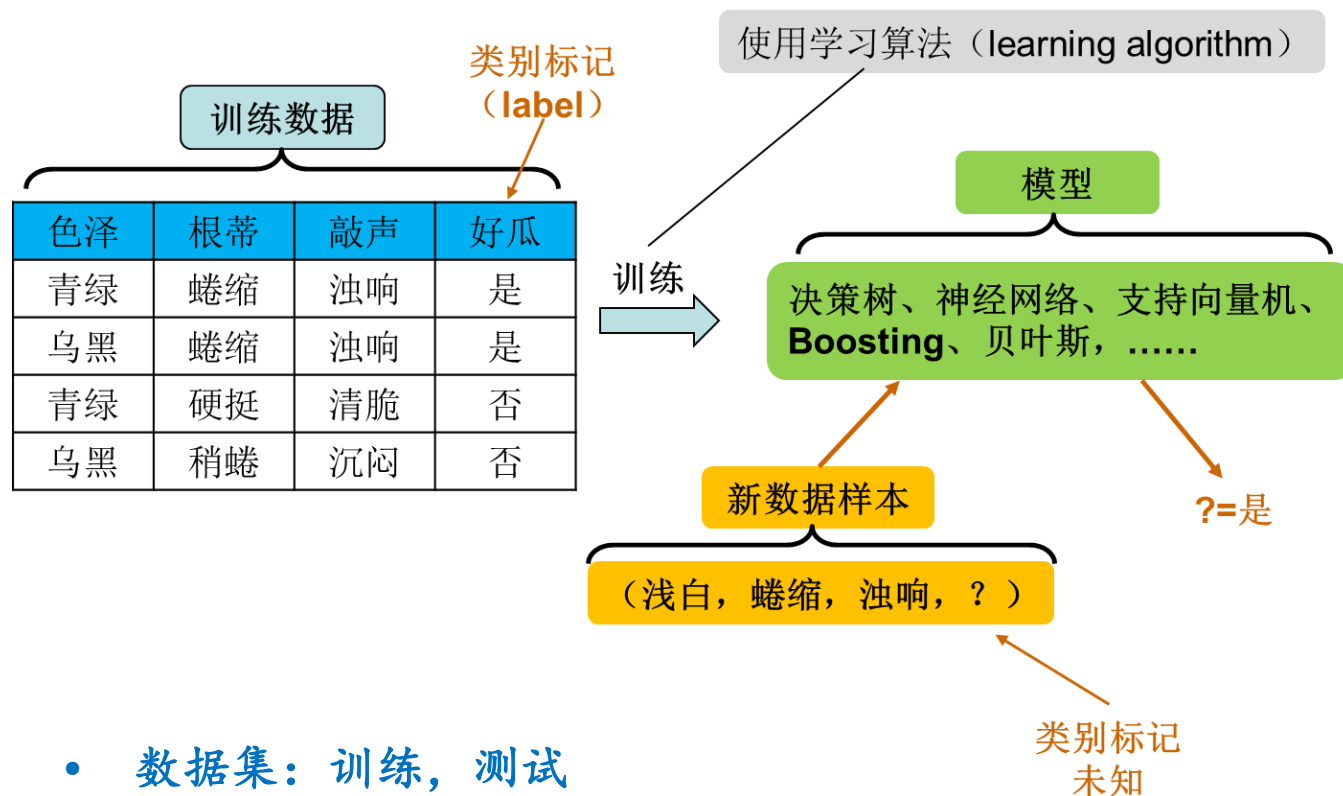
◆ 计算学习理论是机器学习的理论基础

◆ 最重要的理论模型：

PAC (probably Approximately Correct, 概率近似正确) learning model [Valiant, 1984]

$$P(|f(x) - y| \leq \epsilon) \geq 1 - \delta$$

机器学习 - 基本术语



- 假设 (hypothesis)
- 真相 (ground-truth)
- 学习器 (learner)

- 分类, 回归
- 二分类, 多分类
- 正类, 反类

- 数据集: 训练, 测试
- 示例 (instance), 样例 (example)
- 样本 (sample)
- 属性 (attribute), 属性值
- 属性空间, 样本空间, 输入空间
- 特征向量
- 标记空间, 输出空间

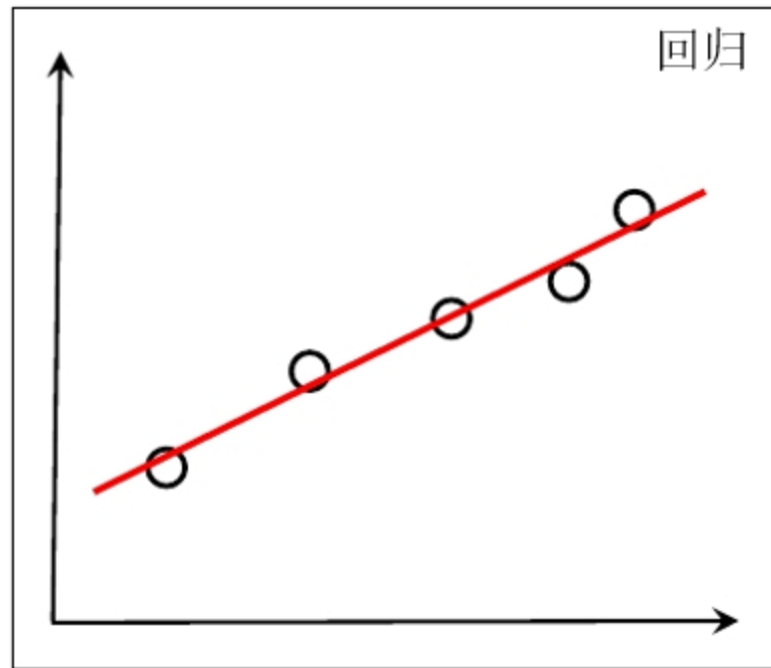
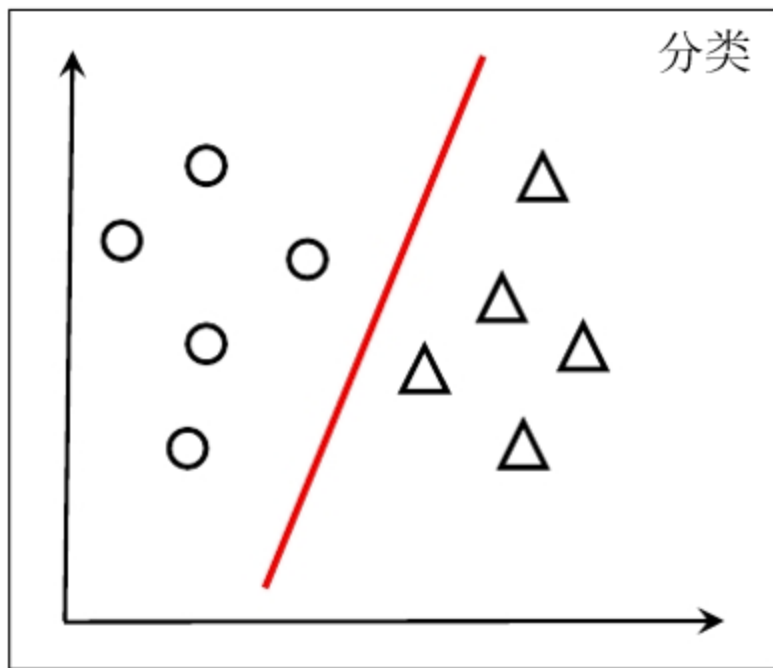
- 未见样本 (unseen instance)
- 未知分布
- 独立同分布 (i.i.d)

第2章 线性模型

线性模型

- 线性模型 (linear model) 是机器学习中最为基础模型，很多复杂模型都可以认为是由线性模型衍生得到
- 本章内容：两种线性模型（线性回归、对率回归）分别是回归和分类任务上常用的算法

线性模型



线性模型(linear model)试图学得一个通过属性的线性组合来进行预测的函数

$$f(x) = w_1x_1 + w_2x_2 + \dots + w_dx_d + b$$

向量形式: $f(x) = w^T x + b$

简单、基本、可理解性好

线性回归（Linear Regression）

$$f(x) = wx_i + b \quad \text{使得} \quad f(x_i) \simeq y_i$$

- 线性回归：擅长处理数值属性（连续）
- 离散属性怎么处理？
- 考虑是否存在“序”的关系

对离散属性，若属性值之间**存在**“序”（order）关系，可通过**连续化**将其转化为连续值：如三值属性“高度”的取值“高”“中”“低”可转化为 $\{1.0, 0.5, 0.0\}$

对离散属性，若属性值之间**不存在**“序”（order）关系，假定有**k个属性值**，则通常转化为**k维向量**：如属性“瓜类”的取值“西瓜”“南瓜”“黄瓜”可转化为 $(0, 0, 1), (0, 1, 0), (1, 0, 0)$

线性回归 (Linear Regression)

$$f(x) = wx_i + b \quad \text{使得} \quad f(x_i) \simeq y_i$$

令均方误差最小化, 有 $(w^*, b^*) = \arg \min_{(w, b)} \sum_{i=1}^m (f(x_i) - y_i)^2$

$$= \arg \min_{(w, b)} \sum_{i=1}^m (y_i - wx_i - b)^2$$

对 $E_{(w, b)} = \sum_{i=1}^m (y_i - wx_i - b)^2$ 进行最小二乘参数估计

线性回归（Linear Regression）

对 $E_{(w,b)} = \sum_{i=1}^m (y_i - wx_i - b)^2$ 进行最小二乘参数估计

最小二乘参数估计（Least Squares Estimation, LSE）：

- 数学优化技术
- 基本思想：通过最小化误差的平方和来寻找数据的最佳函数匹配
- 最小二乘通常有解析解，即通过求解偏导数并令其为0，可以得到正规方程组，直接计算出参数的估计值

线性回归 (Linear Regression)

$$E_{(w,b)} = \sum_{i=1}^m (y_i - wx_i - b)^2$$

分别对 w 和 b 求导:

$$\frac{\partial E_{(w,b)}}{\partial w} = 2 \left(w \sum_{i=1}^m x_i^2 - \sum_{i=1}^m (y_i - b) x_i \right)$$

$$\frac{\partial E_{(w,b)}}{\partial b} = 2 \left(mb - \sum_{i=1}^m (y_i - wx_i) \right)$$

令导数为 0, 得到闭式(closed-form)解 (解析解):

$$w = \frac{\sum_{i=1}^m y_i (x_i - \bar{x})}{\sum_{i=1}^m x_i^2 - \frac{1}{m} \left(\sum_{i=1}^m x_i \right)^2} \quad b = \frac{1}{m} \sum_{i=1}^m (y_i - wx_i)$$

多元(multi-variate)线性回归

$$f(\mathbf{x}_i) = \mathbf{w}^T \mathbf{x}_i + b \quad \text{使得} \quad f(\mathbf{x}_i) \simeq y_i$$

$$\mathbf{x}_i = (x_{i1}; x_{i2}; \dots; x_{id}) \quad y_i \in \mathbb{R}$$

把 $\mathbf{w}$ 和 b 吸收入向量形式 $\hat{\mathbf{w}} = (\mathbf{w}; b)$ ，数据集表示为

$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1d} & 1 \\ x_{21} & x_{22} & \cdots & x_{2d} & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{md} & 1 \end{pmatrix} = \begin{pmatrix} \mathbf{x}_1^T & 1 \\ \mathbf{x}_2^T & 1 \\ \vdots & \vdots \\ \mathbf{x}_m^T & 1 \end{pmatrix} \quad \mathbf{y} = (y_1; y_2; \dots; y_m)$$

多元(multi-variate)线性回归

同样采用最小二乘法求解，有

$$\hat{\mathbf{w}}^* = \arg \min_{\hat{\mathbf{w}}} (\mathbf{y} - \mathbf{X}\hat{\mathbf{w}})^T (\mathbf{y} - \mathbf{X}\hat{\mathbf{w}})$$

令 $E_{\hat{\mathbf{w}}} = (\mathbf{y} - \mathbf{X}\hat{\mathbf{w}})^T (\mathbf{y} - \mathbf{X}\hat{\mathbf{w}})$ $\hat{\mathbf{w}}$ 求导：

$$\frac{\partial E_{\hat{\mathbf{w}}}}{\partial \hat{\mathbf{w}}} = 2\mathbf{X}^T (\mathbf{X}\hat{\mathbf{w}} - \mathbf{y})$$
 令其为零可得 $\hat{\mathbf{w}}$

然而，麻烦来了：涉及矩阵求逆！

□若 $\mathbf{X}^T\mathbf{X}$ 满秩或正定，则 $\hat{\mathbf{w}}^* = (\mathbf{X}^T\mathbf{X})^{-1} \mathbf{X}^T\mathbf{y}$

□若 $\mathbf{X}^T\mathbf{X}$ 不满秩，则可解出多个 $\hat{\mathbf{w}}$

此时需求助于归纳偏好，或引入 正则化 (regularization)

线性模型的变化

对于样例 (x, y) , $y \in \mathbb{R}$, 若希望线性模型的预测值逼近真实标记, 则得到线性回归模型 $y = w^T x + b$

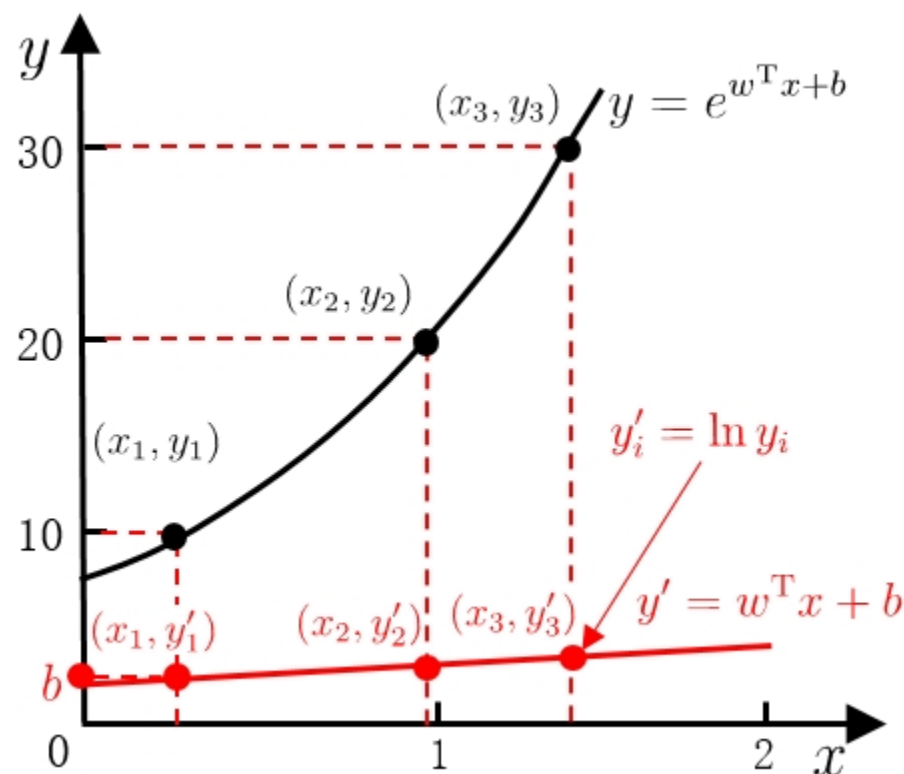
令预测值逼近 y 的衍生物?

若令 $\ln y = w^T x + b$

则得到对数线性回归

(log-linear regression)

实际是在用 $e^{w^T x + b}$ 逼近 y



广义(generalized)线性模型

一般形式: $y = \underline{g^{-1}}(\mathbf{w}^T \mathbf{x} + b)$



单调可微的 联系函数

令 $g(\cdot) = \ln(\cdot)$ 则得到 对数线性回归

$$\ln y = \mathbf{w}^T \mathbf{x} + b$$

...

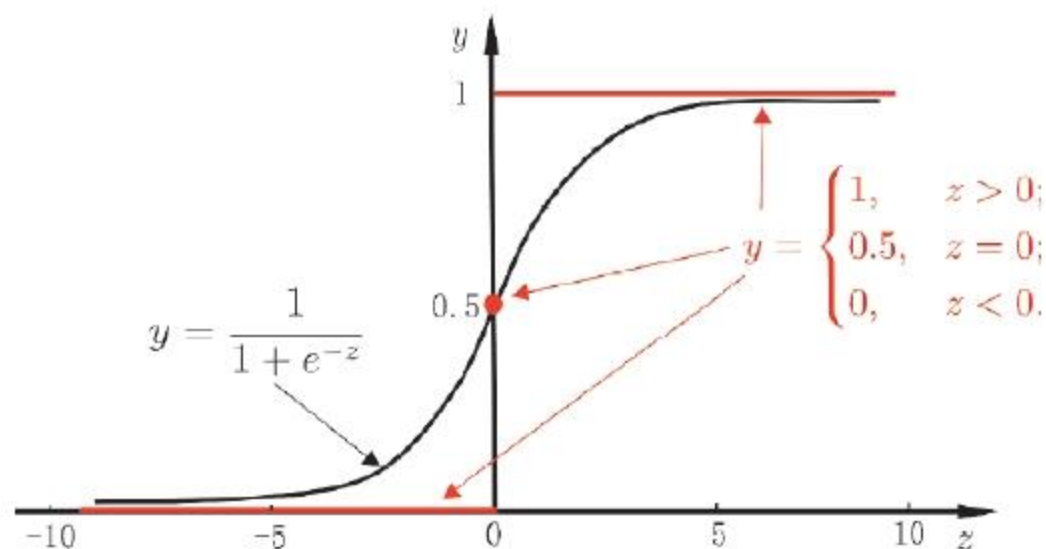
二分类任务

线性回归模型产生的实值输出 $z = \mathbf{w}^T \mathbf{x} + b$
期望输出 $y \in \{0, 1\}$

找 z 和 y 的
联系函数

理想的“单位阶跃函数”
(unit-step function)

$$y = \begin{cases} 0, & z < 0; \\ 0.5, & z = 0; \\ 1, & z > 0, \end{cases}$$



性质不好,
需找“替代函数”
(surrogate function)

常用
单调可微、任意阶可导

$$y = \frac{1}{1 + e^{-z}}$$

对数几率函数
(logistic function)
简称“对率函数”

对率回归 (了解)

以对率函数为联系函数:

$$y = \frac{1}{1 + e^{-z}} \quad \text{变为} \quad y = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}$$

即:

$$\ln \frac{y}{1 - y} = \mathbf{w}^T \mathbf{x} + b$$

“对数几率”
(log odds, 亦称 logit)

几率(odds), 反映了 $\mathbf{x}$ 作为正例的相对可能性

“对数几率回归” (logistic regression)
简称“对率回归”

- 无需事先假设数据分布
- 可得到“类别”的近似概率预测
- 可直接应用现有数值优化算法求取最优解

注意：它是分类学习算法

求解思路 (了解)

若将 y 看作类后验概率估计 $p(y = 1 \mid \mathbf{x})$, 则

$$\ln \frac{y}{1-y} = \mathbf{w}^T \mathbf{x} + b \quad \text{可写为} \quad \ln \frac{p(y = 1 \mid \mathbf{x})}{p(y = 0 \mid \mathbf{x})} = \mathbf{w}^T \mathbf{x} + b$$

于是, 可使用“极大似然法” (maximum likelihood method)

给定数据集 $\{(\mathbf{x}_i, y_i)\}_{i=1}^m$

最大化“对数似然” (log-likelihood) 函数

$$\ell(\mathbf{w}, b) = \sum_{i=1}^m \ln p(y_i \mid \mathbf{x}_i; \mathbf{w}, b)$$

求解思路 (了解)

令 $\boldsymbol{\beta} = (\mathbf{w}; b)$, $\hat{\mathbf{x}} = (\mathbf{x}; 1)$, 则 $\mathbf{w}^T \mathbf{x} + b$ 可简写为 $\boldsymbol{\beta}^T \hat{\mathbf{x}}$

$$\text{再令 } p_1(\hat{\mathbf{x}}_i; \boldsymbol{\beta}) = p(y = 1 \mid \hat{\mathbf{x}}_i; \boldsymbol{\beta}) = \frac{e^{\mathbf{w}^T \mathbf{x} + b}}{1 + e^{\mathbf{w}^T \mathbf{x} + b}}$$

$$p_0(\hat{\mathbf{x}}_i; \boldsymbol{\beta}) = p(y = 0 \mid \hat{\mathbf{x}}_i; \boldsymbol{\beta}) = 1 - p_1(\hat{\mathbf{x}}_i; \boldsymbol{\beta}) = \frac{1}{1 + e^{\mathbf{w}^T \mathbf{x} + b}}$$

则似然项可重写为 $p(y_i \mid \mathbf{x}_i; \mathbf{w}_i, b) = y_i p_1(\hat{\mathbf{x}}_i; \boldsymbol{\beta}) + (1 - y_i) p_0(\hat{\mathbf{x}}_i; \boldsymbol{\beta})$

于是, 最大化似然函数 $\ell(\mathbf{w}, b) = \sum_{i=1}^m \ln p(y_i \mid \mathbf{x}_i; \mathbf{w}, b)$

等价于最小化 $\ell(\boldsymbol{\beta}) = \sum_{i=1}^m \left(-y_i \boldsymbol{\beta}^T \hat{\mathbf{x}}_i + \ln \left(1 + e^{\boldsymbol{\beta}^T \hat{\mathbf{x}}_i} \right) \right)$

高阶可导连续凸函数, 可用经典的数值优化方法
如梯度下降法/牛顿法 [Boyd and Vandenberghe, 2004]

模型性能度量

□ 如何评估模型性能优劣

- 性能度量是衡量模型泛化能力的评价标准
- 使用不同的性能度量往往会导致不同的评判结果
- 什么样的模型是“好”的，不仅取决于算法和数据，还取决于任务需求

□ 回归任务常用均方误差

$$E(f; D) = \frac{1}{m} \sum_{i=1}^m (f(\mathbf{x}_i) - y_i)^2$$

模型性能度量

□ 分类任务

□ 错误率:

$$E(f; D) = \frac{1}{m} \sum_{i=1}^m \mathbb{I}(f(\mathbf{x}_i) \neq y_i)$$

□ 精度:

$$\begin{aligned} \text{acc}(f; D) &= \frac{1}{m} \sum_{i=1}^m \mathbb{I}(f(\mathbf{x}_i) = y_i) \\ &= 1 - E(f; D) . \end{aligned}$$

模型性能度量

□ 分类任务

查准率 VS. 查全率

表 2.1 分类结果混淆矩阵

真实情况	预测结果	
	正例	反例
正例	TP (真正例)	FN (假反例)
反例	FP (假正例)	TN (真反例)

□ 查准率:

$$P = \frac{TP}{TP + FP}$$

所有预测为正例的样本中有多大比例是
真的正例

□ 查全率:

$$R = \frac{TP}{TP + FN}$$

所有正例中有多大比例被预测为正例

模型性能度量

□ 分类任务

F1 度量

表 2.1 分类结果混淆矩阵

真实情况	预测结果	
	正例	反例
正例	TP (真正例)	FN (假反例)
反例	FP (假正例)	TN (真反例)

$$F1 = \frac{2 \times P \times R}{P + R}$$

$$\frac{1}{F1} = \frac{1}{2} \cdot \left(\frac{1}{P} + \frac{1}{R} \right)$$

□ 查准率:

$$P = \frac{TP}{TP + FP}$$

□ 查全率:

$$R = \frac{TP}{TP + FN}$$

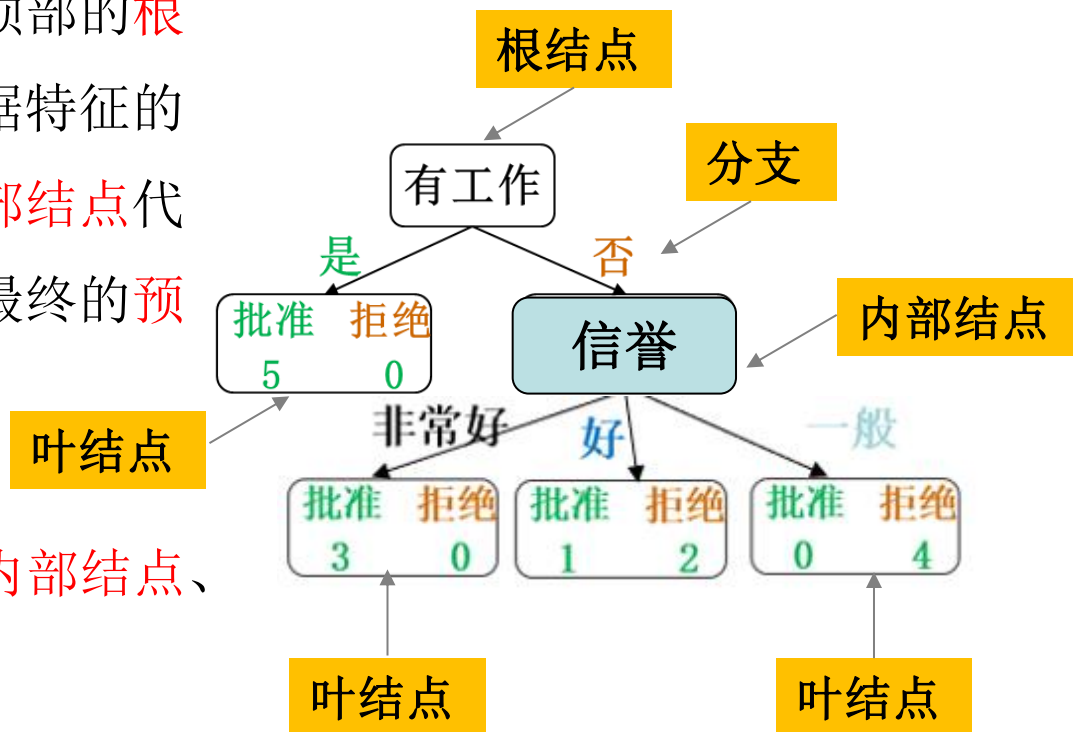
$$= \frac{2 \times TP}{\text{样例总数} + TP - TN}$$

第3章 决策树

决策树 - 相关概念

□ **核心思想**: 一种**监督学习算法**，广泛应用于分类和回归任务，通过从顶部的**根结点**到底部的**叶结点**，依据数据特征的递归分割来建模决策，每个**内部结点**代表一个**属性**，每个**叶结点**代表最终的**预测结果**

□ **模型结构**: 决策树由**根结点**、**内部结点**、**叶结点**和**分支**组成



决策树 - 相关概念

决策树的训练与测试

□ **训练阶段：**通过对**训练样本**的分析来确定划分属性(即内部结点对应的属性)

□ **测试阶段：**将测试示例从根结点开始，沿着划分属性构成的判定测试序列下行，直到叶结点

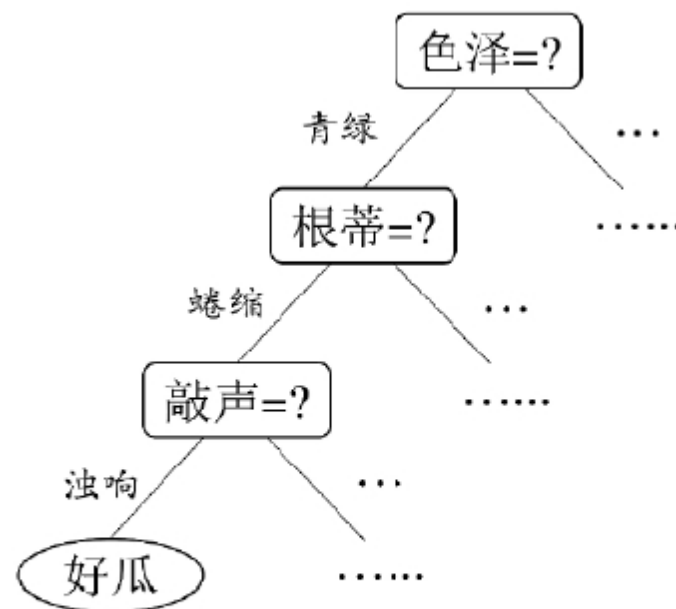


图 4.1 西瓜问题的一棵决策树

决策树 – 发展史

- 第一个决策树算法：CLS (Concept Learning System)
[E. B. Hunt, J. Marin, and P. T. Stone's book "*Experiments in Induction*" published by Academic Press in 1966]
- 使决策树受到关注、成为机器学习主流技术的算法：ID3
[J. R. Quinlan's paper in a book "*Expert Systems in the Micro Electronic Age*" edited by D. Michie, published by Edinburgh University Press in 1979]
- 最常用的决策树算法：C4.5
[J. R. Quinlan's book "*C4.5: Programs for Machine Learning*" published by Morgan Kaufmann in 1993]



决策树 – 发展史

- 可以用于回归任务的决策树算法： **CART** (Classification and Regression Tree)
[L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone's book "Classification and Regression Trees" published by Wadsworth in 1984]
- 基于决策树的**最强大算法**： RF (Random Forest)
[L. Breiman's MLJ'01 paper "Random Forest"]
这是一种“集成学习”方法 → 第 8 章

决策树 - 基本流程

- 决策树算法的基本流程遵循“分而治之”的策略
- 自根至叶的递归过程
- 在每个中间结点寻找一个“划分” (split or test) 属性

三种停止条件:

- (1) 当前结点包含的样本全属于同一类别，无需划分;
- (2) 当前属性集为空, 或是所有样本在所有属性上取值相同，无法划分;
- (3) 当前结点包含的样本集合为空，不能划分.

决策树 - 基本流程

三种停止条件:

- (1) 当前结点包含的样本全属于同一类别，无需划分;
- (2) 当前属性集为空,或是所有样本在所有属性上取值相同，无法划分;
- (3) 当前结点包含的样本集合为空，不能划分.

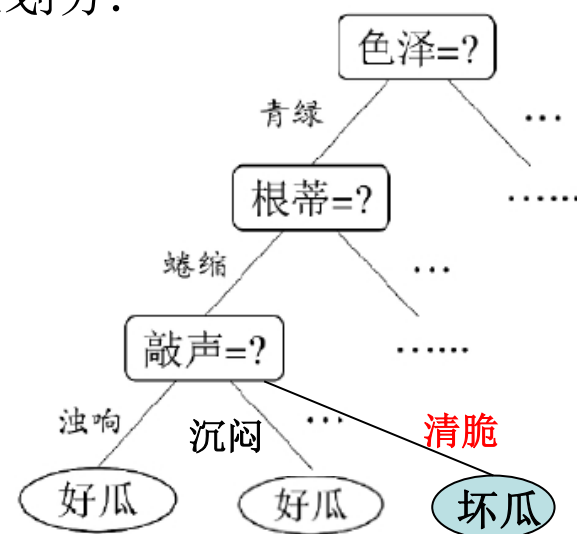
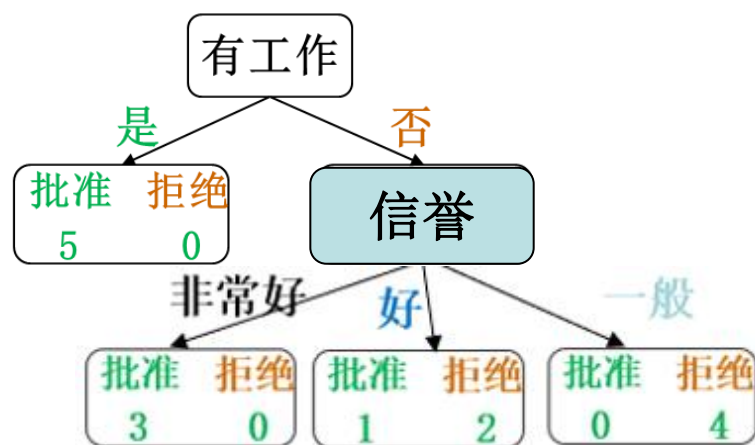


图 4.1 西瓜问题的一棵决策树

决策树 – 最优属性选择

□ 问题：根结点的选择该用哪个特征呢？接下来如何切分？

□ 目标：通过一种衡量标准，来计算通过不同特征进行分支选择后的分类情况，找出最好的那个当成根结点，以此类推。

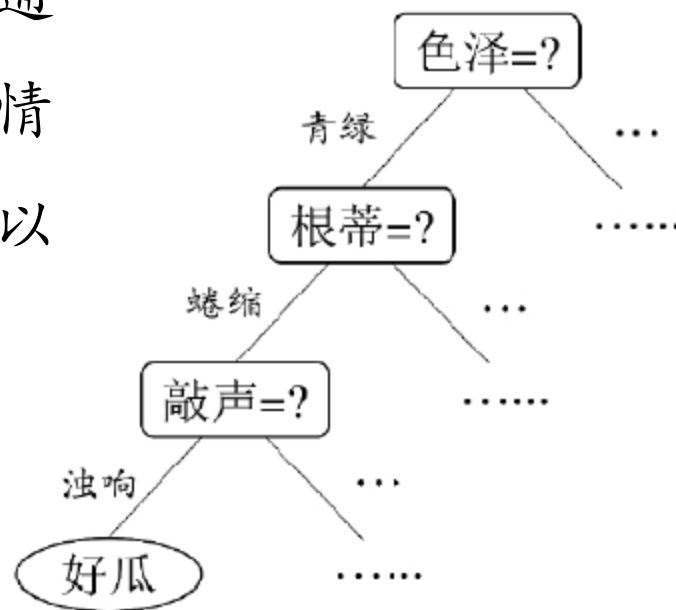


图 4.1 西瓜问题的一棵决策树

决策树 – 信息熵 (Entropy)

□信息熵：表示随机变量不确定性的度量，即度量样本集合纯度

假定当前样本集合 D 中有 y 类，第 k 类样本所占的比例为 p_k
信息熵的计算公式：

$$Ent(D) = - \sum_{k=1}^{|y|} p_k \log_2(p_k)$$

计算信息熵时约定：

若 $p = 0$, 则 $p \log_2 p = 0$

$Ent(D)$ 的最小值为 0，最大值为 $\log_2 |y|$

$Ent(D)$ 的值越小，则 D 的纯度越高

决策树 – 信息熵 (Entropy)

□ 一个例子：A集合 [1,1,1,1,1,1,1,1,2,2]，B集合[1,2,3,4,5,6,7,8,9,10]

假定当前样本集合 D 中有 y 类，第 k 类样本所占的比例为 p_k
信息熵的计算公式：

$$Ent(D) = - \sum_{k=1}^{|y|} p_k \log_2(p_k)$$

➤ 显然A集合的信息熵要低（A只有两种类别），相对稳定，
而B中类别太多，信息熵会大很多

决策树 – 信息增益 (Information Gain)

ID3算法中使用

□ 离散属性 $\mathbf{a}$ 的取值 $\{a^1, a^2, \dots, a^V\}$

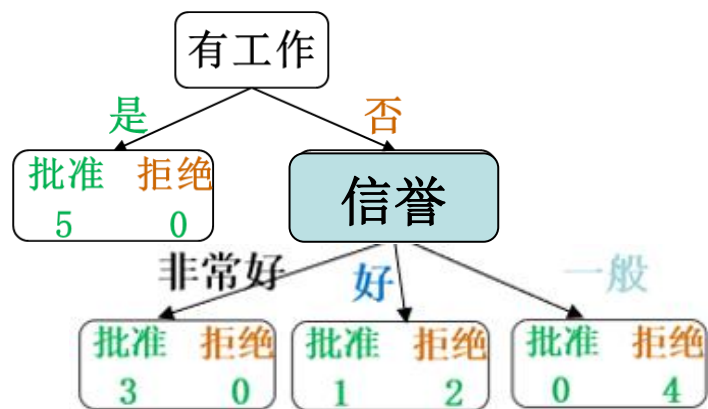
□ D^v : 样本集合 $\mathbf{D}$ 中在 $\mathbf{a}$ 上取值 $= a^v$ 的样本集合

$$Ent(D) = - \sum_{k=1}^{|y|} p_k \log_2(p_k)$$

衡量我们选择某个属性进行划分时信息熵的变化

$$Gain(D, a) = \underbrace{Ent(D)}_{\text{划分前的信息熵}} - \sum_{v=1}^V \underbrace{\frac{|D^v|}{|D|}}_{\text{第 } v \text{ 个分支的权重, 样本越多越重要}} \underbrace{Ent(D^v)}_{\text{划分后的信息熵}}$$

以属性 $\mathbf{a}$ 对数据 $\mathbf{D}$ 进行划分所得的信息增益



第 v 个分支的权重,
样本越多越重要

不确定性降低的程度

决策树 - 信息增益(information gain)

决策树构造实例 - 西瓜数据集

- ✓ 数据: 17个样本
- ✓ 特征: 6种
- ✓ 目标: 构造决策树

编号	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
1	青绿	蜷缩	浊响	清晰	凹陷	硬滑	是
2	乌黑	蜷缩	沉闷	清晰	凹陷	硬滑	是
3	乌黑	蜷缩	浊响	清晰	凹陷	硬滑	是
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	是
5	浅白	蜷缩	浊响	清晰	凹陷	硬滑	是
6	青绿	稍蜷	浊响	清晰	稍凹	软粘	是
7	乌黑	稍蜷	浊响	稍糊	稍凹	软粘	是
8	乌黑	稍蜷	浊响	清晰	稍凹	硬滑	是
9	乌黑	稍蜷	沉闷	稍糊	稍凹	硬滑	否
10	青绿	硬挺	清脆	清晰	平坦	软粘	否
11	浅白	硬挺	清脆	模糊	平坦	硬滑	否
12	浅白	蜷缩	浊响	模糊	平坦	软粘	否
13	青绿	稍蜷	浊响	稍糊	凹陷	硬滑	否
14	浅白	稍蜷	沉闷	稍糊	凹陷	硬滑	否
15	乌黑	稍蜷	浊响	清晰	稍凹	软粘	否
16	浅白	蜷缩	浊响	模糊	平坦	硬滑	否
17	青绿	蜷缩	沉闷	稍糊	稍凹	硬滑	否

决策树 - 信息增益(information gain)

决策树构造实例 - 西瓜数据集

- ✓ 在历史数据17个样本中，8个好瓜，9个坏瓜，此时历史数据集的信息熵为：

$$Ent(D) = -\frac{8}{17} \log_2 \left(\frac{8}{17} \right) - \frac{9}{17} \log_2 \left(\frac{9}{17} \right) = 0.998$$

D	是	否
Pr ob	$\frac{8}{17}$	$\frac{9}{17}$

- ✓ 6个特征选谁当根结点呢？
- ✓ 6个特征逐一分析，分别计算其熵值

决策树 - 信息增益(information gain)

决策树构造实例 - 西瓜数据集

$$Ent(D) = - \sum_{k=1}^{|y|} p_k \log_2(p_k)$$

□ 以属性“色泽”为例，其对应的3个数据子集分别为 D^1 (色泽=青绿), D^2 (色泽=乌黑), D^3 (色泽=浅白)

□ 子集 D^1 包含编号为{1, 4, 6, 10, 13, 17}的6个样例，其中正例占 $p_1 = \frac{3}{6}$ ，反例占 $p_2 = \frac{3}{6}$ ， D^2 、 D^3 同理，3个结点的信息熵为：

$$Ent(D^1) = -(\frac{3}{6} \log_2 \frac{3}{6} + \frac{3}{6} \log_2 \frac{3}{6}) = 1.000$$

$$Ent(D^2) = -(\frac{4}{6} \log_2 \frac{4}{6} + \frac{2}{6} \log_2 \frac{2}{6}) = 0.918$$

$$Ent(D^3) = -(\frac{1}{5} \log_2 \frac{1}{5} + \frac{4}{5} \log_2 \frac{4}{5}) = 0.722$$

□ 属性“色泽”的信息增益为

$$\begin{aligned} \text{Gain}(D, \text{色泽}) &= Ent(D) - \sum_{v=1}^3 \frac{|D^v|}{|D|} Ent(D^v) \\ &= 0.998 - (\frac{6}{17} \times 1.000 + \frac{6}{17} \times 0.918 + \frac{5}{17} \times 0.722) \\ &= 0.109 \end{aligned}$$

决策树 - 信息增益(information gain)

决策树构造实例 - 西瓜数据集

- ✓ 数据: 17个样本
- ✓ 特征: 6种
- ✓ 目标: 构造决策树

编号	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
1	青绿	蜷缩	浊响	清晰	凹陷	硬滑	是
2	乌黑	蜷缩	沉闷	清晰	凹陷	硬滑	是
3	乌黑	蜷缩	浊响	清晰	凹陷	硬滑	是
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	是
5	浅白	蜷缩	浊响	清晰	凹陷	硬滑	是
6	青绿	稍蜷	浊响	清晰	稍凹	软粘	是
7	乌黑	稍蜷	浊响	稍糊	稍凹	软粘	是
8	乌黑	稍蜷	浊响	清晰	稍凹	硬滑	是
9	乌黑	稍蜷	沉闷	稍糊	稍凹	硬滑	否
10	青绿	硬挺	清脆	清晰	平坦	软粘	否
11	浅白	硬挺	清脆	模糊	平坦	硬滑	否
12	浅白	蜷缩	浊响	模糊	平坦	软粘	否
13	青绿	稍蜷	浊响	稍糊	凹陷	硬滑	否
14	浅白	稍蜷	沉闷	稍糊	凹陷	硬滑	否
15	乌黑	稍蜷	浊响	清晰	稍凹	软粘	否
16	浅白	蜷缩	浊响	模糊	平坦	硬滑	否
17	青绿	蜷缩	沉闷	稍糊	稍凹	硬滑	否

决策树 - 信息增益(information gain)

决策树构造实例 - 西瓜数据集

□ 类似的，其他属性的信息增益为

$$\text{Gain}(D, \text{根蒂}) = 0.143$$

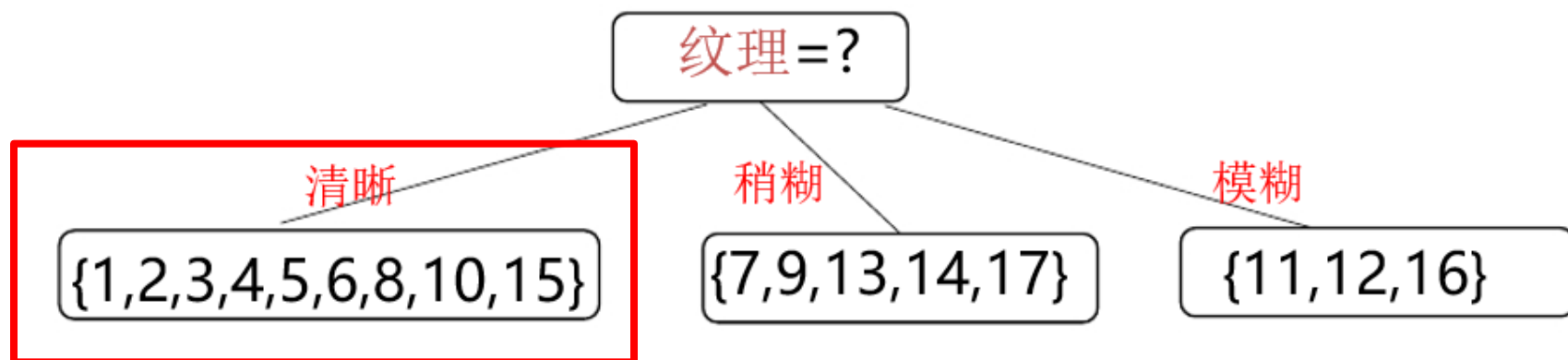
$$\text{Gain}(D, \text{敲声}) = 0.141$$

$$\text{Gain}(D, \text{纹理}) = 0.381$$

$$\text{Gain}(D, \text{脐部}) = 0.289$$

$$\text{Gain}(D, \text{触感}) = 0.006$$

□ 显然，属性“纹理”的信息增益最大，其被选为划分属性



决策树 - 信息增益(information gain)

决策树构造实例 - 西瓜数据集

$$\text{Gain}(D, \text{色泽}) = 0.043$$

$$\text{Gain}(D, \text{根蒂}) = 0.458$$

$$\text{Gain}(D, \text{敲声}) = 0.331$$

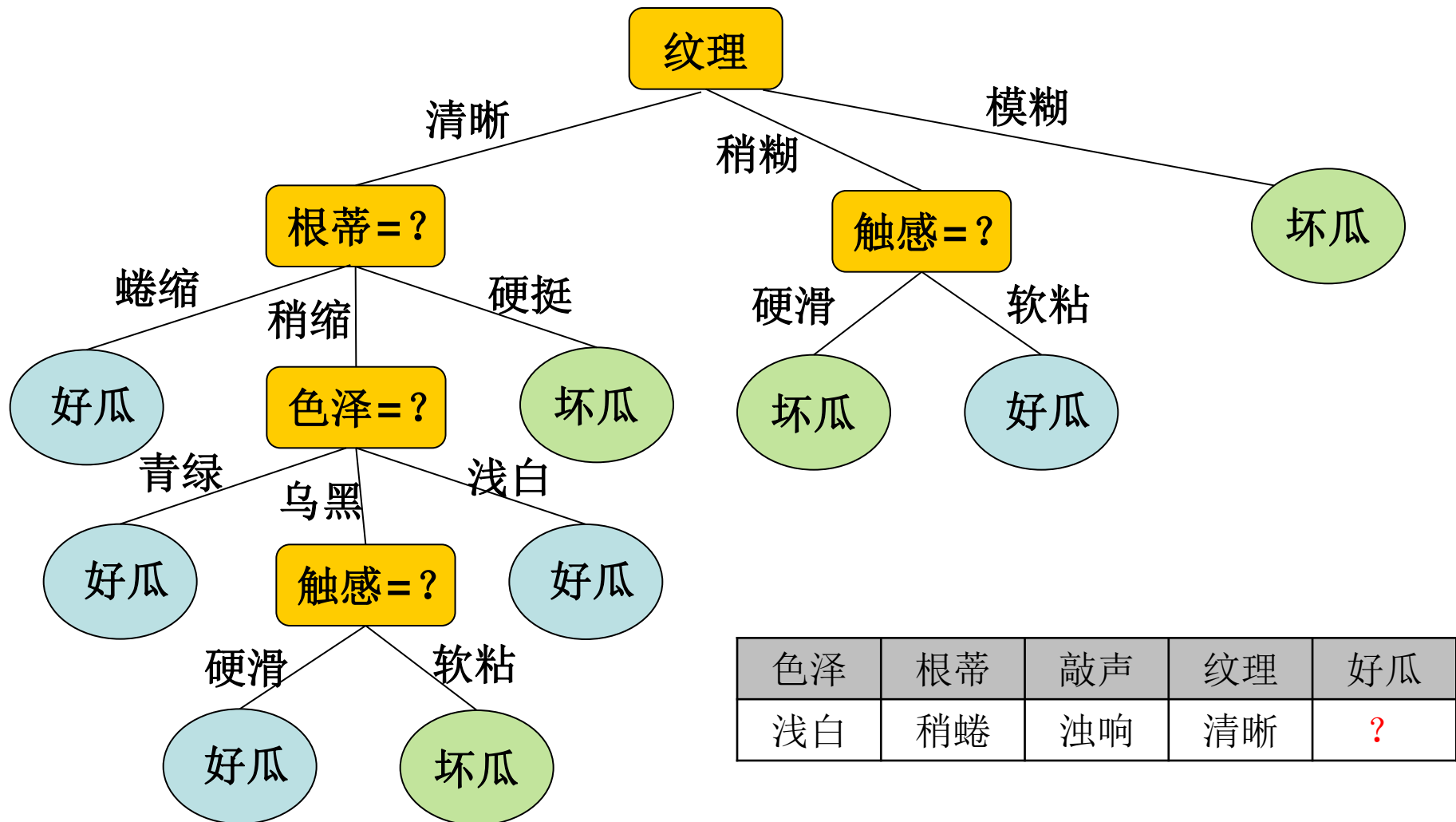
$$\text{Gain}(D, \text{脐部}) = 0.458$$

$$\text{Gain}(D, \text{触感}) = 0.458$$

编号	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
1	青绿	蜷缩	浊响	清晰	凹陷	硬滑	是
2	乌黑	蜷缩	沉闷	清晰	凹陷	硬滑	是
3	乌黑	蜷缩	浊响	清晰	凹陷	硬滑	是
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	是
5	浅白	蜷缩	浊响	清晰	凹陷	硬滑	是
6	青绿	稍蜷	浊响	清晰	稍凹	软粘	是
7	乌黑	稍蜷	浊响	稍糊	稍凹	软粘	是
8	乌黑	稍蜷	浊响	清晰	稍凹	硬滑	是
9	乌黑	稍蜷	沉闷	稍糊	稍凹	硬滑	否
10	青绿	硬挺	清脆	清晰	平坦	软粘	否
11	浅白	硬挺	清脆	模糊	平坦	硬滑	否
12	浅白	蜷缩	浊响	模糊	平坦	软粘	否
13	青绿	稍蜷	浊响	稍糊	凹陷	硬滑	否
14	浅白	稍蜷	沉闷	稍糊	凹陷	硬滑	否
15	乌黑	稍蜷	浊响	清晰	稍凹	软粘	否
16	浅白	蜷缩	浊响	模糊	平坦	硬滑	否
17	青绿	蜷缩	沉闷	稍糊	稍凹	硬滑	否

决策树 - 信息增益(information gain)

决策树构造实例 - 西瓜数据集



色泽	根蒂	敲声	纹理	好瓜
浅白	稍蜷	浊响	清晰	?

决策树 – 增益率 (Gain Ratio)

C4.5算法中使用

- ✓ 信息增益存在的问题：偏向取值较多的特征
- ✓ 原因：当特征取值较多时，根据此特征划分更容易得到纯度更高的子集，因此划分后的熵更低，信息增益更大

不具有泛化能力，无法对新样本进行有效预测

例如：考虑将“编号”作为一个属性

□ 离散属性 $\mathbf{a}$ 的取值 $\{a^1, a^2, \dots, a^V\}$

□ D^v ：样本集合 $\mathbf{D}$ 中在 $\mathbf{a}$ 上取值 $= a^v$ 的样本集合

信息增益率：
$$\text{Gain_ratio}(D, a) = \frac{\text{Gain}(D, a)}{\text{IV}(a)}$$

$$\text{IV}(a) = - \sum_{v=1}^V \frac{|D^v|}{|D|} \log_2 \frac{|D^v|}{|D|}$$

决策树 – 基尼指数(Gini Index)

CART算法中使用

$$\text{Gini}(D) = \sum_{k=1}^{|\mathcal{Y}|} \sum_{k' \neq k} p_k p_{k'}$$

反映了从 D 中随机抽取两个样例，其类别标记不一致的概率

$$= 1 - \sum_{k=1}^{|\mathcal{Y}|} p_k^2$$

$\text{Gini}(D)$ 越小，数据集 D 的纯度越高

两个是同一类的概率

属性 a 的基尼指数:

$$\text{Gini_index}(D, a) = \sum_{v=1}^V \frac{|D^v|}{|D|} \text{Gini}(D^v)$$

在候选属性集合中，选取那个使划分后基尼指数最小的属性

决策树 – 划分选择 vs. 剪枝

研究表明：划分选择的各种准则虽然对决策树的尺寸有较大影响，但对泛化性能的影响很有限

例如信息增益与基尼指数产生的结果，仅在约 2% 的情况下不同

剪枝方法和程度对决策树泛化性能的影响更为显著

在数据带噪时甚至可能将泛化性能提升 25%

剪枝 (pruning) 是决策树对付“过拟合”的主要手段！



什么是过拟合？

泛化误差 vs. 经验误差

泛化误差：在“未来”样本上的误差

经验误差：在训练集上的误差，亦称“训练误差”

- 泛化误差越小越好
- 经验误差是否越小越好？

NO! 因为会出现“过拟合” (overfitting)

过拟合 (overfitting) vs. 欠拟合 (underfitting)

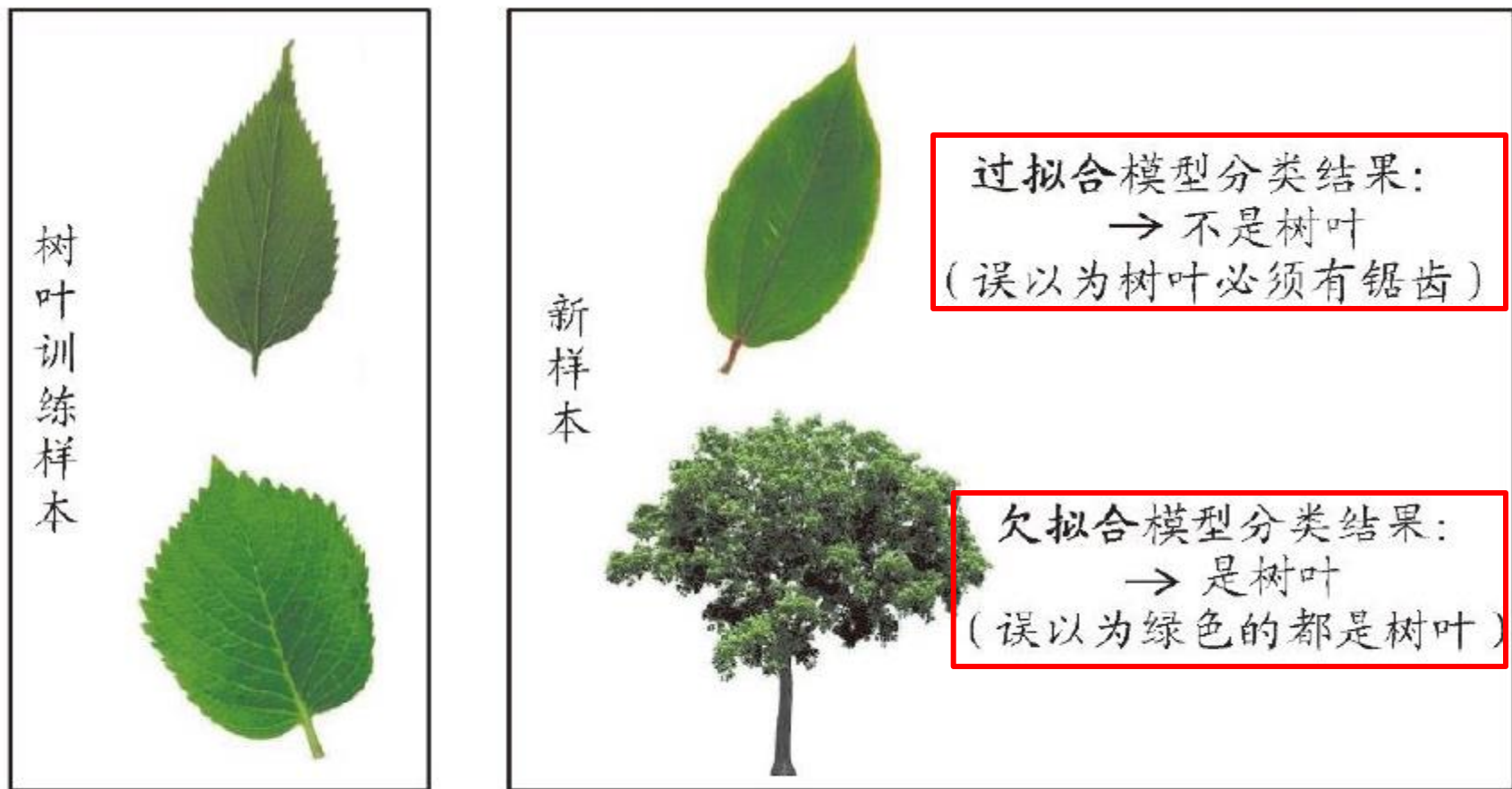


图 2.1 过拟合、欠拟合的直观类比

决策树 – 剪枝

为了尽可能正确分类训练样本，有可能造成分支过多 → 过拟合
可通过主动去掉一些分支来降低过拟合的风险

决策树剪枝基本策略：

- 预剪枝 (pre-pruning): 提前终止某些分支的生长
- 后剪枝 (post-pruning): 生成一棵完全树，再“回头”剪枝

剪枝过程中需评估剪枝前后决策树的优劣

现在我们假定使用“留出法”

决策树 – 剪枝

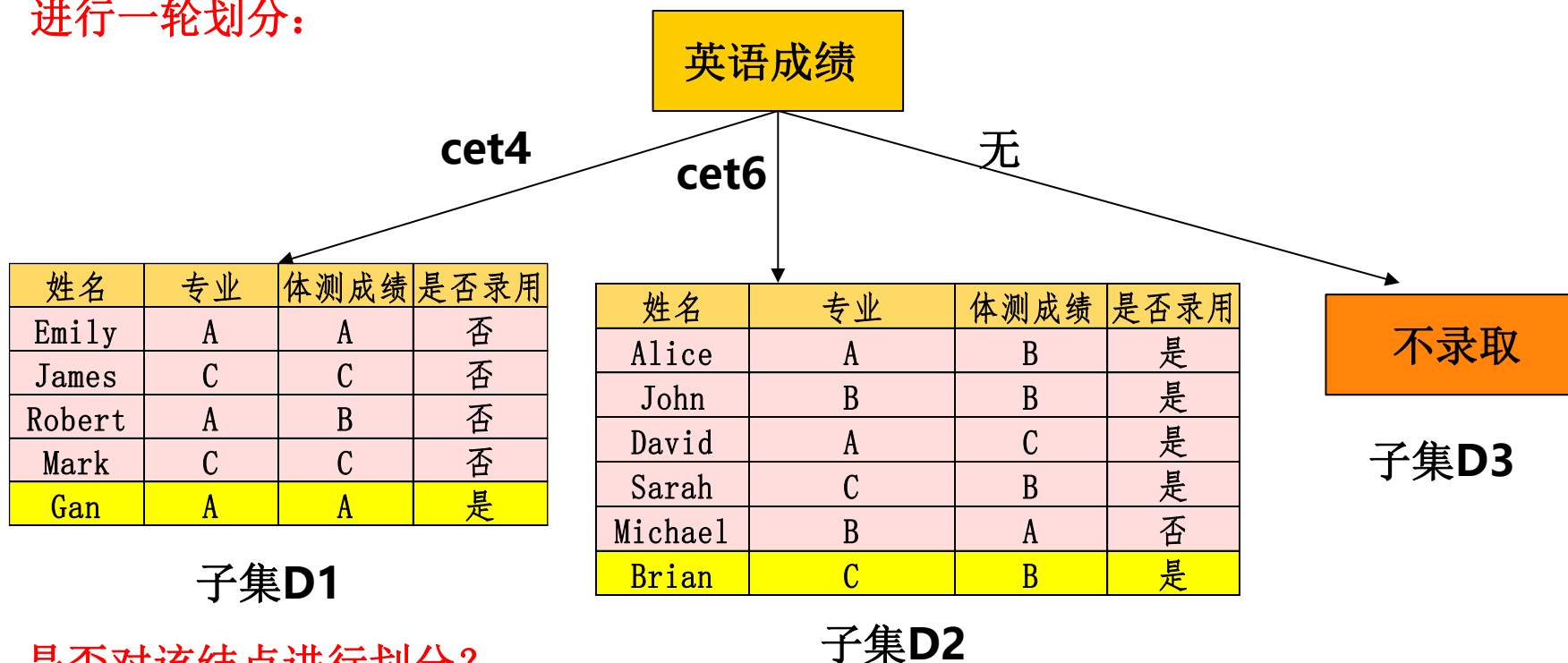
一个例子

训练集	编号	姓名	专业	英语	体测成绩	是否录用
	1	Alice	A	cet6	B	是
验证集	2	John	B	cet6	B	是
	3	Emily	A	cet4	A	否
	4	Mark	C	cet4	C	否
	...	...	...	...	...	...
	14	David	A	cet6	C	是
	15	Joy	B	cet4	A	否
	16	Bar	A	cet4	C	否
	17	Judy	C	cet4	B	否

决策树 – 预剪枝

- 预剪枝 (pre-pruning): 提前终止某些分支的生长
- 使用决策树的验证集, 通过决策树结点划分是否能提高验证集精度来确认是否划分, 若划分不能提高验证集精度, 则将其划为叶结点

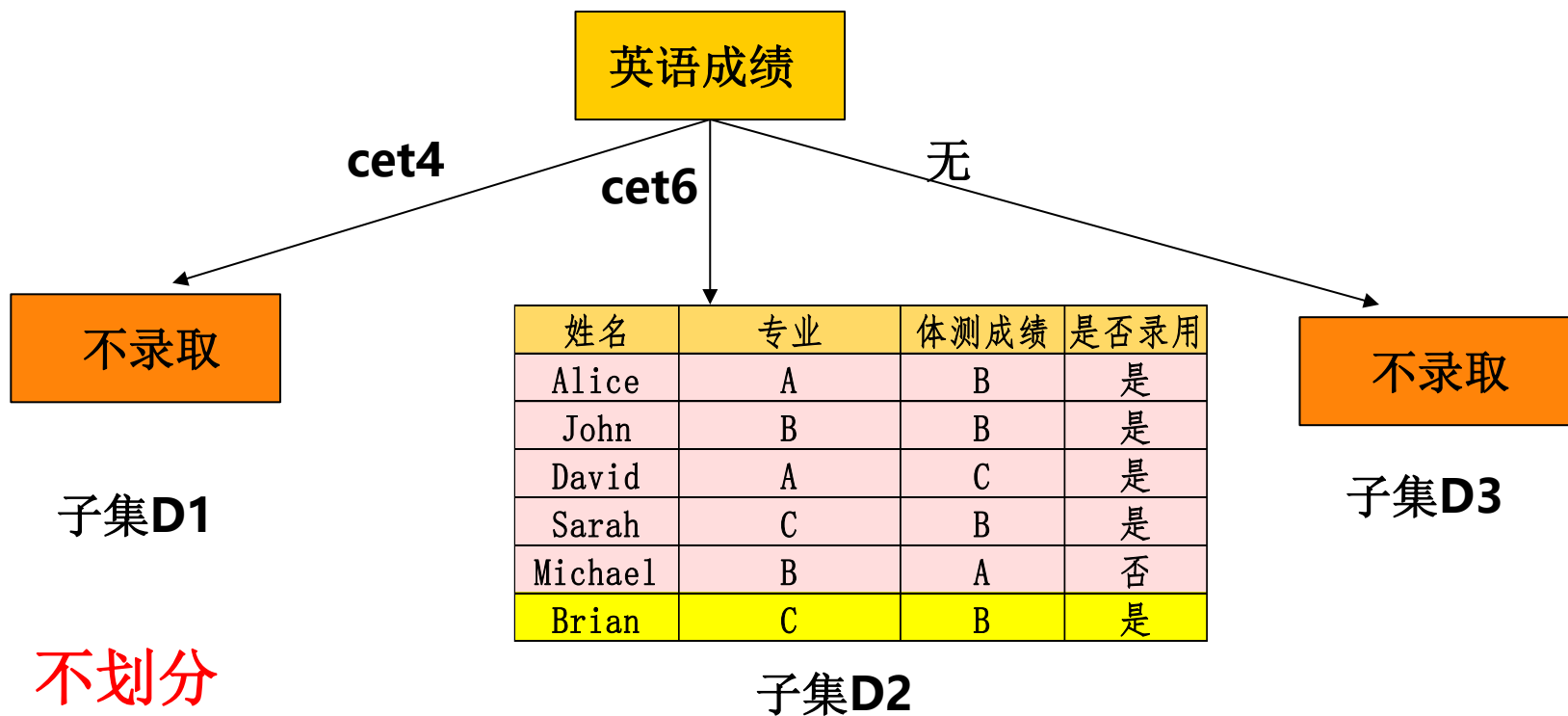
通过“英语成绩”属性
进行一轮划分:



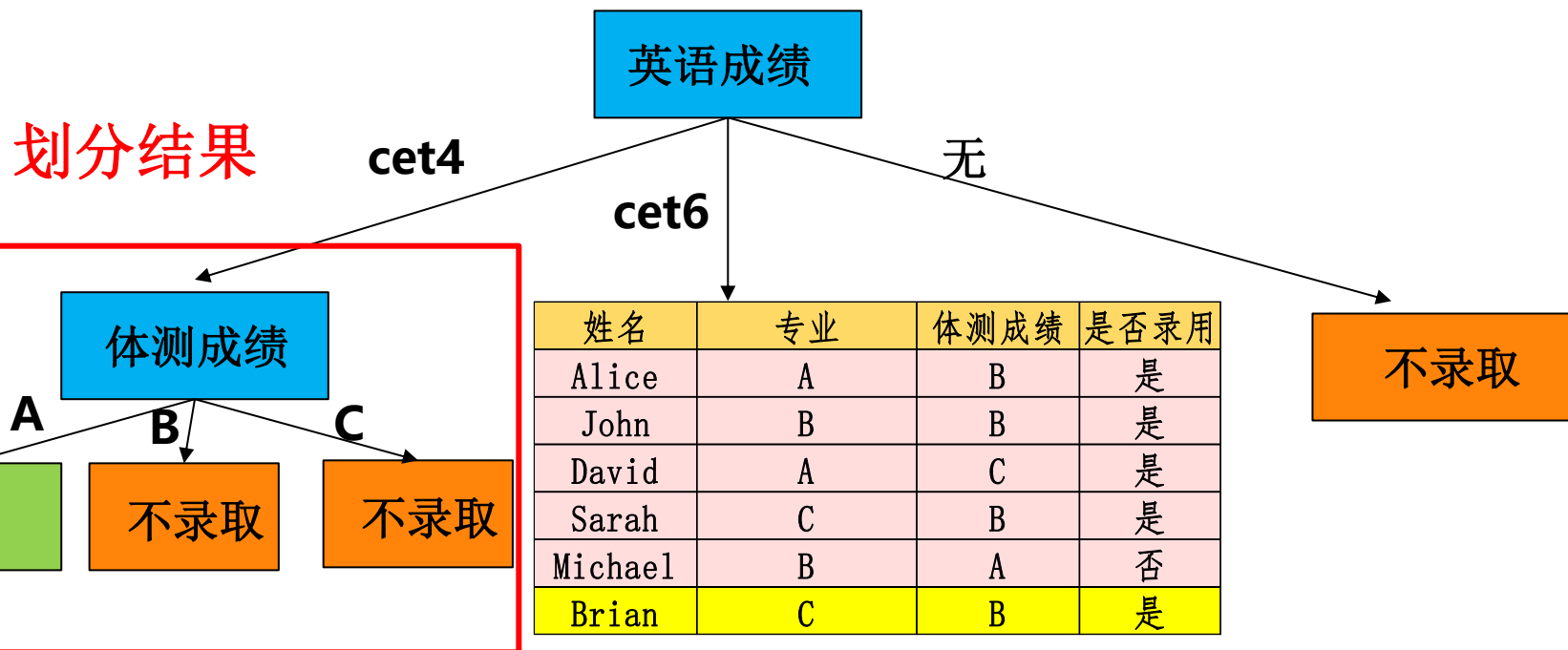
是否对该结点进行划分?

决策树 – 预剪枝

- 预剪枝 (pre-pruning): 提前终止某些分支的生长
- 使用决策树的验证集, 通过决策树结点划分是否能提高验证集精度来确认是否划分, 若划分不能提高验证集精度, 则将其划为叶结点



决策树 – 预剪枝



这个划分有没有必要呢？

验证集

编号	姓名	专业	英语	体测成绩	是否录用
15	Joy	B	cet4	A	否
16	Bar	A	cet4	C	否
17	Judy	C	cet4	B	否

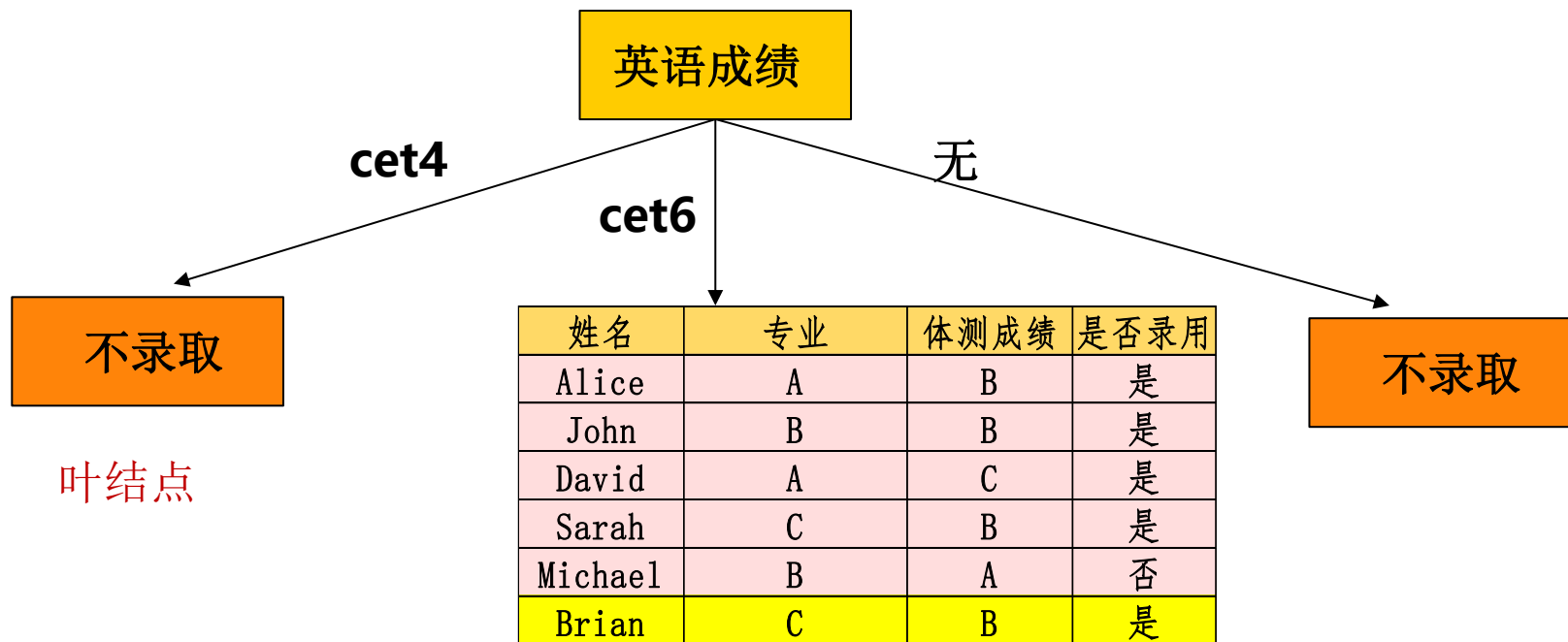
不划分：验证集精度**100%**

划分：验证集精度**66.7%**

划分后验证集精度下降，
则不划分

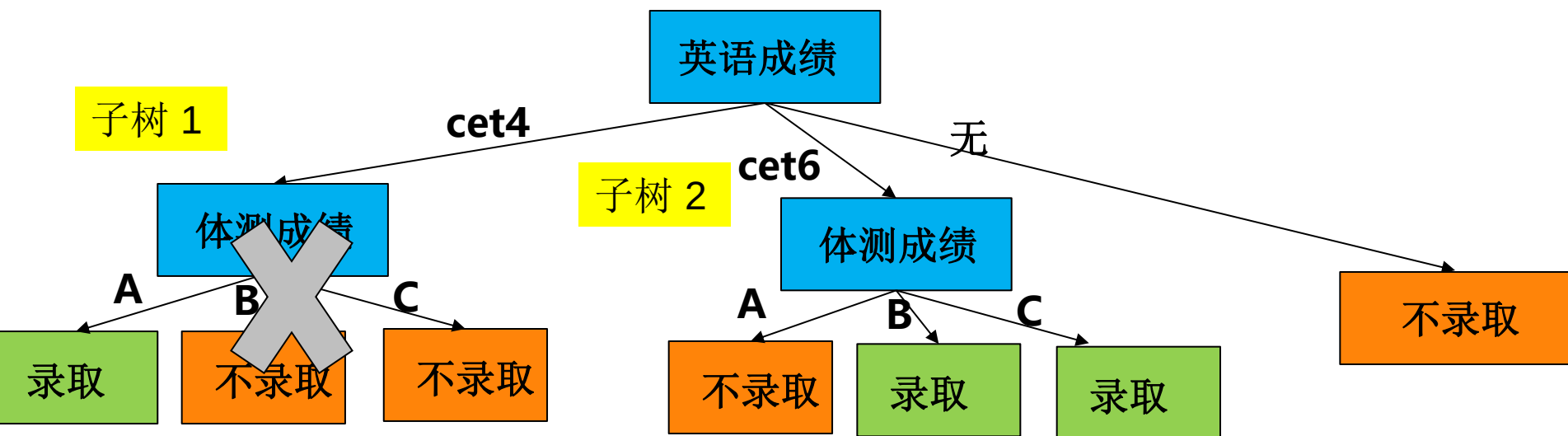
决策树 – 预剪枝

- 剪枝后的决策树



决策树 – 后剪枝

- 后剪枝 (post-pruning): 生成一棵完全树, 再“回头”剪枝
- 生成一颗完整的决策树, 再自底向上对非叶结点进行考察, 若该结点对应的子树变成叶结点可提高性能, 则将其划为叶节点



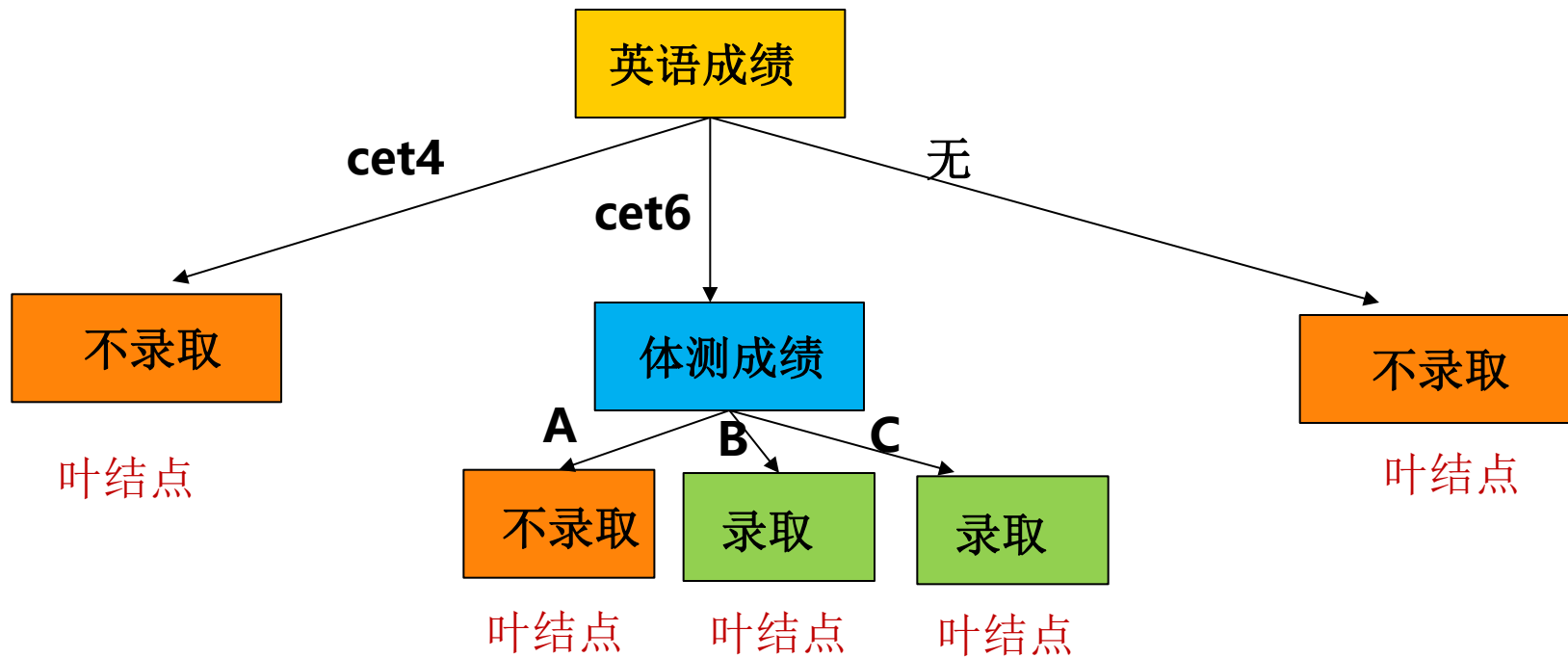
编号	姓名	专业	英语	体测成绩	是否录用
15	Joy	B	cet4	A	否
16	Bar	A	cet4	C	否
17	Judy	C	cet4	B	否
18	Cool	A	cet6	A	否
19	Hot	B	cet6	B	是
20	Fire	C	cet6	C	是

子树1: 划分后66.7%, 不划分100%, 不划分

子树2: 划分后100%, 不划分66.7%, 划分

决策树 – 后剪枝

- 剪枝后的决策树



第4章 支持向量机

主要内容

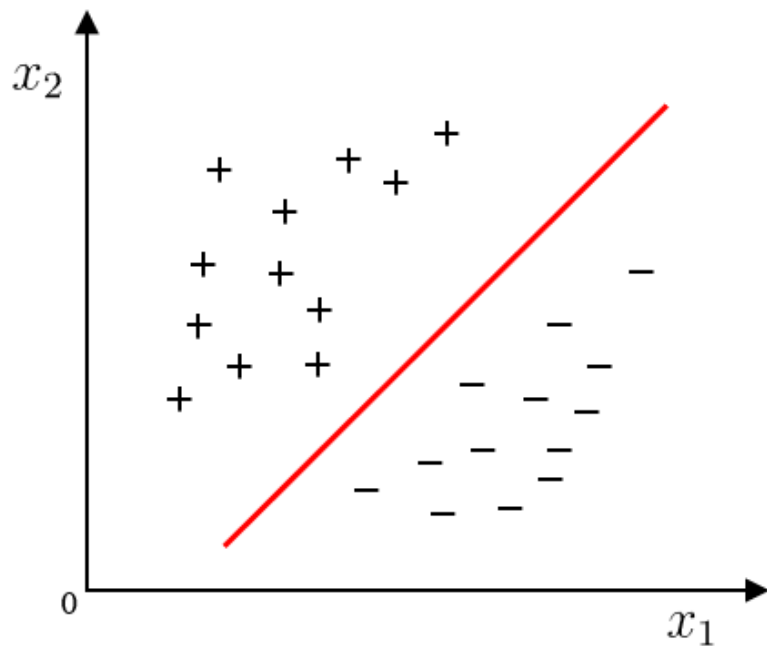
- 间隔与支持向量
- 对偶问题
- 软间隔优化
- 核函数

支持向量机 (Support Vector Machine)

- 本章要讲解的模型是大名鼎鼎的支持向量机 SVM，这是曾经在机器学习界有着近乎垄断地位的模型，影响力持续多年。直至今日，即使深度学习神经网络的影响力逐渐增强，但 SVM 在中小型数据集上依旧有着可以和神经网络抗衡的极好效果和模型鲁棒性。

支持向量机 (Support Vector Machine)

线性模型：在样本空间中寻找一个超平面，将不同类别的样本分开

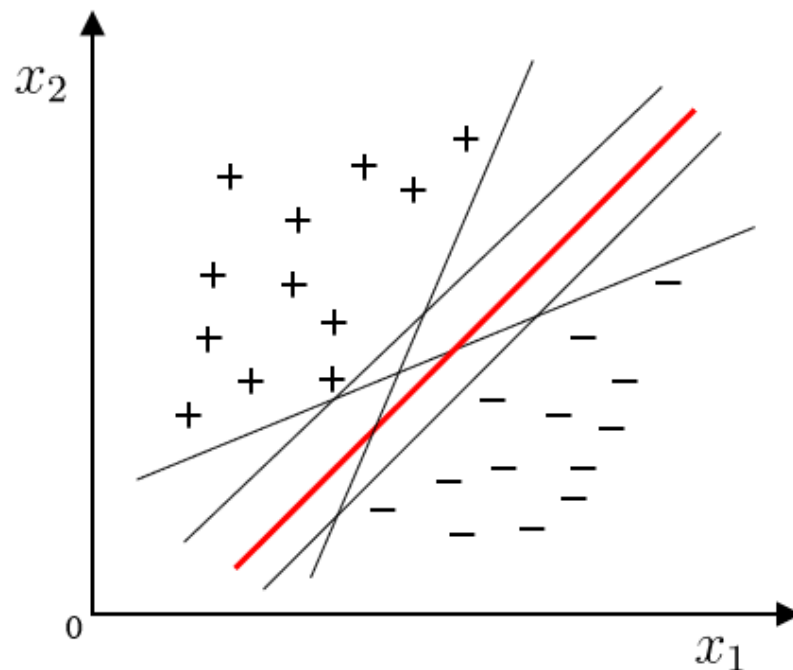
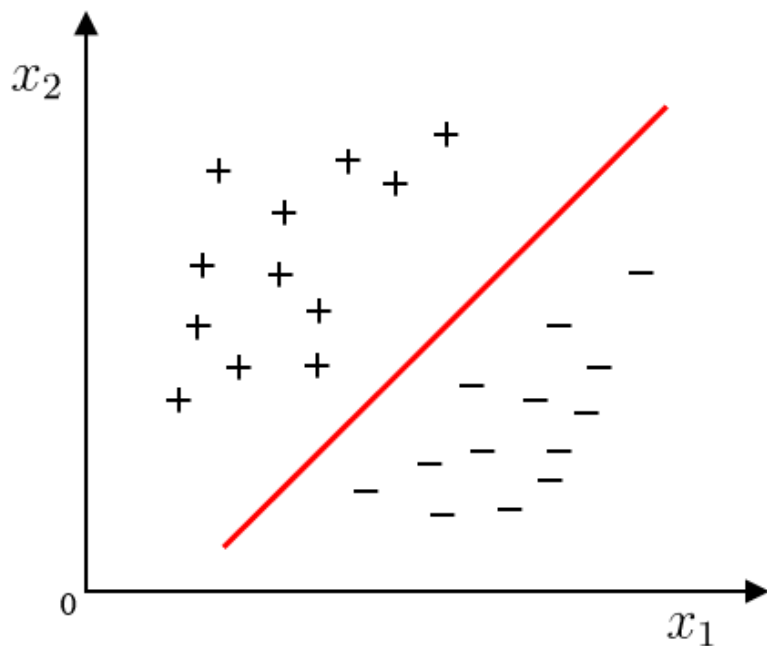


应用场景	类型	结果类别
垃圾邮件识别	二分类	垃圾邮件 正常邮件
图像内容识别	多分类	猫 狗 人 都不是
文本情感分析	二分类	褒 贬

支持向量机 (Support Vector Machine)

➤ 要解决的问题是什么：什么样的决策边界才是最好的？

线性模型：在样本空间中寻找一个超平面，将不同类别的样本分开



直觉：正中间的鲁棒性最好，泛化能力最强

支持向量机 (Support Vector Machine)

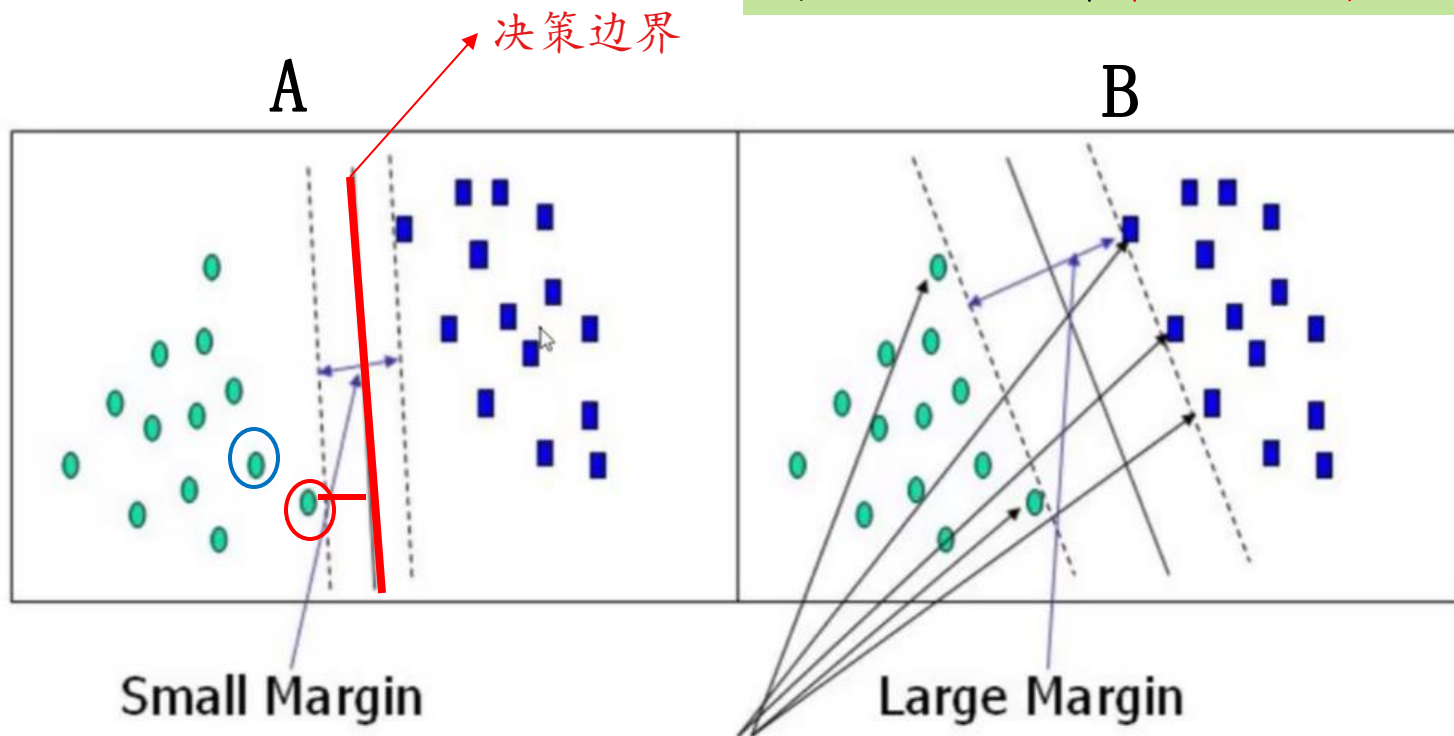
- 要解决的问题是什么：什么样的决策边界才是最好的？
- 特征数据本身很难分，怎么办？
- 计算复杂度怎么样，能实际应用吗？
- 目标：基于上述问题对SVM进行推导

支持向量机 (Support Vector Machine)

例子

- 决策边界：选出来离雷区最远的（雷区就是边界上的点，要Large Margin）

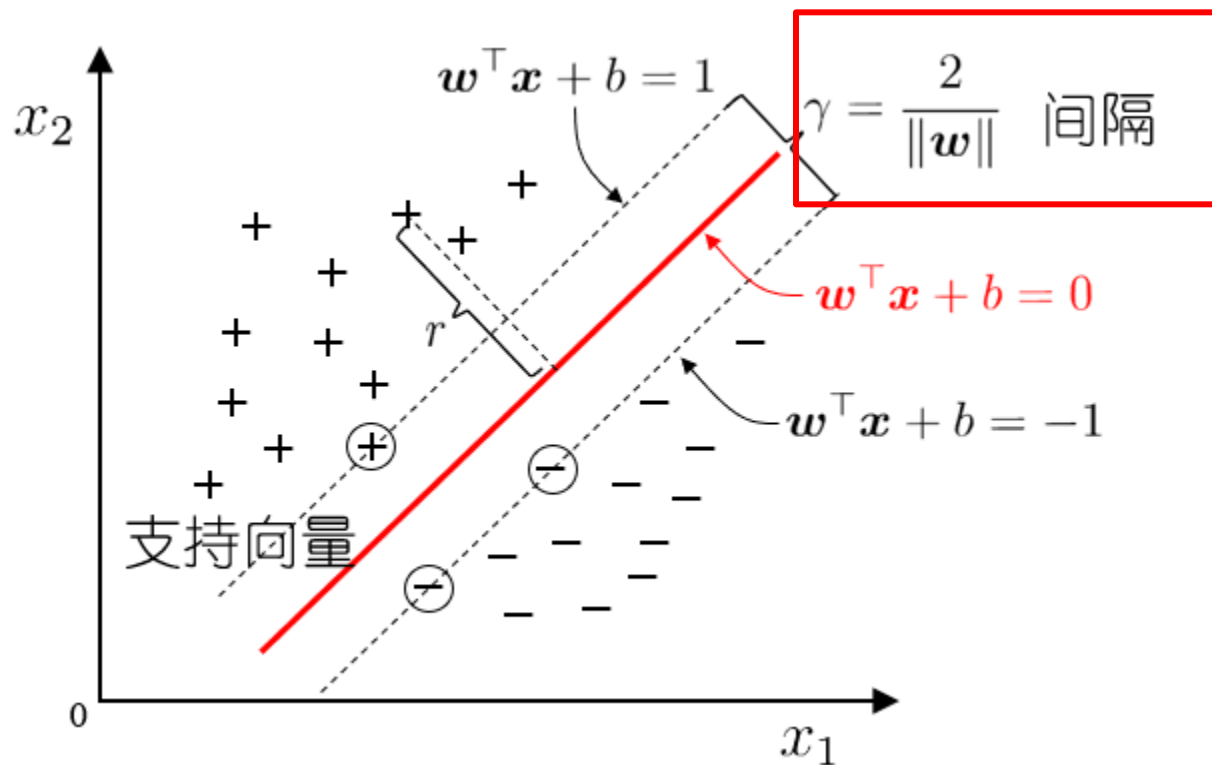
目标：决策边界到离它最近的那个点距离越大越好(即**最大化决策边界的间隔**)



支持向量机 (Support Vector Machine)

□ 间隔与支持向量

超平面方程: $w^\top x + b = 0$



支持向量机 (Support Vector Machine)

□ 回顾：点到超平面的距离

对于一个给定的超平面方程 $w \cdot X + b = 0$ ，其中 w 是法向量， b 是偏移量，点 X 到这个超平面的距离 r 可以用以下公式计算：

$$r = \frac{|w \cdot X + b|}{\|w\|}$$

这里， $w \cdot X + b$ 表示点 X 到超平面的有符号距离，而 $\|w\|$ 是法向量的范数（也就是长度），绝对值确保距离为非负值。

假设我们有一个二维空间中的点 $P(2, 3)$ 和一个直线方程 $x + 2y - 4 = 0$ 。我们想要计算点 P 到这条直线的距离。

支持向量机 (Support Vector Machine)

➤ 数据标签定义

✓ 数据集: $(x_1, y_1), (x_2, y_2) \cdots (x_n, y_n)$

✓ y 为样本的类别: 当 x 为正例时 $y = +1$, 当 x 为负例时 $y = -1$

✓ 决策方程: $y(x) = w^T \Phi(x) + b$

$$\begin{aligned} \Rightarrow y(x_i) > 0 &\Leftrightarrow y_i = +1 \\ y(x_i) < 0 &\Leftrightarrow y_i = -1 \end{aligned} \quad \Rightarrow y_i \cdot y(x_i) > 0$$

支持向量机 (Support Vector Machine)

➤ 优化目标

✓ 最大化决策边界的间隔

✓ 通俗解释: 找到一条线 (二维空间), 使得离该线最近的点 (雷点) 能够最远

由于 $y_i \cdot y(x_i) > 0$, 所以将绝对值展开依旧成立

$$r = \frac{|w \cdot X + b|}{\|w\|}$$

✓ 点到直线的距离化简得到: $\frac{y_i \cdot (w^T \cdot \Phi(x_i) + b)}{\|w\|}$

支持向量机 (Support Vector Machine)

➤ 目标函数

$$\arg \max_{w,b} \left\{ \frac{1}{\|w\|} \min_i \left[y_i \cdot (w^T \cdot \Phi(x_i) + b) \right] \right\}$$

支持向量机 (Support Vector Machine)

➤ 目标函数

$$\arg \max_{w,b} \left\{ \frac{1}{\|w\|} \min_i [y_i \cdot (w^T \cdot \Phi(x_i) + b)] \right\}$$

✓ 缩放变换：对于决策方程 (w, b) 可以通过缩放使得其结果值 $|y| \geq 1$

$$\Rightarrow y_i \cdot (w^T \cdot \Phi(x_i) + b) \geq 1 \quad (\text{之前认为恒大于0, 现在严格了一些})$$

➤ 化简目标函数

$$\max_{w,b} \frac{1}{\|w\|}, \text{ 约束条件: } y_i (w^T \cdot \Phi(x_i) + b) \geq 1$$

支持向量机 (Support Vector Machine)

➤ 目标函数

$$\max_{w,b} \frac{1}{\|w\|}, \text{ 约束条件: } y_i(w^T \cdot \Phi(x_i) + b) \geq 1$$

✓ 常规套路：把求极大值问题转化为极小值问题

$$\arg \min_{w,b} \frac{1}{2} \|w\|^2$$

✓ 如何求解/优化：应用拉格朗日乘子法

支持向量机 (Support Vector Machine)

➤ 拉格朗日乘子法

带约束的优化问题：

$$\min_x f_0(x)$$

subject to $f_i(x) \leq 0, i = 1, \dots, m$
 $h_i(x) = 0, i = 1, \dots, q$

原式转换：

$$\min L(x, \lambda, v) = f_0(x) + \sum_{i=1}^m \lambda_i f_i(x) + \sum_{i=1}^q v_i h_i(x)$$

我们的式子：

$$L(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_{i=1}^n \alpha_i (y_i (w^T \cdot \Phi(x_i) + b) - 1)$$

约束条件： $y_i (w^T \cdot \Phi(x_i) + b) \geq 1$

支持向量机 (Support Vector Machine)

➤ SVM求解

分别对 w 和 b 求偏导,分别得到两个条件 (由于对偶性质)

$$\min_{w,b} \max_{\alpha} L(w,b,\alpha) \rightarrow \max_{\alpha} \min_{w,b} L(w,b,\alpha)$$

$$L(w,b,\alpha) = \frac{1}{2} \|w\|^2 - \sum_{i=1}^n \alpha_i (y_i (w^T \cdot \Phi(x_i) + b) - 1)$$

$$\text{对 } w \text{ 求偏导: } \frac{\partial L}{\partial w} = 0 \Rightarrow w = \sum_{i=1}^n \alpha_i y_i \Phi(x_i)$$

$$\text{对 } b \text{ 求偏导: } \frac{\partial L}{\partial b} = 0 \Rightarrow 0 = \sum_{i=1}^n \alpha_i y_i$$

支持向量机 (Support Vector Machine)

➤ SVM求解

带入原始： $L(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_{i=1}^n \alpha_i (y_i (w^T \Phi(x_i) + b) - 1)$

其中 $w = \sum_{i=1}^n \alpha_i y_i \Phi(x_i)$ $0 = \sum_{i=1}^n \alpha_i y_i$

$$= \frac{1}{2} w^T w - w^T \sum_{i=1}^n \alpha_i y_i \Phi(x_i) - b \sum_{i=1}^n \alpha_i y_i + \sum_{i=1}^n \alpha_i$$

$$= \sum_{i=1}^n \alpha_i - \frac{1}{2} \left(\sum_{i=1}^n \alpha_i y_i \Phi(x_i) \right)^T \sum_{i=1}^n \alpha_i y_i \Phi(x_i) \quad \longrightarrow \text{完成了第一步求解 } \min_{w, b} L(w, b, \alpha)$$

$$= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1, j=1}^n \alpha_i \alpha_j y_i y_j \Phi^T(x_i) \Phi(x_j)$$

支持向量机 (Support Vector Machine)

➤ SVM求解

继续对 α 求极大值：
$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (\Phi(x_i) \cdot \Phi(x_j))$$

条件：
$$\sum_{i=1}^n \alpha_i y_i = 0$$
$$\alpha_i \geq 0$$

极大值转换成求极小值：
$$\min_{\alpha} \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (\Phi(x_i) \cdot \Phi(x_j)) - \sum_{i=1}^n \alpha_i$$

条件：
$$\sum_{i=1}^n \alpha_i y_i = 0$$
$$\alpha_i \geq 0$$

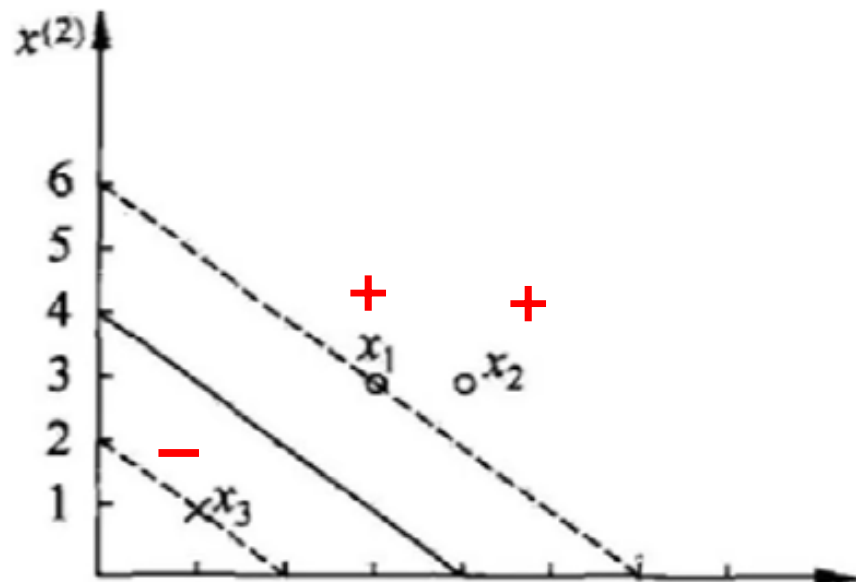
支持向量机 (Support Vector Machine)

➤ SVM求解实例

✓ 数据：3个点，其中正例 $x_1(3, 3)$, $x_2(4, 3)$ ，负例 $x_3(1, 1)$

$$\text{求解：} \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \sum_{i=1}^n \alpha_i$$

$$\begin{aligned} \text{约束条件：} \quad & \alpha_1 + \alpha_2 - \alpha_3 = 0 \\ & \alpha_i \geq 0, \quad i = 1, 2, 3 \end{aligned}$$



$$\text{极大值转换成求极小值：} \min_{\alpha} \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (\Phi(x_i) \cdot \Phi(x_j)) - \sum_{i=1}^n \alpha_i$$

$$\begin{aligned} \text{条件：} \quad & \sum_{i=1}^n \alpha_i y_i = 0 \\ & \alpha_i \geq 0 \end{aligned}$$

支持向量机 (Support Vector Machine)

➤ SVM求解实例

✓ 数据：3个点，其中正例X1 (3, 3), X2 (4, 3)，负例X3 (1, 1)

$$\text{求解：} \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \sum_{i=1}^n \alpha_i \quad \text{约束条件：} \begin{cases} \alpha_1 + \alpha_2 - \alpha_3 = 0 \\ \alpha_i \geq 0, \quad i = 1, 2, 3 \end{cases}$$

$$\text{原式：} \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \sum_{i=1}^n \alpha_i, \text{ 将数据代入}$$

$$\frac{1}{2} (18\alpha_1^2 + 25\alpha_2^2 + 2\alpha_3^2 + 42\alpha_1\alpha_2 - 12\alpha_1\alpha_3 - 14\alpha_2\alpha_3) - \alpha_1 - \alpha_2 - \alpha_3$$

$$\text{由于：} \alpha_1 + \alpha_2 = \alpha_3 \text{ 化简可得：} 4\alpha_1^2 + \frac{13}{2}\alpha_2^2 + 10\alpha_1\alpha_2 - 2\alpha_1 - 2\alpha_2$$

求它的极小值

支持向量机 (Support Vector Machine)

➤ SVM求解实例

分别对 α_1 和 α_2 求偏导，偏导等于0可得：
 $\alpha_1 = 1.5$
 $\alpha_2 = -1$

(并不满足约束条件 $\alpha_i \geq 0, i=1,2,3$ ，所以解应在边界上)

$\alpha_1 = 0$
 $\alpha_2 = -2/13$  带入原式 = -0.153 (不满足约束)

$\alpha_1 = 0.25$
 $\alpha_2 = 0$  带入原式 = -0.25 (满足啦！)

最小值在(0.25,0,0.25)处取得

支持向量机 (Support Vector Machine)

➤ SVM求解实例

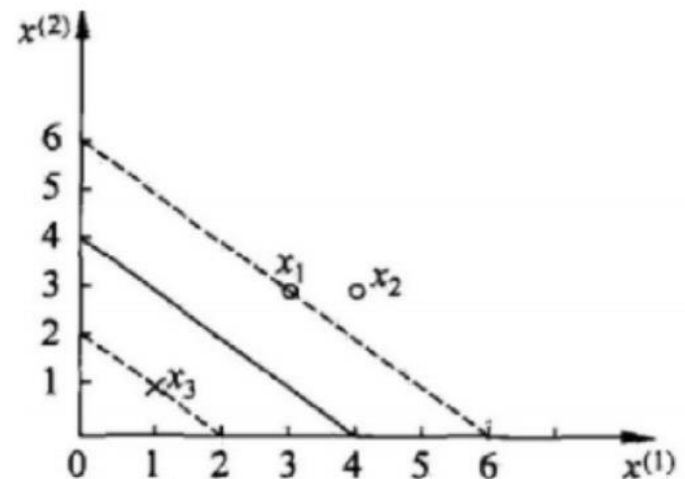
将 α 结果带入求解 $w = \sum_{i=1}^n \alpha_i y_i \Phi(x_n)$

$$w = \frac{1}{4} * 1 * (3,3) + \frac{1}{4} * (-1) * (1,1) = \left(\frac{1}{2}, \frac{1}{2}\right)$$

$$b = y_i - \sum_{i=1}^n \alpha_i y_i (x_i x_j) = 1 - \left(\frac{1}{4} * 1 * 18 + \frac{1}{4} * (-1) * 6\right) = -2$$

平面方程为： $0.5x_1 + 0.5x_2 - 2 = 0$

支持向量机中发挥作用的数据点， α 不为0的真正点



支持向量机 (Support Vector Machine)

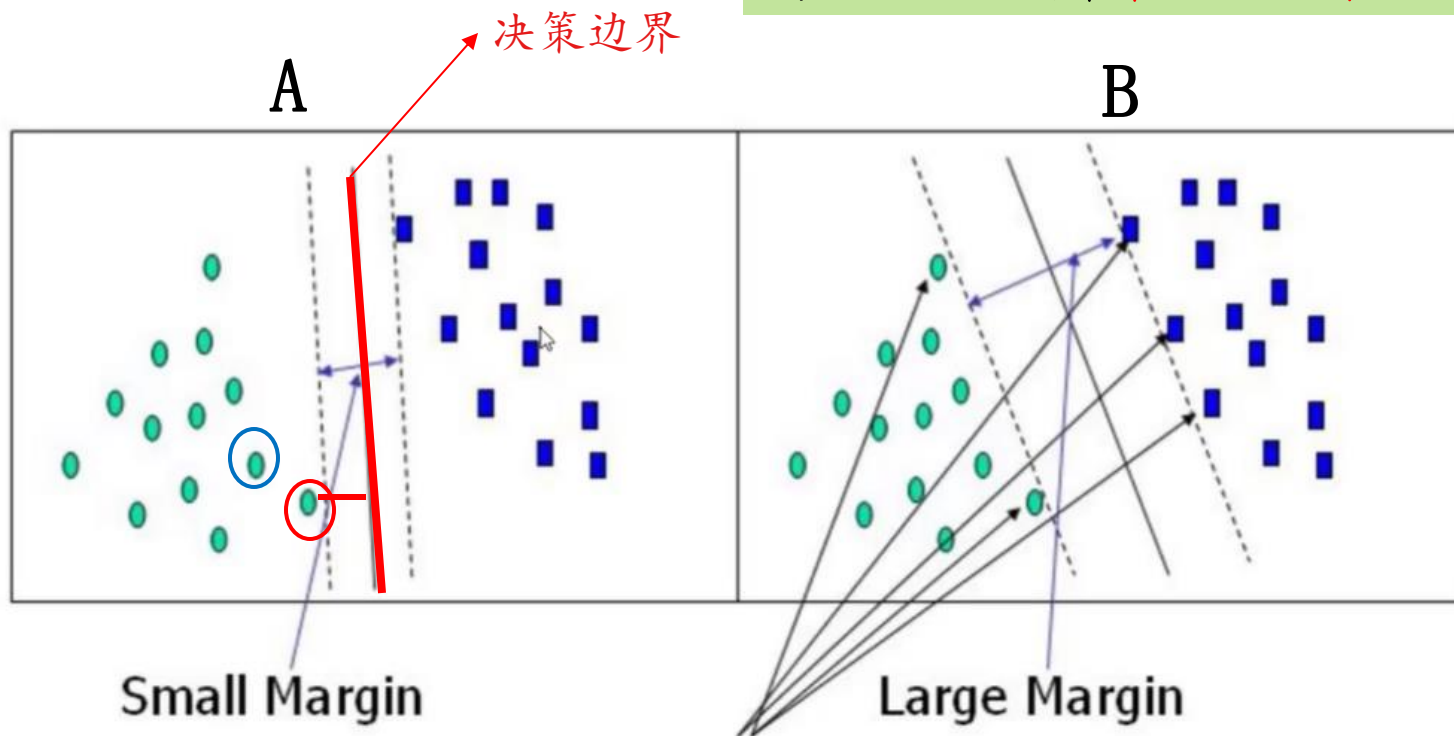
- 要解决的问题是什么：什么样的决策边界才是最好的？
- 特征数据本身很难分，怎么办？
- 计算复杂度怎么样，能实际应用吗？
- 目标：基于上述问题对SVM进行推导

支持向量机 (Support Vector Machine)

例子

- 决策边界：选出来离雷区最远的（雷区就是边界上的点，要Large Margin）

目标：决策边界到离它最近的那个点距离越大越好(即**最大化决策边界的间隔**)



支持向量机 (Support Vector Machine)

➤ 目标函数

$$\arg \max_{w,b} \left\{ \frac{1}{\|w\|} \min_i [y_i \cdot (w^T \cdot \Phi(x_i) + b)] \right\}$$

✓ 缩放变换：对于决策方程 (w, b) 可以通过缩放使得其结果值 $|y| \geq 1$

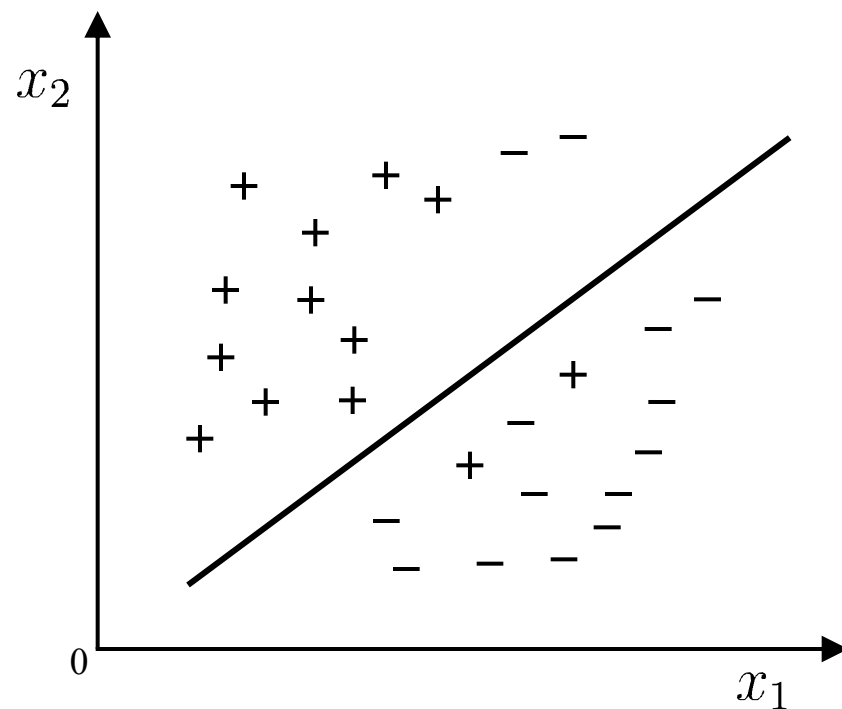
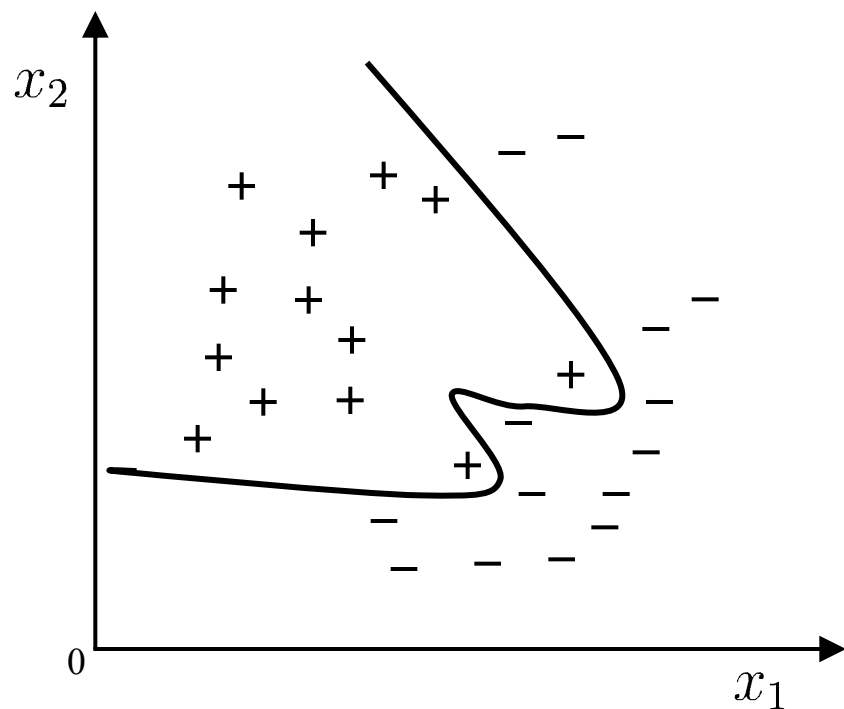
$$\Rightarrow y_i \cdot (w^T \cdot \Phi(x_i) + b) \geq 1 \quad (\text{之前认为恒大于0, 现在严格了一些})$$

➤ 化简目标函数

$$\max_{w,b} \frac{1}{\|w\|}, \text{ 约束条件: } y_i (w^T \cdot \Phi(x_i) + b) \geq 1$$

支持向量机 (Support Vector Machine)

问题: 实际应用中完全线性可分的情况太少见, 如数据中有噪声点



- 用曲线将其完全分开, 即非线性的决策边界 (核函数)
- 使用直线, 但不追求完全可分, 包容分错的情况 (惩罚函数)

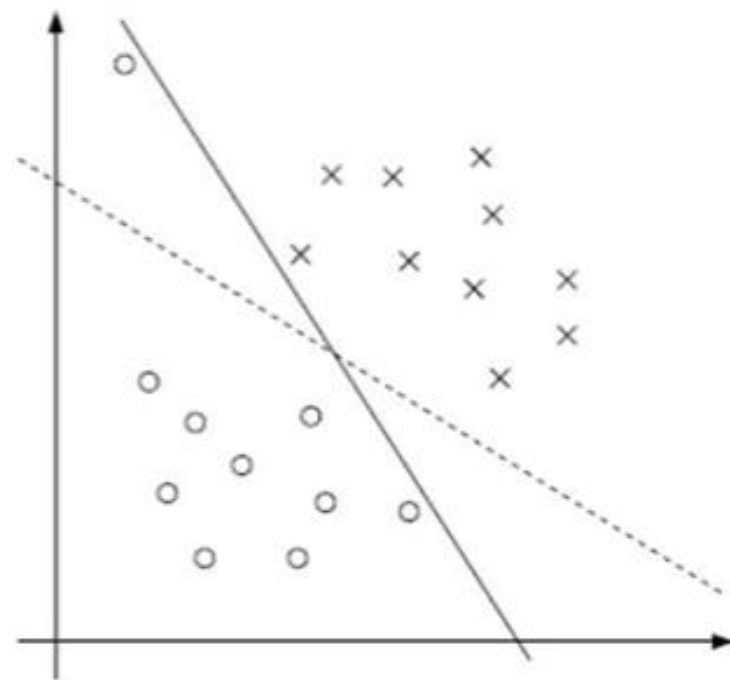
支持向量机 (Support Vector Machine)

➤ 软间隔优化 (soft-margin)

如何让分错的点不要太多太离谱，可以放松一点！

为了解决该问题，引入松弛因子

$$y_i(w \cdot x_i + b) \geq 1 - \xi_i$$



支持向量机 (Support Vector Machine)

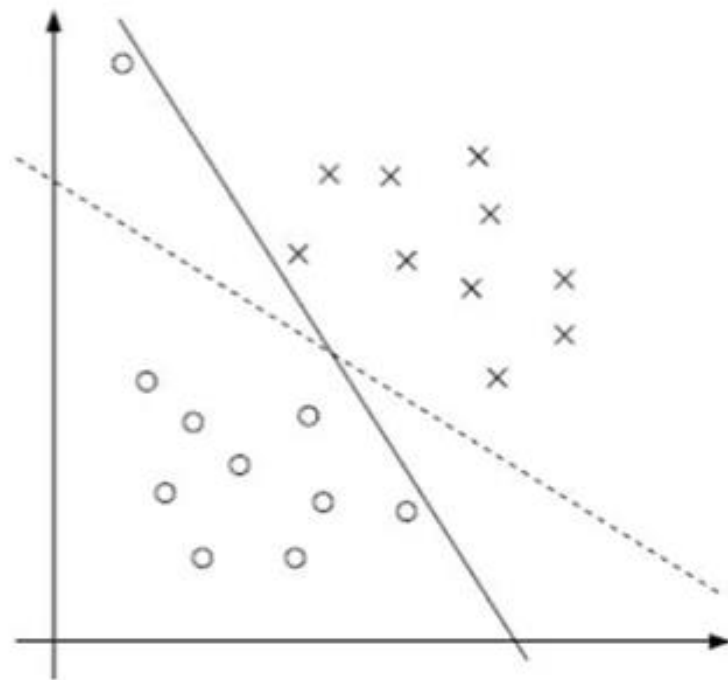
➤ 软间隔优化 (soft-margin)

新的目标函数：

$$\min \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i$$

- 当C很大的时候，严格分类不能有错误
- 当C很小的时候，有更大的错误容忍

C是我们需要指定的一个参数



支持向量机 (Support Vector Machine)

➤ 软间隔优化 (soft-margin)

拉格朗日乘子法：

了解即可！

$$L(w, b, \xi, \alpha, \mu) = \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i - \sum_{i=1}^n \alpha_i (y_i (w \cdot x_i + b) - 1 + \xi_i) - \sum_{i=1}^n \mu_i \xi_i$$

$$w = \sum_{i=1}^n \alpha_i y_i \phi(x_i)$$

$$\min_{\alpha} \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \sum_{i=1}^n \alpha_i$$

约束： $0 = \sum_{i=1}^n \alpha_i y_i$

同样的解法： $\sum_{i=1}^n \alpha_i y_i = 0$

$$C - \alpha_i - \mu_i = 0$$

$$0 \leq \alpha_i \leq C$$

$$\alpha_i \geq 0 \quad \mu_i \geq 0$$

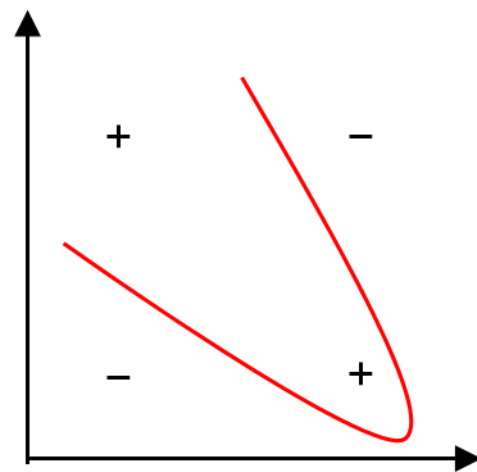
支持向量机 (Support Vector Machine)

➤ 核函数

复杂分类问题，线性不可分：

为了解决该问题，将样本从原始空间映射到一个高维空间，使得样本在这个高维空间内线性可分（即在高维空间中找到最优决策边界），然后再用SVM求解

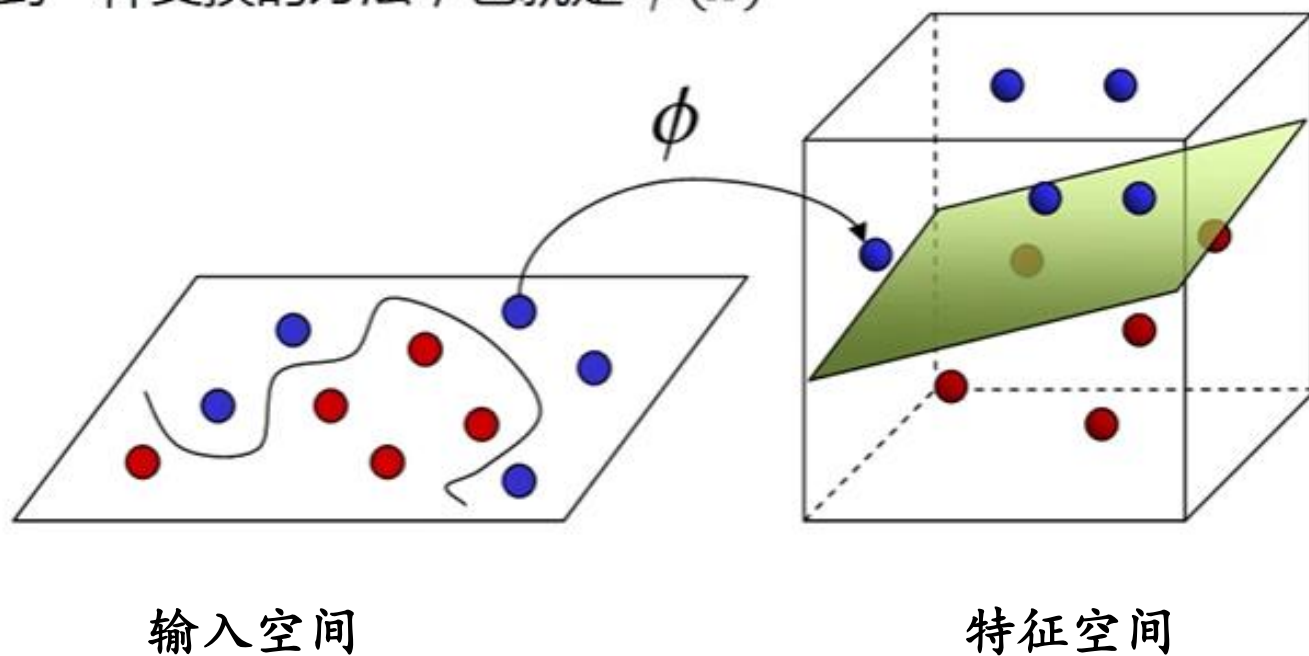
决策方程： $y(x) = w^T \Phi(x) + b$



支持向量机 (Support Vector Machine)

➤ 核函数

目标：找到一种变换的方法，也就是 $\phi(X)$

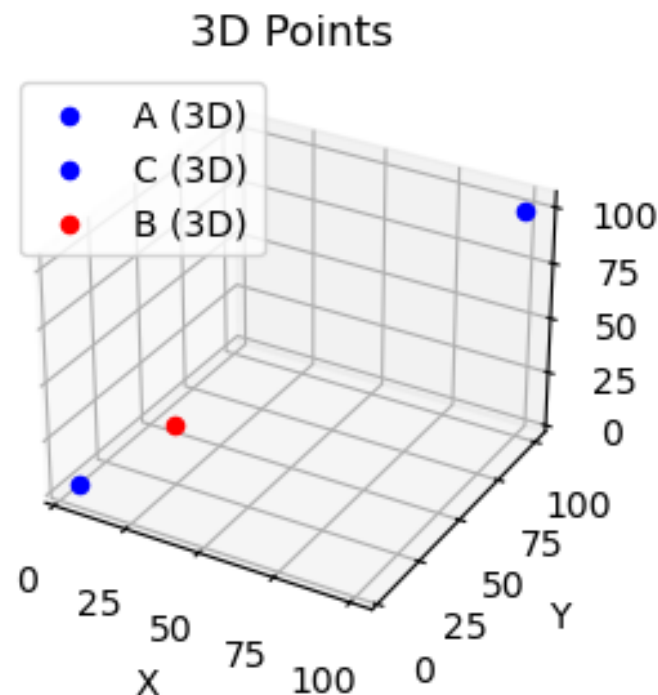
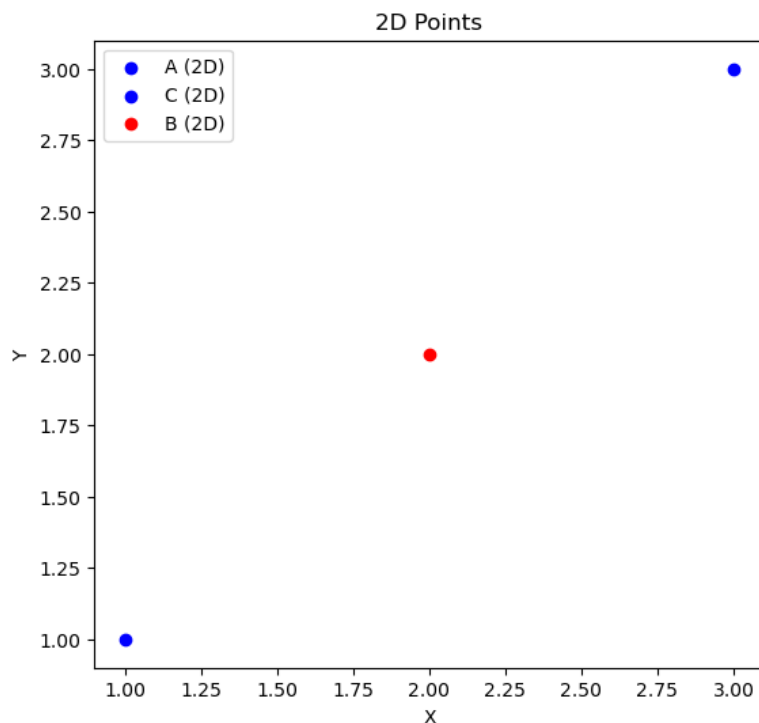


支持向量机 (Support Vector Machine)

➤ 核函数变换例子

二维空间中：正例 A (1, 1)，正例 B (2, 2)，负例 C (3, 3)

使用多项式核函数来将这些点映射到三维空间 $K(x, y) = (x \cdot y + 1)^d$



支持向量机 (Support Vector Machine)

➤ 核函数变换例子

二维空间中：正例 A (1, 1)，正例 B (2, 2)，负例 C (3, 3)

使用多项式核函数来将这些点映射到三维空间 $K(x, y) = (x \cdot y + 1)^d$

1. 计算点积：

- 点 A 的点积： $1 \times 1 = 1$
- 点 B 的点积： $2 \times 2 = 4$
- 点 C 的点积： $3 \times 3 = 9$

2. 应用核函数：

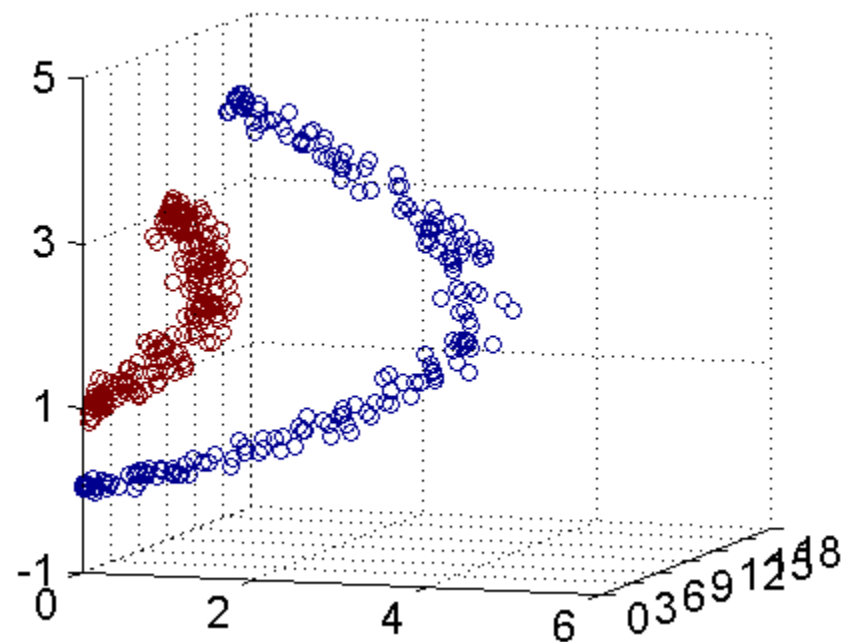
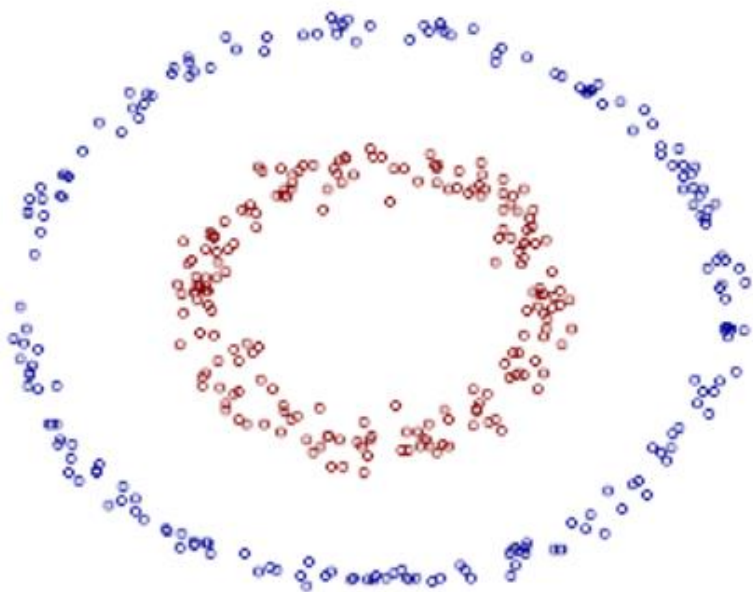
- 点 A 的核函数值： $(1 \times 1 + 1)^2 = 4$
- 点 B 的核函数值： $(4 \times 1 + 1)^2 = 25$
- 点 C 的核函数值： $(9 \times 1 + 1)^2 = 100$

3. 构建特征向量：

- 点 A 映射到三维空间的坐标：(4, 4, 4)
- 点 B 映射到三维空间的坐标：(25, 25, 25)
- 点 C 映射到三维空间的坐标：(100, 100, 100)

支持向量机 (Support Vector Machine)

➤ 核函数变换例子



支持向量机 (Support Vector Machine)

➤ 常用核函数

□ 线性核 (Linear Kernel) $K(x, y) = x \cdot y$

□ 多项式核 (Polynomial Kernel) $K(x, y) = (\gamma x \cdot y + r)^d$

□ 径向基函数 (Radial Basis Function, RBF) 核 $K(x, y) = e^{-\gamma \|x - y\|^2}$

□ Sigmoid核 $K(x, y) = \tanh(\gamma x \cdot y + r)$

第5章 神经网络

什么是神经网络？

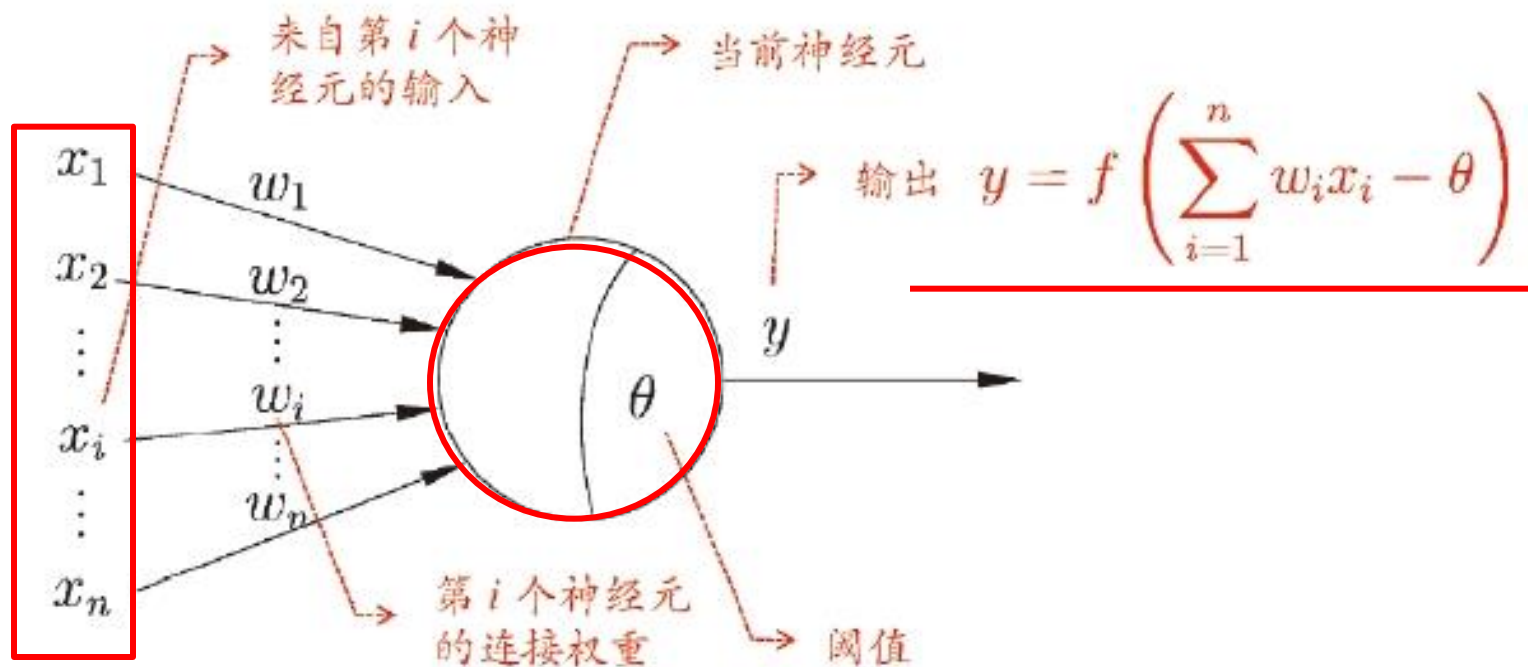
- 神经网络是一个具有适应性的简单单元组成的广泛并行互联的网络，它的组织能够模拟生物神经系统对真实世界物体所作出的交互反应。
[T. Kohonen, NN88]

- 神经网络是一个很大的学科领域，本课程仅讨论它与机器学习的交集，即“神经网络学习”亦称“连接主义”学习

简单单元：神经元模型（最小神经网络）

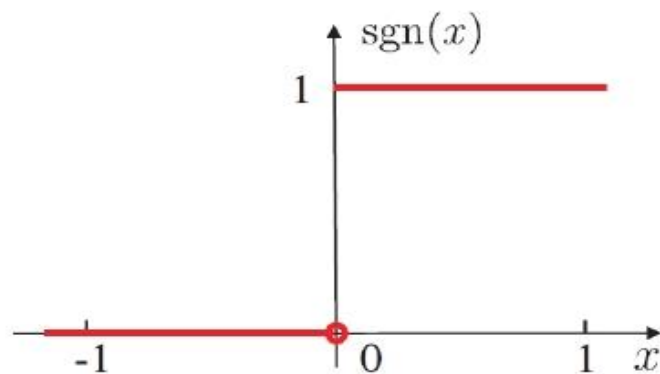
M-P 神经元模型 [McCulloch and Pitts, 1943]

- 神经网络模仿的是人脑中的**神经元**结构
- 在这个模型中，神经元接收到来自n个其它神经元传递过来的输入信号，这些输入信号通过带权重的连接进行传递，神经元接收到的总输入值与神经元的阈值进行比较，然后通过**激活函数**处理以产生神经元的输出
- 神经网络学得的知识蕴含在连接**权重**与**阈值**中



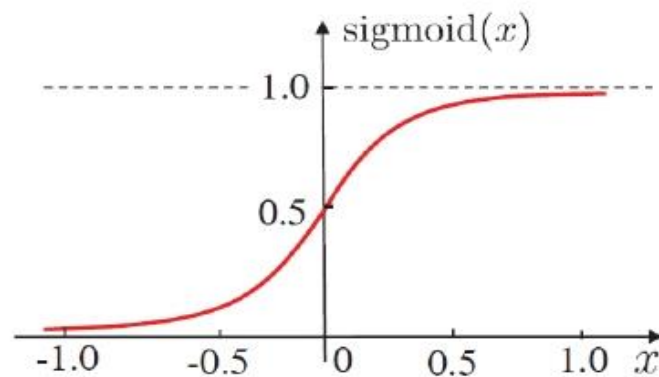
神经元的激活函数

- 理想激活函数是阶跃函数，0表示抑制神经元而1表示激活神经元
- 阶跃函数具有不连续、不光滑等不好的性质，常用的是 Sigmoid 函数，它把可能在较大范围内变化的输入值挤压到(0, 1)输出值范围内，也称为挤压函数
- 激活函数在神经网络中的主要作用是引入非线性，帮助网络学习复杂的函数



$$\text{sgn}(x) = \begin{cases} 1, & \text{if } x \geq 0; \\ 0, & \text{if } x < 0. \end{cases}$$

(a) 阶跃函数



$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

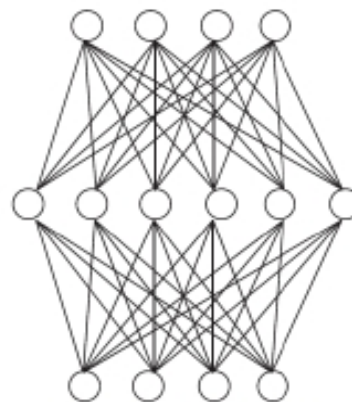
(b) Sigmoid 函数

图 5.2 典型的神经元激活函数

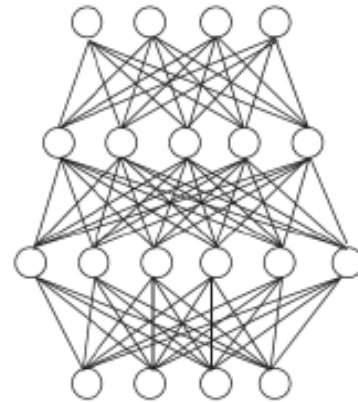
多层前馈网络结构

多层网络：包含隐层的网络

前馈网络：由输入层、隐藏层和输出层组成。神经元之间不存在同层连接也不存在跨层连接，即网络中无环或者回路。



(a) 单隐层前馈网络



(b) 双隐层前馈网络

隐层和输出层神经元亦称“功能单元”，其中隐藏层的主要作用是增加网络的非线性能力

多层前馈网络有强大的表示能力（万有逼近性）

只需一个包含足够多神经元的隐层，多层前馈神经网络就能以任意精度逼近任意复杂度的连续函数 [Hornik et al., 1989]

但是，如何设置隐层神经元数是未决问题。实际常用“试错法”

误差逆传播算法 (BackPropagation, BP)

逆传播/反向传播算法是最成功、最常用的神经网络学习算法。现实任务中使用神经网络时，大多是在使用BP算法进行训练，值得指出的是，BP算法不仅可以用于多层前馈神经网络，还可以用于其它类型的神经网络，例如卷积神经网络

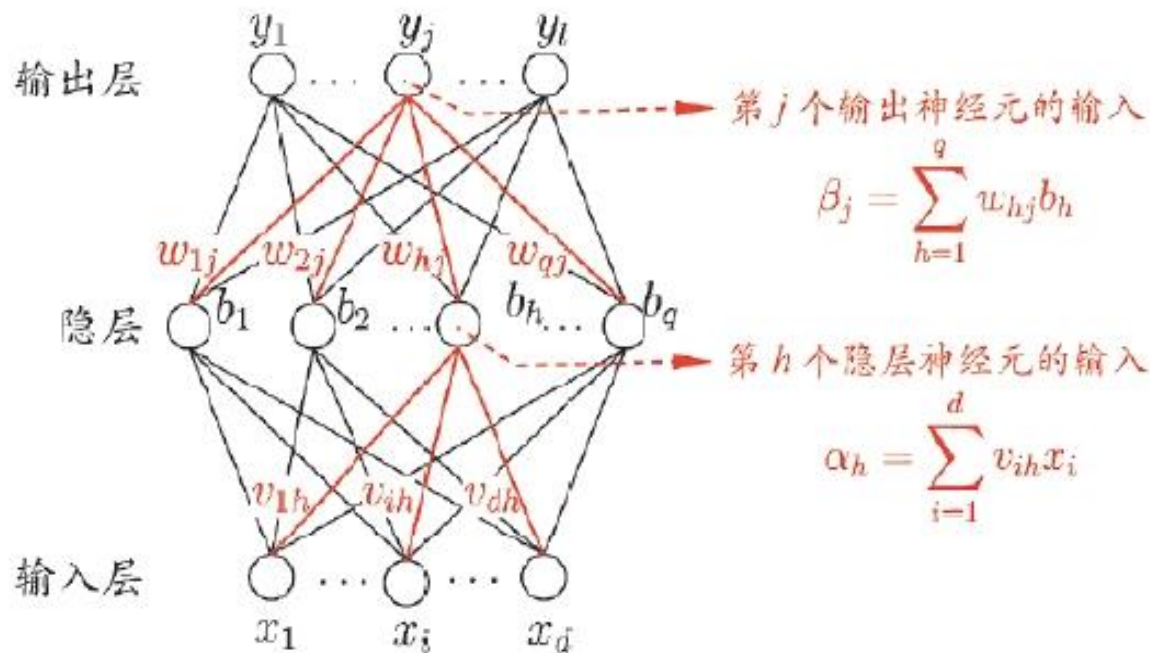
给定训练集 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$, $x_i \in \mathbb{R}^d$, $y_i \in \mathbb{R}^l$

输入： d 维特征向量

输出： l 个输出值

隐层：假定使用 q 个
隐层神经元

假定功能单元均使用
Sigmoid 函数



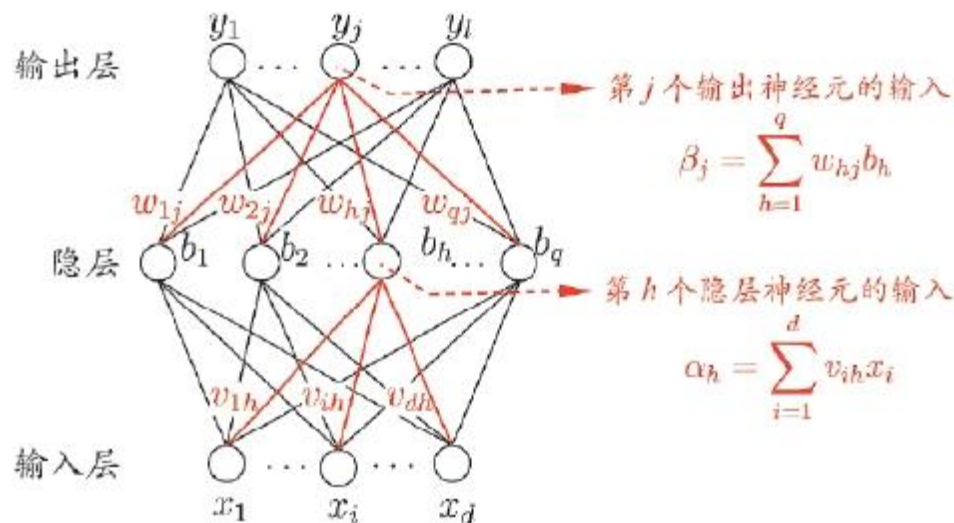
BP 算法推导

对于训练例 $(\mathbf{x}_k, \mathbf{y}_k)$, 假定网络的实际输出为 $\hat{\mathbf{y}}_k = (\hat{y}_1^k, \hat{y}_2^k, \dots, \hat{y}_l^k)$

$$\hat{y}_j^k = f(\beta_j - \theta_j)$$

则网络在 $(\mathbf{x}_k, \mathbf{y}_k)$ 上的均方误差为:

$$E_k = \frac{1}{2} \sum_{j=1}^l (\hat{y}_j^k - y_j^k)^2$$



需通过学习确定的参数数目: $(d + l + 1)q + l$

BP 是一个迭代学习算法, 在迭代的每一轮中采用如下误差修正:

$$v \leftarrow v + \boxed{\Delta v}$$

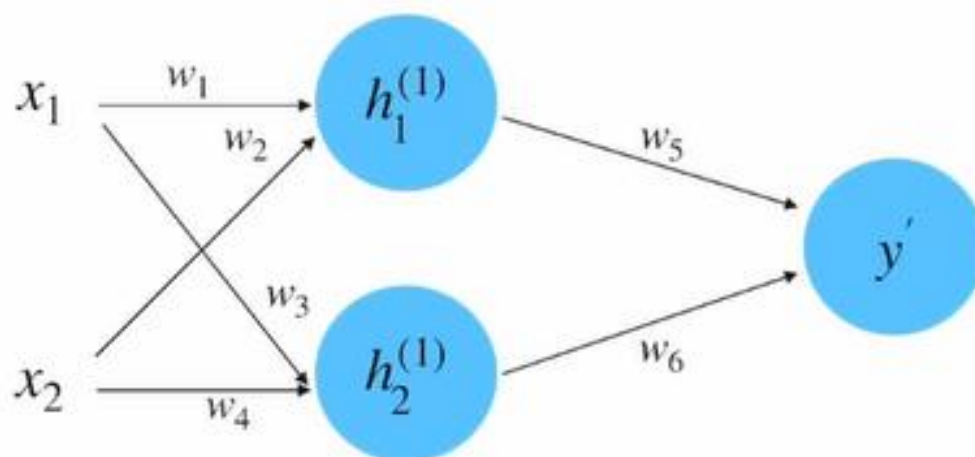
BP 算法推导

□ 梯度下降法 (Gradient descent)

- 梯度下降是一种优化算法，用于最小化一个函数，它在神经网络训练中被广泛用于找到使损失函数最小的参数（权重和偏置）
- 梯度下降的核心思想是利用梯度（即损失函数关于参数的导数）来迭代地更新参数，从而逐步逼近函数的最小值

BP 算法推导

构建一个只有一层的神经网络



假设这个样本的特征有两个维度，
每个维度的值为

$$x_1 = 0.5$$

$$x_2 = 1.0$$

假设这个样本的真实值为：

$$y = 0.8$$

神经网络所有参数
随机初始化，且恰好如下

$$w_1 = 1.0$$

$$w_2 = 0.5$$

$$w_3 = 0.5$$

$$w_4 = 0.7$$

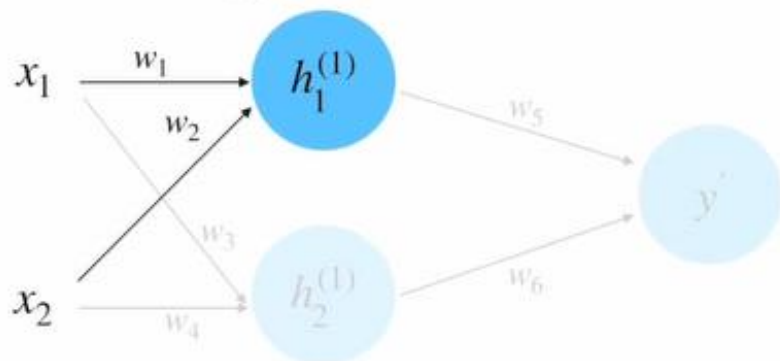
$$w_5 = 1.0$$

$$w_6 = 2.0$$

BP 算法推导 – 正向传播

正向传播：计算第一个神经元的输出

$$\begin{aligned}h_1^{(1)} &= w_1 \cdot x_1 + w_2 \cdot x_2 \\&= 1.0 \cdot 0.5 + 0.5 \cdot 1.0 \\&= 1.0\end{aligned}$$



假设这个样本的特征有两个维度，
每个维度的值为

$$\begin{aligned}x_1 &= 0.5 \\x_2 &= 1.0\end{aligned}$$

假设这个样本的真实值为：

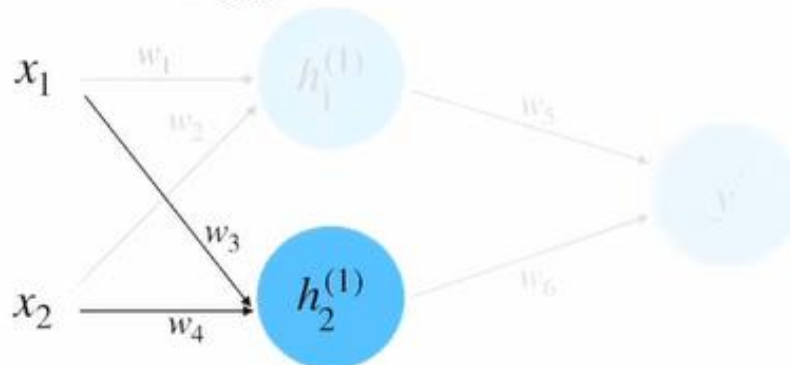
$$y = 0.8$$

神经网络所有参数
随机初始化，且恰好如下

$$\begin{aligned}w_1 &= 1.0 \\w_2 &= 0.5 \\w_3 &= 0.5 \\w_4 &= 0.7 \\w_5 &= 1.0 \\w_6 &= 2.0\end{aligned}$$

正向传播：计算第二个神经元的输出

$$\begin{aligned}h_2^{(1)} &= w_3 \cdot x_1 + w_4 \cdot x_2 \\&= 0.5 \cdot 0.5 + 0.7 \cdot 1.0 \\&= 0.95\end{aligned}$$



假设这个样本的特征有两个维度，
每个维度的值为

$$\begin{aligned}x_1 &= 0.5 \\x_2 &= 1.0\end{aligned}$$

假设这个样本的真实值为：

$$y = 0.8$$

神经网络所有参数
随机初始化，且恰好如下

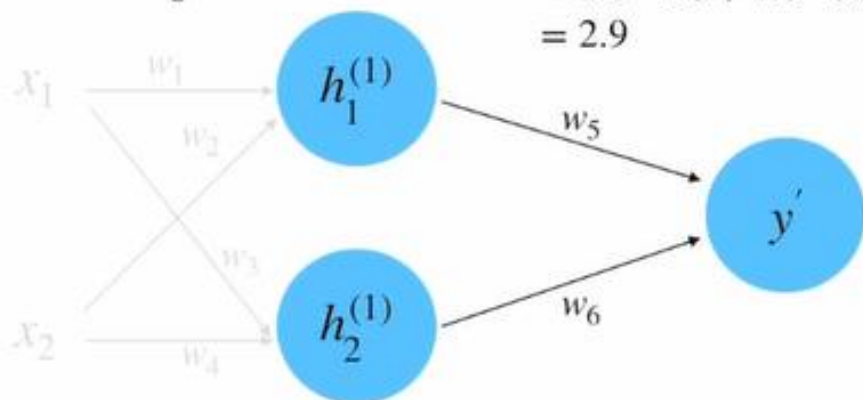
$$\begin{aligned}w_1 &= 1.0 \\w_2 &= 0.5 \\w_3 &= 0.5 \\w_4 &= 0.7 \\w_5 &= 1.0 \\w_6 &= 2.0\end{aligned}$$

BP 算法推导 – 正向传播

正向传播：计算最后一个神经元的输出

$$h_1^{(1)} = 1.0$$
$$h_2^{(1)} = 0.95$$

$$y' = w_5 \cdot h_1^{(1)} + w_6 \cdot h_2^{(1)}$$
$$= 1.0 \cdot 1.0 + 2.0 \cdot 0.95$$
$$= 2.9$$



假设这个样本的特征有两个维度，
每个维度的值为

$$x_1 = 0.5$$
$$x_2 = 1.0$$

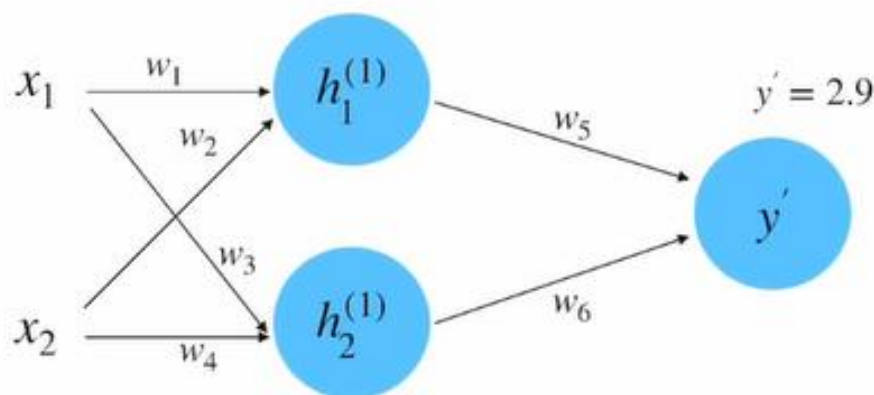
假设这个样本的真实值为：

$$y = 0.8$$

神经网络所有参数
随机初始化，且恰好如下

$$w_1 = 1.0$$
$$w_2 = 0.5$$
$$w_3 = 0.5$$
$$w_4 = 0.7$$
$$w_5 = 1.0$$
$$w_6 = 2.0$$

正向传播完成



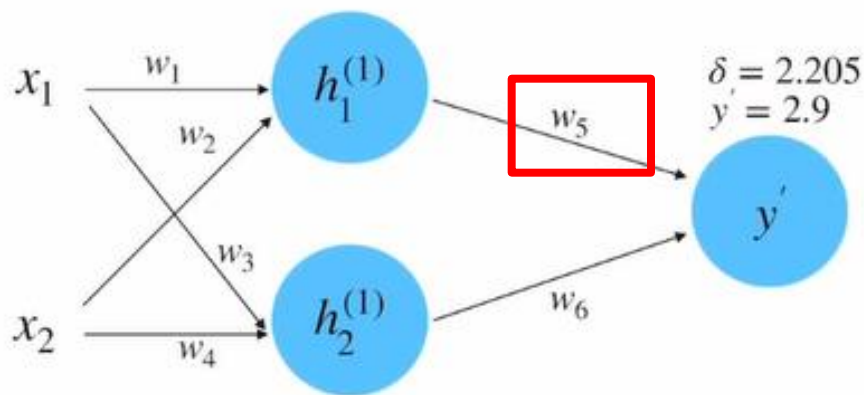
这一轮正向传播后
误差如右所示：

(这个公式是损失函数的计算，
一般不同任务，损失函数会有所不同，
这里暂不详细说损失函数的问题)

$$\delta = \frac{1}{2}(y - y')^2$$
$$= 0.5(0.8 - 2.9)^2$$
$$= 2.205$$

BP 算法推导 – 反向传播

backward propagation 反向传播



以计算 w_5 为例, w_6 同理

$$\frac{\partial \delta}{\partial w_5} = \frac{\partial \delta}{\partial y'} \cdot \frac{\partial y'}{\partial w_5} = 2.1$$

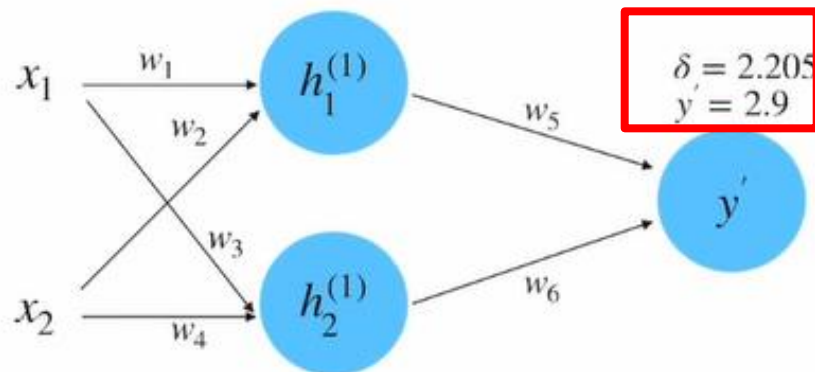
$$\delta = \frac{1}{2}(y - y')^2$$

$$\begin{aligned}\frac{\partial \delta}{\partial y'} &= 2 \cdot \frac{1}{2} \cdot (y - y')(-1) \\ &= y' - y \\ &= 2.9 - 0.8 = 2.1\end{aligned}$$

$$y' = w_5 \cdot h_1^{(1)} + w_6 \cdot h_2^{(1)}$$

$$\begin{aligned}\frac{\partial y'}{\partial w_5} &= h_1^{(1)} + 0 \\ &= 1.0\end{aligned}$$

backward propagation 反向传播



由前一页可知

$$\frac{\partial \delta}{\partial w_5} = 2.1 \cdot 1.0 = 2.1$$

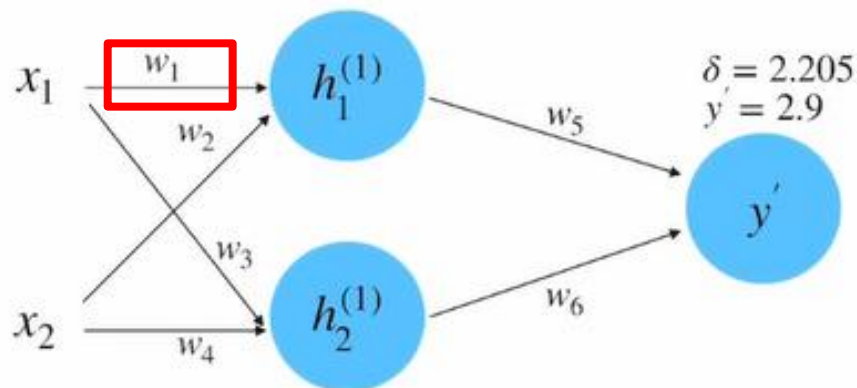
那么便可更新 w_5 的值

$$\begin{aligned}w_5^{(update)} &= w_5 - \eta \cdot \frac{\partial \delta}{\partial w_5} \\ &= 1.0 - 0.1 \cdot 2.1 \\ &= 0.79\end{aligned}$$

学习率, 属于超参数, 自行设定

BP 算法推导 – 反向传播

backward propagation 反向传播



以计算 w_1 为例,
 w_2, w_3, w_4 同理

$$\frac{\partial \delta}{\partial w_1} = \frac{\partial \delta}{\partial y'} \cdot \frac{\partial y'}{\partial h_1^{(1)}} \cdot \frac{\partial h_1^{(1)}}{\partial w_1}$$

$$\delta = \frac{1}{2}(y - y')^2$$

$$\begin{aligned}\frac{\partial \delta}{\partial y'} &= 2 \cdot \frac{1}{2} \cdot (y - y')(-1) \\ &= y' - y \\ &= 2.9 - 0.8 \\ &= 2.1\end{aligned}$$

$$y' = w_5 \cdot h_1^{(1)} + w_6 \cdot h_2^{(1)}$$

$$\begin{aligned}\frac{\partial y'}{\partial h_1^{(1)}} &= w_5 + 0 \\ &= 1.0\end{aligned}$$

$$h_1^{(1)} = w_1 \cdot x_1 + w_2 \cdot x_2$$

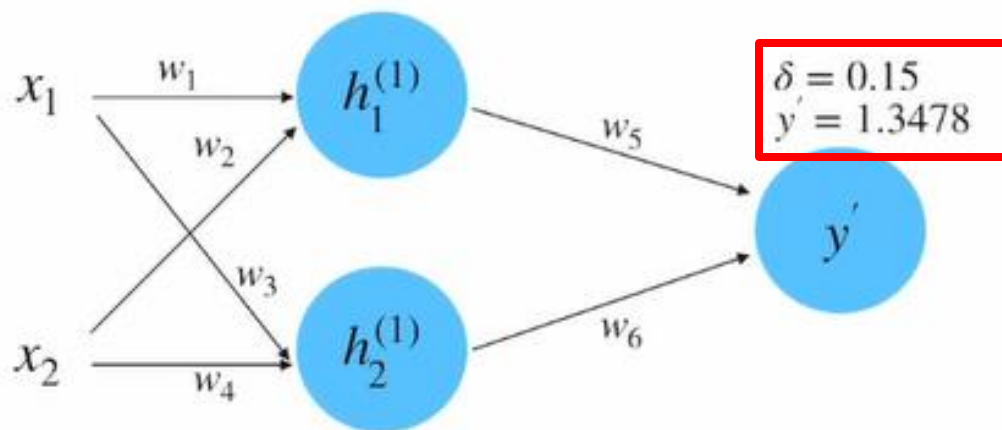
$$\begin{aligned}\frac{\partial h_1^{(1)}}{\partial w_1} &= x_1 + 0 \\ &= 0.5\end{aligned}$$



$$\frac{\partial \delta}{\partial w_1} = 2.1 \cdot 1.0 \cdot 0.5 = 1.05$$

BP 算法推导

第二轮正向传播得到的误差



$$\begin{aligned}h_1^{(1)} &= w_1 \cdot x_1 + w_2 \cdot x_2 \\&= 0.895 \cdot 0.5 + 0.29 \cdot 1.0 \\&= 0.7375\end{aligned}$$

$$\begin{aligned}h_2^{(1)} &= w_3 \cdot x_1 + w_4 \cdot x_2 \\&= 0.29 \cdot 0.5 + 0.28 \cdot 1.0 \\&= 0.425\end{aligned}$$

$$\begin{aligned}y' &= w_5 \cdot h_1^{(1)} + w_6 \cdot h_2^{(1)} \\&= 0.79 \cdot 0.7375 + 1.8005 \cdot 0.425 \\&= 1.3478\end{aligned}$$

应用：图像识别任务

□ Softmax 分类器

- 如何把一个得分值转换成一个概率值？
- 将得分值归一化

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

假设神经网络的输出层有三个神经元，输出： $z = [z_1, z_2, z_3] = [2, 1, 0.5]$

步骤 1: 计算每个分数的指数

$$\begin{aligned} e^{z_1} &= e^2 \\ e^{z_2} &= e^1 \\ e^{z_3} &= e^{0.5} \end{aligned}$$

步骤 2: 计算指数的总和

$$\sum_j e^{z_j} = e^2 + e^1 + e^{0.5}$$

步骤 3: 每个类别的softmax概率

$$\begin{aligned} \text{Softmax}(z_1) &\approx \frac{7.389}{11.756} \approx 0.628 \\ \text{Softmax}(z_2) &\approx \frac{2.718}{11.756} \approx 0.231 \\ \text{Softmax}(z_3) &\approx \frac{1.649}{11.756} \approx 0.140 \end{aligned}$$

- 类别 1 的概率： ≈ 0.628
- 类别 2 的概率： ≈ 0.231
- 类别 3 的概率： ≈ 0.140

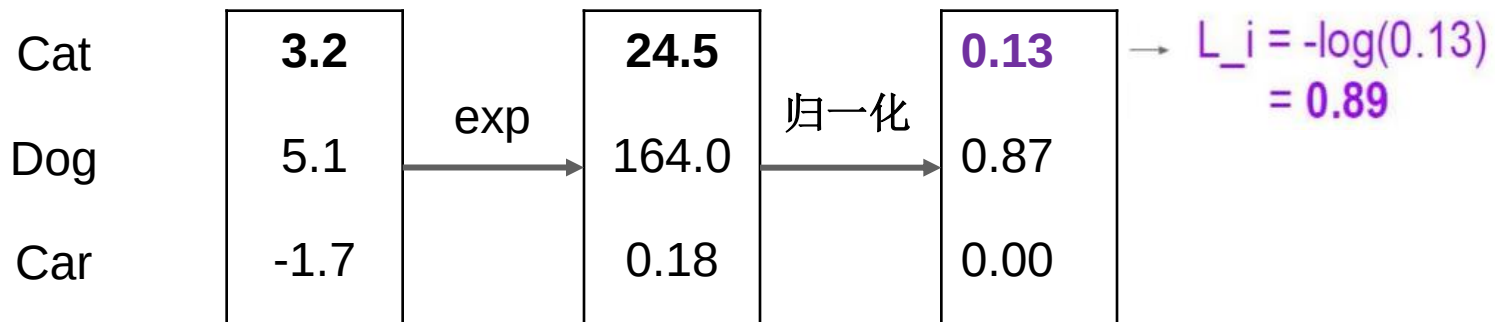
应用：图像识别任务

□ Softmax 分类器

- 如何把一个得分值转换成一个概率值？
- 将得分值归一化

□ 交叉熵损失函数

$$L(y, \hat{y}) = - \sum_{i=1}^k y_i \log \hat{y}_i$$



应用：图像识别任务

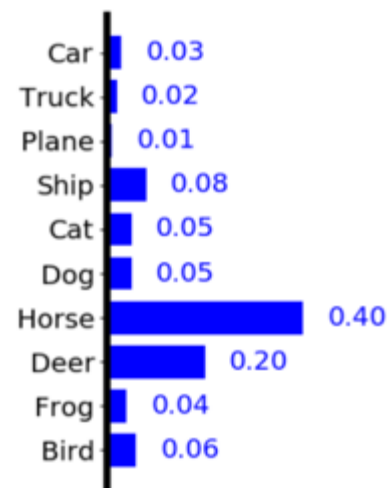
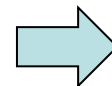
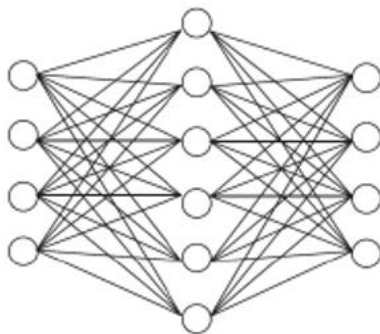
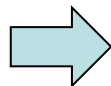
□ Softmax 分类器

- 如何把一个得分值转换成一个概率值？
- 将得分值归一化



Input

$3 \times 32 \times 32$ tensor



第6章 贝叶斯分类器

贝叶斯

➤ 贝叶斯简介

- 贝叶斯（约1701-1761）Thomas Bayes, 英国数学家
- 贝叶斯方法源于他生前为解决一个“逆概”问题写的一篇文章
- 生不逢时，死后它的作品才被世人认可



贝叶斯

➤ 贝叶斯要解决的问题

- **正向概率**：假设袋子里面有 N 个白球， M 个黑球，你伸手进去摸一个，摸出黑球的概率多大？
- **逆向概率**：如果事先不知道袋子里面黑白球的比例，而是闭着眼睛摸出一个（或者几个）球，观察这些取出来的球的颜色之后，那么我们可以就此对袋子里面的黑白球的比例做出什么样的推测

贝叶斯公式推导

➤ 例子



男生：60%
女生：40%

- 男生总是穿长裤，女生则一半穿长裤一半穿裙子
- **正向概率**：随机选取一个学生，他（她）穿长裤的概率和穿裙子的概率是多大？
- **逆向概率**：迎面走来一个穿长裤的学生，你只看见他（她）穿的长裤，而无法确定他（她）的性别，你能够推断出她是女生的概率是多大吗？

贝叶斯公式推导

- 假设学校里面学生总数是 U
- 穿长裤的(男生)人数: $U * P(\text{Boy}) * P(\text{Pants}|\text{Boy})$
 - $P(\text{Boy})$ 是男生的概率 = 60%
 - $P(\text{Pants}|\text{Boy})$ 是条件概率, 即在男生这个条件下穿长裤的概率是多大, 这里是100%
- 穿长裤的(女生)人数: $U * P(\text{Girl}) * P(\text{Pants}|\text{Girl})$

贝叶斯公式推导

➤ 求解：穿长裤的人里面有多少女生

➤ 穿长裤的总人数：

$$U * P(\text{Boy}) * P(\text{Pants}|\text{Boy}) + U * P(\text{Girl}) * P(\text{Pants}|\text{Girl})$$

➤ 穿长裤的是女生的概率： $P(\text{Girl}|\text{Pants})$?

穿长裤-女生-人数 / 穿长裤-总人数

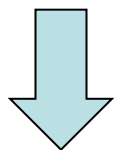
$$P(\text{Girl}|\text{Pants}) = \frac{U * P(\text{Girl}) * P(\text{Pants}|\text{Girl})}{U * P(\text{Boy}) * P(\text{Pants}|\text{Boy}) + U * P(\text{Girl}) * P(\text{Pants}|\text{Girl})}$$

贝叶斯公式推导

➤ 与总人数无关

$$P(\text{Girl}|\text{Pants}) = \frac{P(\text{Girl}) * P(\text{Pants}|\text{Girl})}{P(\text{Boy}) * P(\text{Pants}|\text{Boy}) + P(\text{Girl}) * P(\text{Pants}|\text{Girl})}$$

Girl = A
Pants = B



$P(\text{Pants})$

$$P(A|B) = \frac{P(A) * P(B|A)}{P(B)}$$

似然（类条件概率）
假设A为真的条件下，
观察到证据B的概率

后验概率
在证据B出现
后，假设A为
真的概率

先验概率
样本空间中各类样本所占的
比例，可通过各类样本出现
的频率估计（大数定理）

边缘概率
观察到证据B的总概率

极大似然估计 – 了解

- 估计类条件概率常用的策略：先假定其具有某种确定的概率分布形式，再基于训练样本对概率分布参数进行估计
- 概率模型的训练过程就是参数估计过程，统计学界的两个学派提供了不同的方案：
 - 频率主义学派 (frequentist) 认为参数虽然未知，但却存在客观值，因此可通过优化似然函数等准则来确定参数值
 - 贝叶斯学派 (Bayesian) 认为参数是未观察到的随机变量、其本身也可由分布，因此可假定参数服从一个先验分布，然后基于观测到的数据计算参数的后验分布。

朴素贝叶斯分类器

- 估计后验概率主要困难：类条件概率是所有属性上的联合概率难以从有限的训练样本估计获得
- 朴素贝叶斯分类器(Naïve Bayes Classifier)采用了“**属性条件独立性假设**”(attribute conditional independence assumption): **每个属性独立地对分类结果发生影响**

朴素贝叶斯分类器 - 应用实例

➤ 垃圾邮件识别

$$P(A|B) = \frac{P(A) * P(B|A)}{P(B)}$$

- 问题：给定一封邮件，判断它是否属于垃圾邮件

D表示这封邮件，D是由N个单词组成

h+表示垃圾邮件，h-表示正常邮件

$$P(h+ | D) = P(h+) * P(D | h+) / P(D)$$

$$P(h- | D) = P(h-) * P(D | h-) / P(D)$$

朴素贝叶斯分类器 - 应用实例

➤ 垃圾邮件识别

- 先验概率： $P(h^+)$ 和 $P(h^-)$ 根据观测数据很容易求出来
- D 里面含有 N 个单词 $d_1, d_2, d_3, \dots$
- $P(D | h^+) = P(d_1, d_2, \dots, d_n | h^+)$, 就是说在垃圾邮件中出现和目前这封邮件一样的一封邮件的概率是多大

$P(d_1, d_2, \dots, d_n | h^+)$ 扩展为：

$$P(d_1 | h^+) * P(d_2 | d_1, h^+) * P(d_3 | d_2, d_1, h^+) * \dots$$

朴素贝叶斯分类器 - 应用实例

➤ 垃圾邮件识别

$P(d_1, d_2, \dots, d_n | h_+)$ 扩展为：

所有属性上的联合概率难以获得

$$P(d_1 | h_+) * P(d_2 | d_1, h_+) * P(d_3 | d_2, d_1, h_+) * \dots$$

- 假设 d_i 与 d_{i-1} 是完全条件无关的：朴素贝叶斯假设特征之间是独立的，互不影响
- 化简为： $P(d_1 | h_+) * P(d_2 | h_+) * P(d_3 | h_+) * \dots$
- 对于 $P(d_1 | h_+) * P(d_2 | h_+) * P(d_3 | h_+) * \dots$ ，只需要统计 d_i 这个单词在垃圾邮件中出现的频率即可

贝叶斯公式应用实例

➤ 西瓜分类

现在有一个新西瓜的样本：
青绿，稍蜷，浊响，清晰

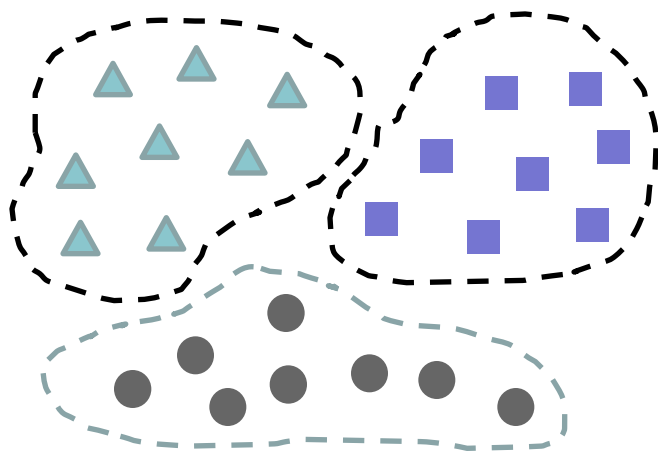
使用朴素贝叶斯算法计算
这个新西瓜是好瓜的概率

编号	色泽	根蒂	敲声	纹理	好瓜
1	青绿	蜷缩	浊响	清晰	是
2	乌黑	蜷缩	沉闷	清晰	是
3	乌黑	蜷缩	浊响	清晰	是
4	青绿	蜷缩	沉闷	清晰	是
5	浅白	蜷缩	浊响	清晰	是
6	青绿	稍蜷	浊响	清晰	是
7	乌黑	稍蜷	浊响	稍糊	是
8	乌黑	稍蜷	浊响	清晰	是
9	乌黑	稍蜷	沉闷	稍糊	否
10	青绿	硬挺	清脆	清晰	否
11	浅白	硬挺	清脆	模糊	否
12	浅白	蜷缩	浊响	模糊	否
13	青绿	稍蜷	浊响	稍糊	否
14	浅白	稍蜷	沉闷	稍糊	否
15	乌黑	稍蜷	浊响	清晰	否
16	浅白	蜷缩	浊响	模糊	否
17	青绿	蜷缩	沉闷	稍糊	否

第7章 聚类 (Clustering)

聚类任务

- 在“**无监督学习**”任务中研究最多、应用最广
- 聚类目标：将数据集中的样本划分为若干个通常不相交的子集（“簇”，cluster）
- 聚类既可以作为一个单独过程（用于找寻数据内在的分布结构），也可作为分类等其他学习任务的前驱过程



为什么需要聚类？

➤ 无限的数据增加, 有限的计算处理能力

- 信息量**急速增长**（大量的数据比如图像、文本、短视频等数据需要处理）
- 需要**自主分析**数据（如何让计算机实现数据自动处理，并找出其中的规律？）

聚类的核心概念

- 俗话说物以类聚人以群分
- 聚类问题：按照某个特定标准把一个数据集分割成不同的类或簇，使得同一个簇内的数据对象的相似性尽可能大，同时不在同一个簇中的数据对象的差异尽可能大
- 换言之，聚类结果的“簇内相似度”（intra-cluster similarity）高，且“簇间相似度”（inter-cluster similarity）低，这样的聚类效果较好

聚类的核心概念

Clustering 聚类 无监督学习

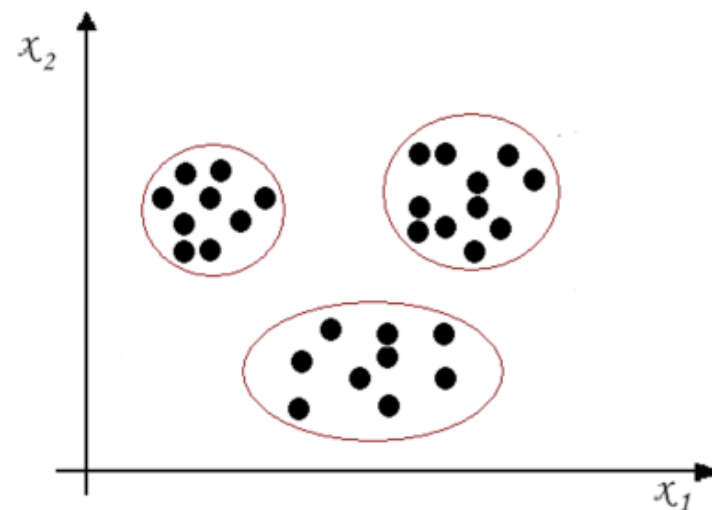
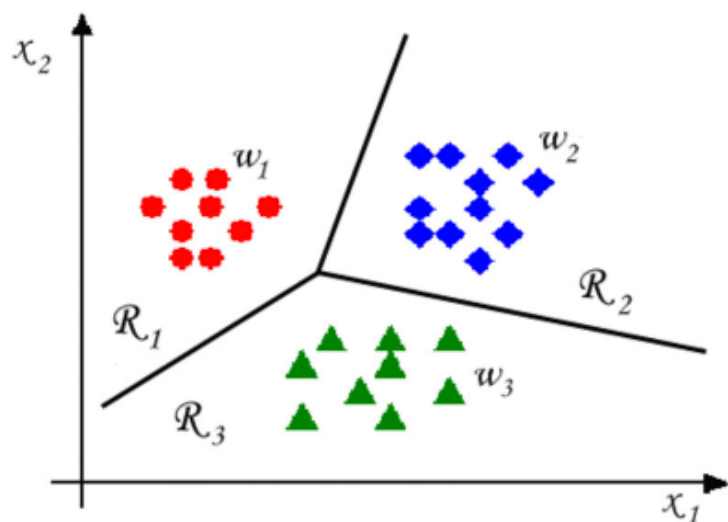
把相似的东西分到一组（按照指定的**标准**）。聚类并不关心某一类是什么，目标只是把相似的东西聚到一起。一个聚类算法通常只需要知道**如何计算相似度**，就可以开始工作了。

Classification 分类 有监督学习

只需要告诉一个(Classifier)分类器这个东西被分为某类这样一些例子。理想情况下，分类器会从训练集中进行学习，从而具备对未知数据进行分类的能力

聚类的核心概念

➤ 分类(Classification) vs. 聚类(Clustering)



- 标签数据
- 监督
- 寻找分类边界



- 无标签数据
- 无监督
- 寻找数据结构聚类

聚类 – 相似性度量

➤ 在聚类之前，首先要分析样本间的相似程度

距离度量

样本 x 和 y 之间的距离 $d(x, y)$ 一般满足以下四项基本性质：

- (1) 非负性： $d(x, y) \geq 0$ ；
- (2) 同一性： $d(x, y) = 0$ ，当且仅当 $x = y$ ；
- (3) 对称性： $d(x, y) = d(y, x)$ ；
- (4) 三角不等式： $d(x, y) \leq d(x, z) + d(z, y)$ 。

聚类 – 相似性度量

➤ 常用距离形式

闵可夫斯基距离 (Minkowski distance)

闵可夫斯基距离是曼哈顿距离和欧氏距离的扩展，定义为：

$$d(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{\frac{1}{p}}$$

其中， p 是一个可变参数。可以发现：

当 $p = 2$ 时，闵可夫斯基距离即为欧氏距离；

当 $p = 1$ 时，闵可夫斯基距离则为曼哈顿距离。

聚类 – 相似性度量

➤ 其它度量形式

角度相似性函数

$$S(x, z) = \frac{x^t z}{\|x\| \bullet \|z\|}$$

- 表征了向量 x, z 之间夹角的余弦，反映了几何上的相似性，当坐标系旋转或者尺度变换， $S(x, z)$ 均保持不变

聚类 – 相似性度量

➤ 其它度量形式

汉明距离 (Hamming Distance)

- 给定两个相同长度的字符串，汉明距离定义为两个字符串中对应位置数值不同的字符的个数
- 如字符串“101101”和“100110”之间的汉明距离为3
- 汉明距离越小，两个样本特征越相似

思考 $H(x,y) = ?$ For $x = [0, 1, 0, 1, 1, 0]^t$;
 $y = [0, 0, 1, 1, 0, 1]^t$;

原型聚类

- 原型聚类也称为“基于原型的聚类”(prototype-based clustering)，此类算法假设聚类结构能通过一组原型刻画
- 算法过程：通常情况下，算法先对原型进行初始化，再对原型进行迭代更新求解。
- 几种著名的原型聚类算法：k均值算法、学习向量量化算法、高斯混合聚类算法
- 主要介绍k均值算法 (k-means)

K-means 算法

➤ 核心概念

- 要得到簇的个数，需要指定K值
- 质心：均值，即向量各维度取均值
- 距离的度量：常用欧几里得距离和余弦相似度（先标准化）

K-means 算法

➤ 优化目标

给定数据集 $D = \{x_1, x_2, \dots, x_m\}$, k 均值算法针对聚类所得簇划分 $C = \{C_1, C_2, \dots, C_k\}$ **最小化平方误差**

$$E = \sum_{i=1}^k \sum_{x \in C_j} \|x - \mu_i\|_2^2$$

**最小化所有点到其
质心距离的平方和**

其中, μ_i 是簇 C_i 的均值向量

E 值在一定程度上刻画了簇内样本围绕簇均值向量的紧密程度, E 值越小, 则簇内样本相似度越高。

算法流程 (迭代优化):

初始化每个簇的均值向量

repeat

1. (更新) 簇划分;

2. 计算每个簇的均值向量

until 当前均值向量均未更新

K-means 算法

➤ 算法伪代码

输入: 样本集 $D = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$;
聚类簇数 k .

过程:

- 1: 从 D 中随机选择 k 个样本作为初始均值向量 $\{\boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \dots, \boldsymbol{\mu}_k\}$
- 2: **repeat**
- 3: 令 $C_i = \emptyset$ ($1 \leq i \leq k$)
- 4: **for** $j = 1, \dots, m$ **do**
- 5: 计算样本 $\mathbf{x}_j$ 与各均值向量 $\boldsymbol{\mu}_i$ ($1 \leq i \leq k$) 的距离: $d_{ji} = \|\mathbf{x}_j - \boldsymbol{\mu}_i\|_2$;
- 6: 根据距离最近的均值向量确定 $\mathbf{x}_j$ 的簇标记: $\lambda_j = \arg \min_{i \in \{1, 2, \dots, k\}} d_{ji}$;
- 7: 将样本 $\mathbf{x}_j$ 划入相应的簇: $C_{\lambda_j} = C_{\lambda_j} \cup \{\mathbf{x}_j\}$;
- 8: **end for**
- 9: **for** $i = 1, \dots, k$ **do**
- 10: 计算新均值向量: $\boldsymbol{\mu}'_i = \frac{1}{|C_i|} \sum_{\mathbf{x} \in C_i} \mathbf{x}$;
- 11: **if** $\boldsymbol{\mu}'_i \neq \boldsymbol{\mu}_i$ **then**
- 12: 将当前均值向量 $\boldsymbol{\mu}_i$ 更新为 $\boldsymbol{\mu}'_i$
- 13: **else**
- 14: 保持当前均值向量不变
- 15: **end if**
- 16: **end for**
- 17: **until** 当前均值向量均未更新
- 18: **return** 簇划分结果

输出: 簇划分 $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$

K-means 算法

➤ 算法实例

- 以西瓜数据集为例，来演示k-means算法的学习过程，将编号为 i 的样本称为 x_i

编号	密度	含糖率	编号	密度	含糖率	编号	密度	含糖率
1	0.697	0.460	11	0.245	0.057	21	0.748	0.232
2	0.774	0.376	12	0.343	0.099	22	0.714	0.346
3	0.634	0.264	13	0.639	0.161	23	0.483	0.312
4	0.608	0.318	14	0.657	0.198	24	0.478	0.437
5	0.556	0.215	15	0.360	0.370	25	0.525	0.369
6	0.403	0.237	16	0.593	0.042	26	0.751	0.489
7	0.481	0.149	17	0.719	0.103	27	0.532	0.472
8	0.437	0.211	18	0.359	0.188	28	0.473	0.376
9	0.666	0.091	19	0.339	0.241	29	0.725	0.445
10	0.243	0.267	20	0.282	0.257	30	0.446	0.459

K-means 算法

➤ 算法实例

假定聚类簇数 $k=3$ ，算法开始时，**随机选择**3个样本 x_6, x_{12}, x_{27} 作为**初始均值向量**，即 $\mu_1 = (0.403; 0.237), \mu_2 = (0.343; 0.099), \mu_3 = (0.533; 0.472)$

考察样本 $x_1 = (0.697; 0.460)$ ，它与当前均值向量 μ_1, μ_2, μ_3 的距离分别为**0.369, 0.506, 0.166**，因此 x_1 将被划入簇 C_3 中。类似的，对数据集中的所有样本考察一遍后，可得当前簇划分为

$$C_1 = \{x_5, x_6, x_7, x_8, x_9, x_{10}, x_{13}, x_{14}, x_{15}, x_{17}, x_{18}, x_{19}, x_{20}, x_{23}\}$$

$$C_2 = \{x_{11}, x_{12}, x_{16}\}$$

$$C_3 = \{x_1, x_2, x_3, x_4, x_{21}, x_{22}, x_{24}, x_{25}, x_{26}, x_{27}, x_{28}, x_{29}, x_{30}\}$$

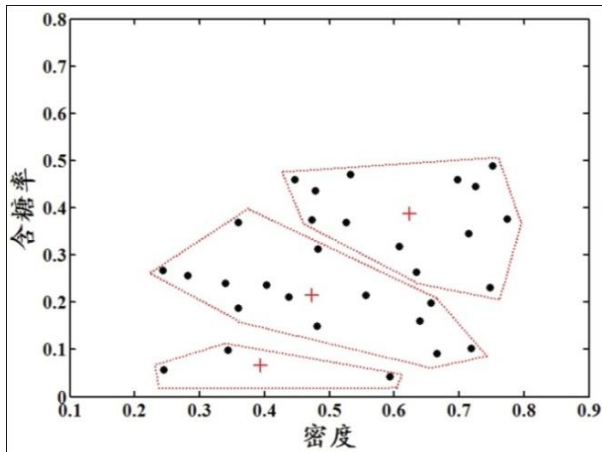
于是，可以从分别**求得新的均值向量（更新质心）**

$$\mu'_1 = (0.473; 0.214), \mu'_2 = (0.394; 0.066), \mu'_3 = (0.623; 0.388)$$

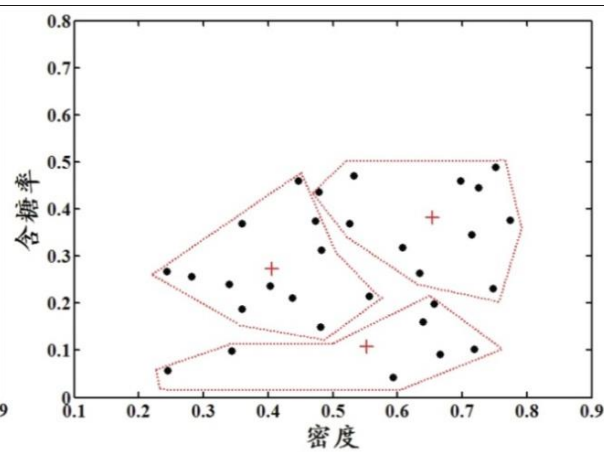
不断重复上述过程，如下图所示

K-means 算法

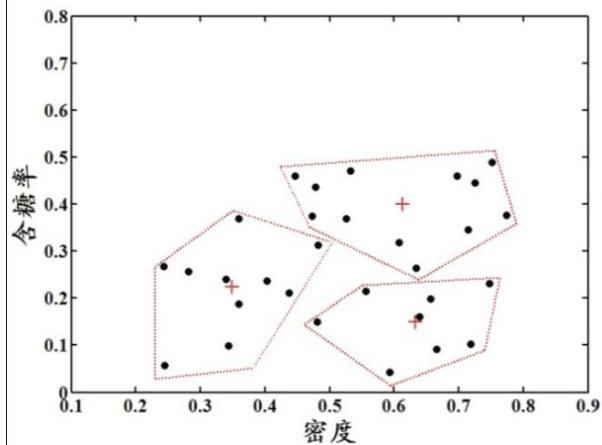
➤ 聚类结果



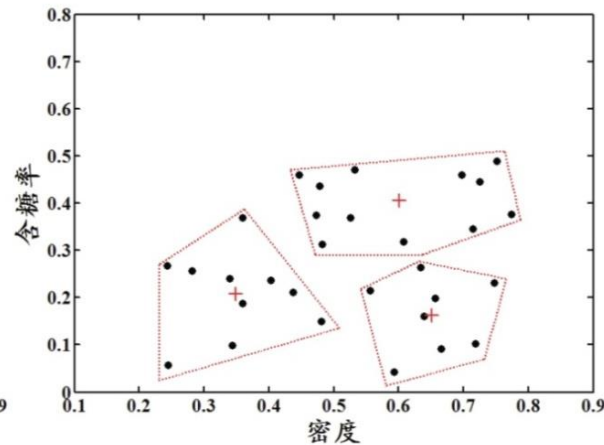
(a) 第一轮迭代后



(b) 第二轮迭代后



(c) 第三轮迭代后



(d) 第四轮迭代后

K-means 算法

➤ 算法实例

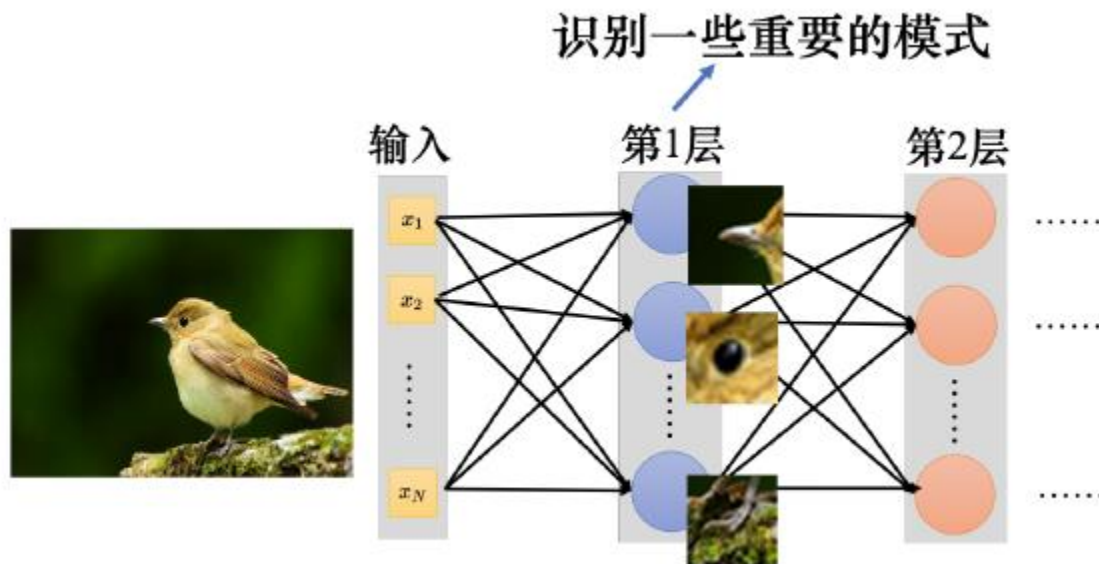
第11章 卷积神经网络

Convolutional Neural Network (CNN)

卷积神经网络

➤ 观察1：检测模式不需要整张图像

- 假设我们的任务是让网络识别出图像的动物，它需要做的就是检测图像里面有没有出现一些特别重要的模式（pattern），这些模式代表了某种动物，比如有三个神经元分别看到了鸟嘴、眼睛、鸟爪3个模式，这就代表网络看到了一只鸟

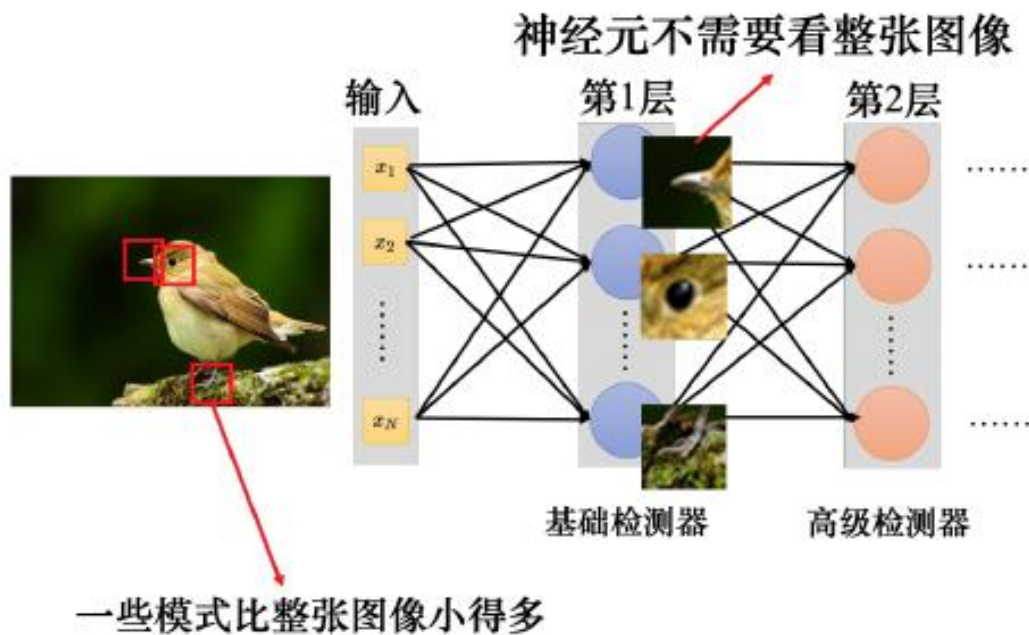


卷积神经网络

➤ 观察1：检测模式不需要整张图像

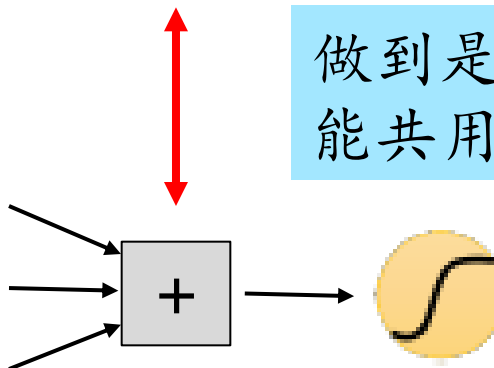
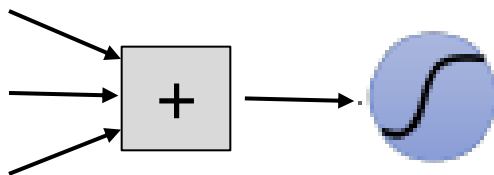
- 人在判断一个物体的时候，往往也是抓**最重要的特征**，看到这些特征后，就会直觉地识别某种物体
- 希望机器也具备这样的能力，即用部分神经元来判断某种模式是否出现

只需要一个神经元连接到一小部分图像



卷积神经网络

- 观察2：同样的模式可能存在于不同的区域



做到是同样一件事：检测鸟的嘴
能共用一组参数

卷积神经网络

➤ 观察3：下采样不影响模式检测

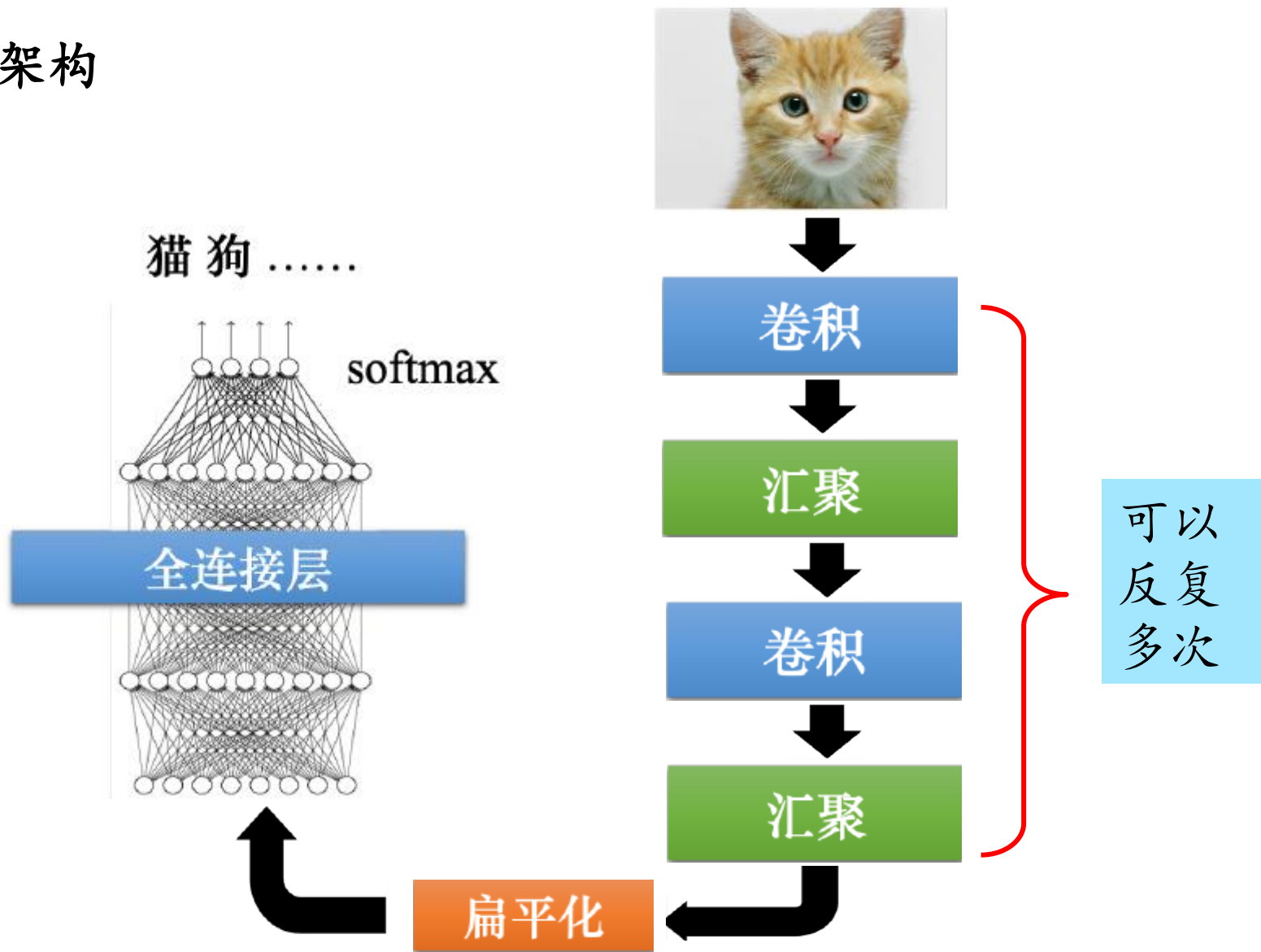
- 例如，去掉图像中奇数行、偶数列的像素点



可以通过下采样减小图像大小，进而减少参数量

卷积神经网络

➤ CNN的架构



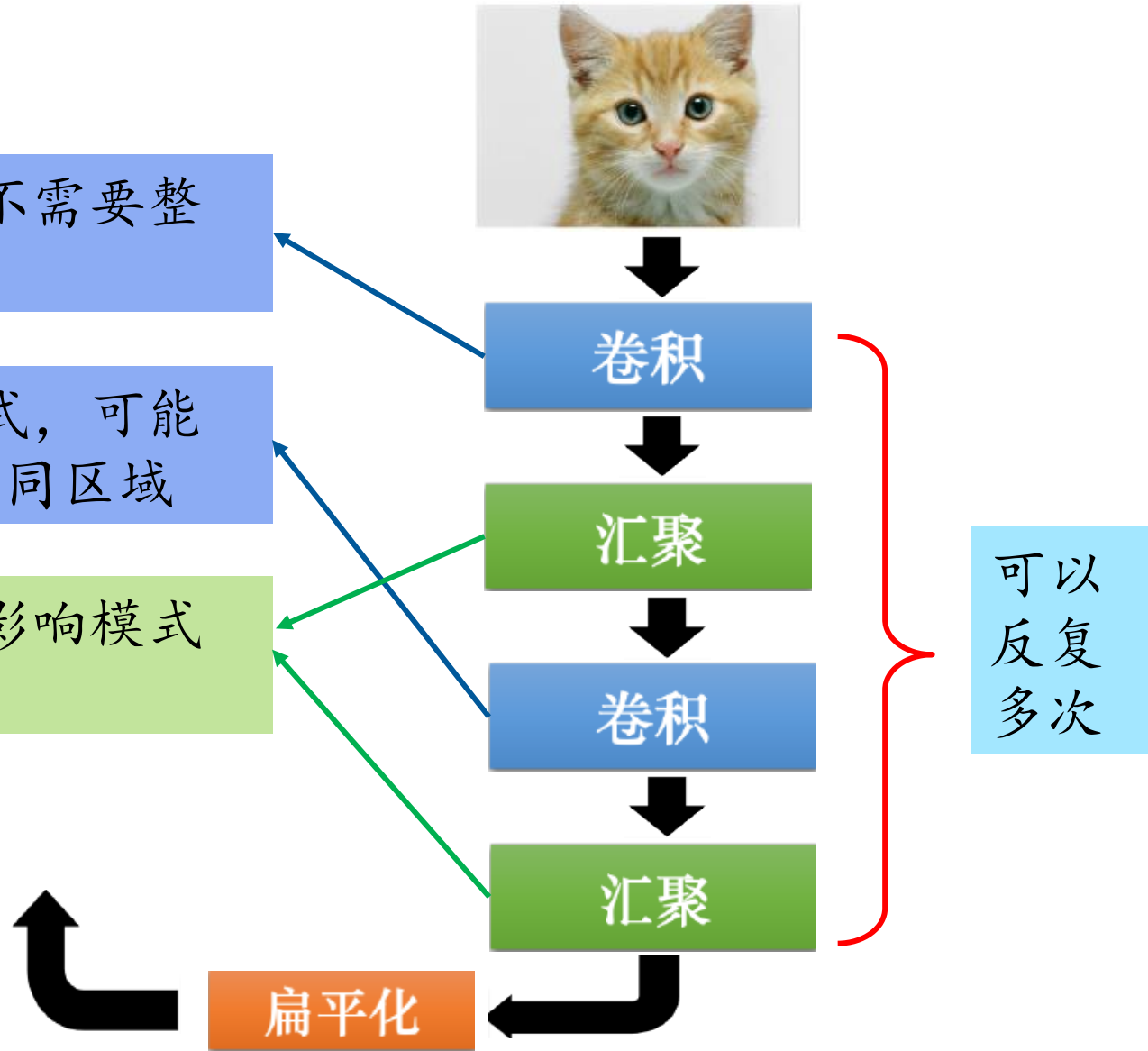
卷积神经网络

➤ CNN的架构

观察1：检测模式不需要整张图像

观察2：同样的模式，可能会出现在图像的不同区域

观察3：下采样不影响模式检测



卷积神经网络

➤ CNN的架构 – 卷积 (Convolution)

滤波器（卷积核）中的参数需要学习

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 × 6 图像



1	-1	-1
-1	1	-1
-1	-1	1

滤波器 1

-1	1	-1
-1	1	-1
-1	1	-1

滤波器 2

⋮

观察1：检测模式
不需要整张图像

每一个滤波器
检测一个模式

卷积神经网络

➤ CNN的架构 – 卷积 (Convolution)

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6×6 图像

1	-1	-1
-1	1	-1
-1	-1	1

滤波器 1

3

卷积神经网络

➤ CNN的架构 – 卷积 (Convolution)

步长=1, padding为0

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 × 6 图像

1	-1	-1
-1	1	-1
-1	-1	1

滤波器 1

3

-1

卷积神经网络

➤ CNN的架构 – 卷积 (Convolution)

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6×6 图像

1	-1	-1
-1	1	-1
-1	-1	1

滤波器 1

3

-1

-3

卷积神经网络

➤ CNN的架构 – 卷积 (Convolution)

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6×6 图像

1	-1	-1
-1	1	-1
-1	-1	1

滤波器 1

3

-1

-3

-1

卷积神经网络

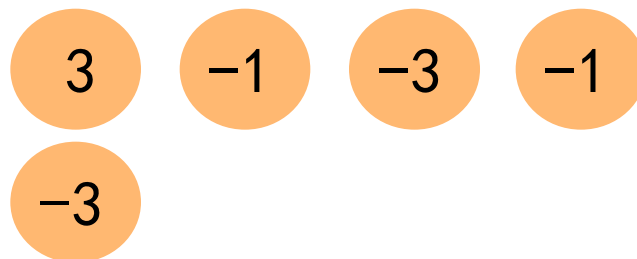
➤ CNN的架构 – 卷积 (Convolution)

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 × 6 图像

1	-1	-1
-1	1	-1
-1	-1	1

滤波器 1



卷积神经网络

➤ CNN的架构 – 卷积 (Convolution)

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6×6 图像

1	-1	-1
-1	1	-1
-1	-1	1

滤波器 1

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

卷积神经网络

➤ CNN的架构 – 卷积 (Convolution)

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 × 6 图像

1	-1	-1
-1	1	-1
-1	-1	1

滤波器 1

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

观察2：同样的模式，可能会出现在图像的不同区域

卷积神经网络

➤ CNN的架构 – 卷积 (Convolution)

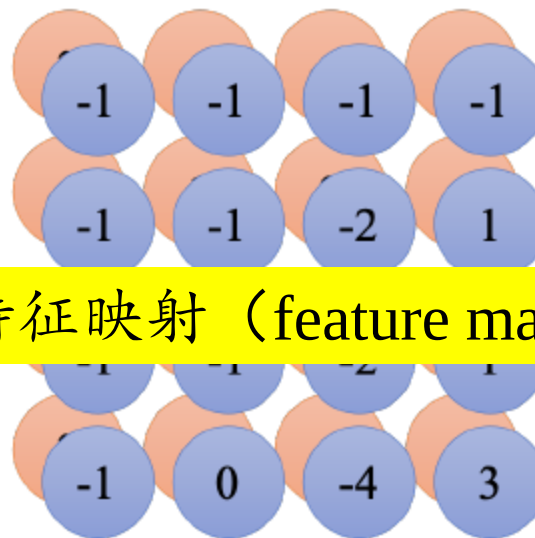
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 × 6 图像

-1	1	-1
-1	1	-1
-1	1	-1

滤波器 2

对每个滤波器做同样的过程



特征映射 (feature map)

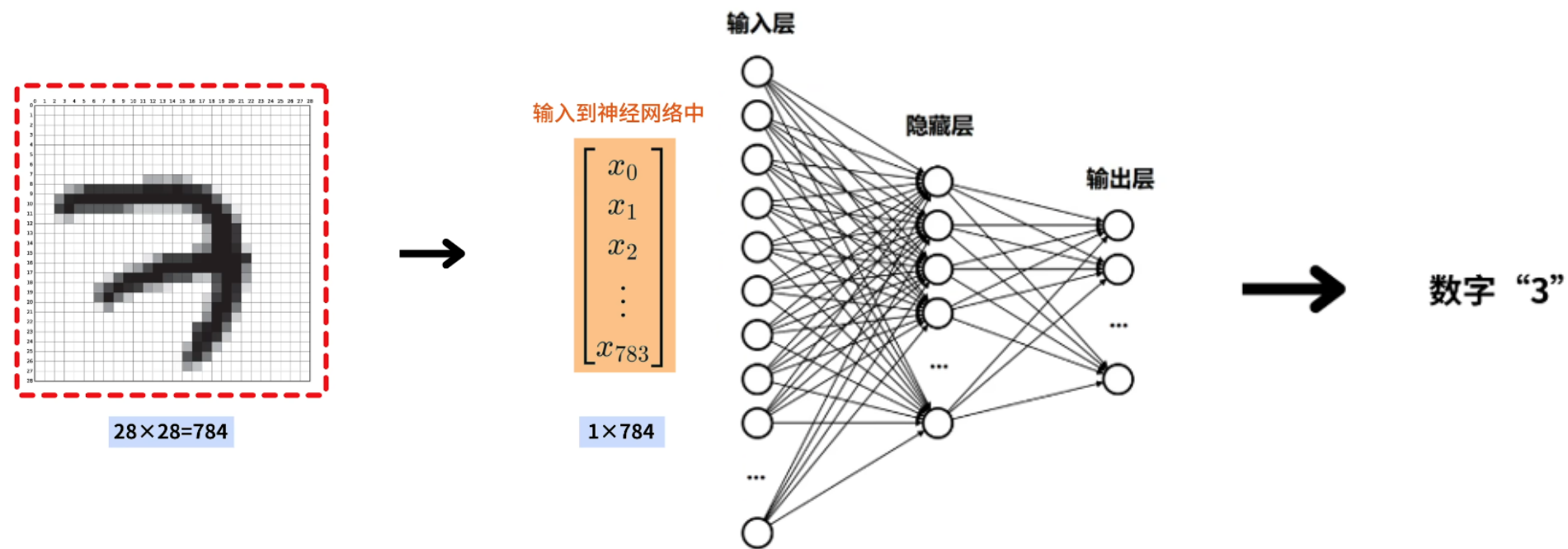
卷积神经网络

➤ CNN的架构 – 卷积 (Convolution)

➤ 卷积和全连接有什么关系？

卷积神经网络

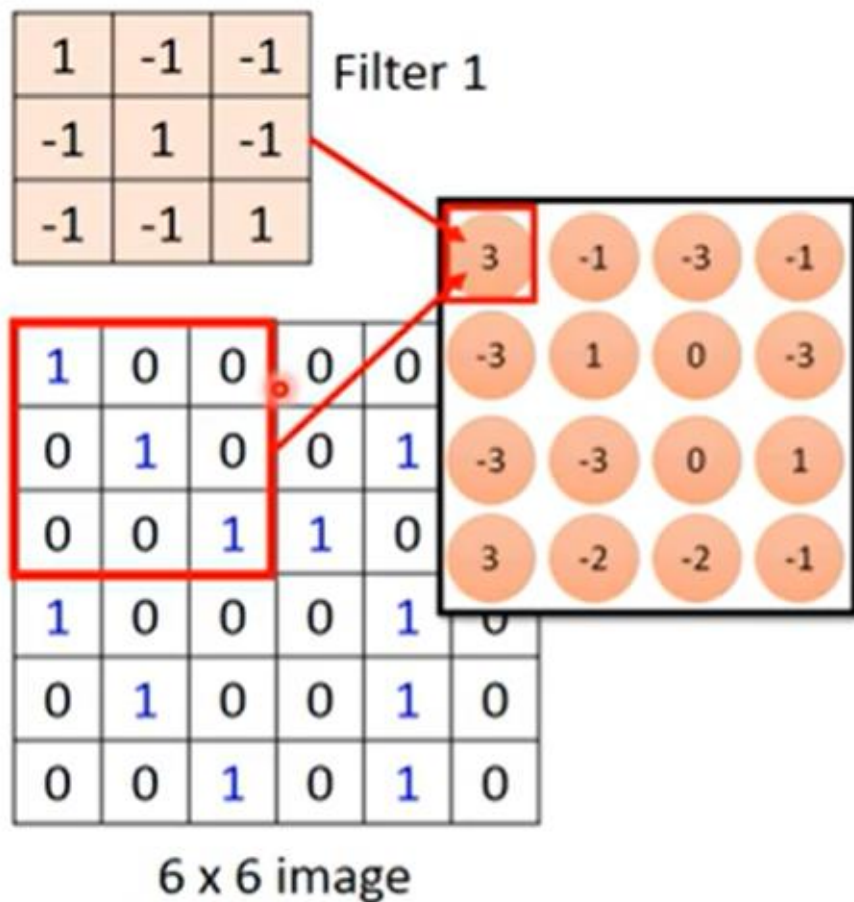
➤ 全连接神经网络



- 把图像展平成一个巨大的向量，作为神经网络的输入(**全连接**)
- 考虑**图像本身的特性**，一定需要全连接吗？

卷积神经网络

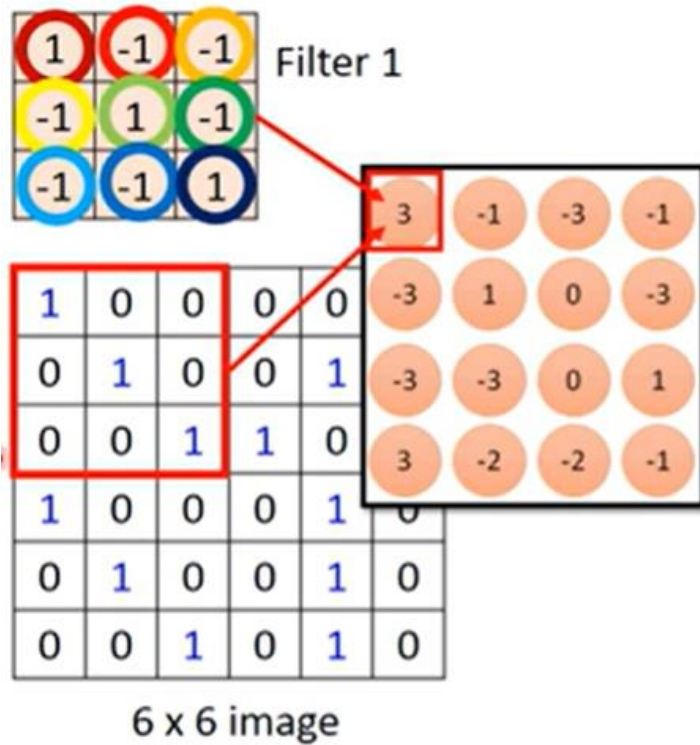
- CNN的架构 – 卷积 (Convolution)
- 卷积和全连接有什么关系？



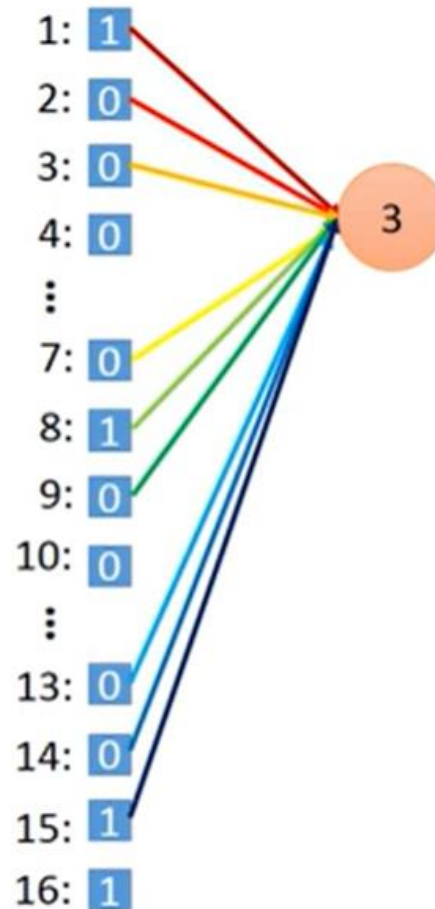
1: 1
2: 0
3: 0
4: 0
⋮
7: 0
8: 1
9: 0
10: 0
⋮
13: 0
14: 0
15: 1
16: 1
⋮

卷积神经网络

- CNN的架构 – 卷积 (Convolution)
- 卷积和全连接有什么关系？

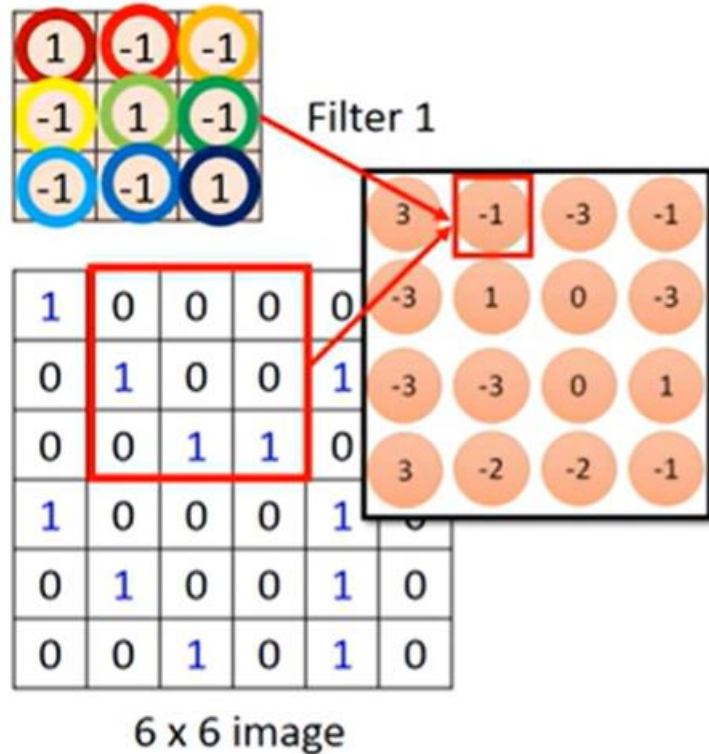


Less parameters!

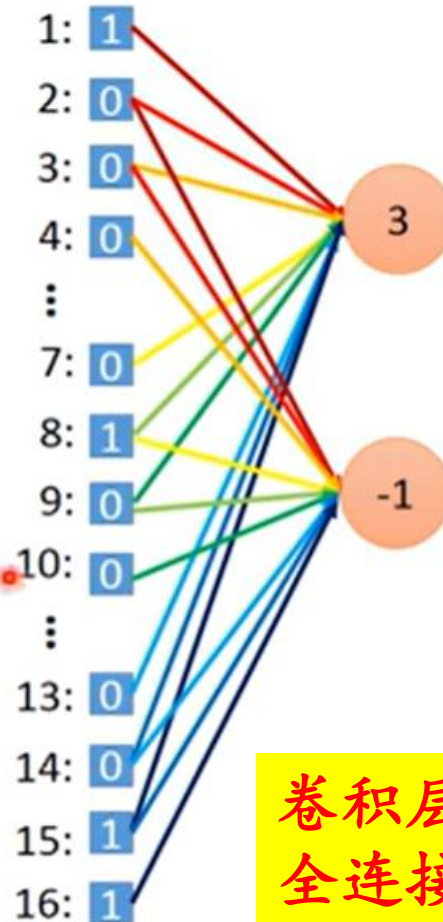


卷积神经网络

- CNN的架构 – 卷积 (Convolution)
- 卷积和全连接有什么关系？



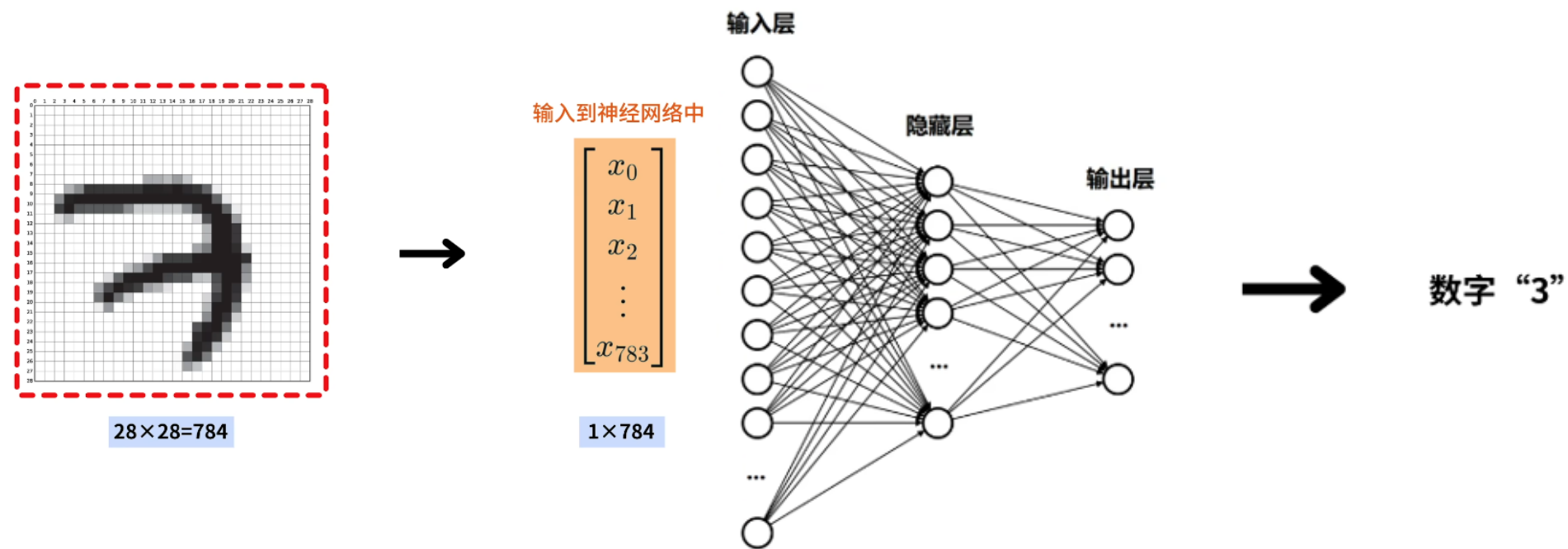
Less parameters!



卷积层有参数/权重共享，
全连接层没有！！！！

卷积神经网络

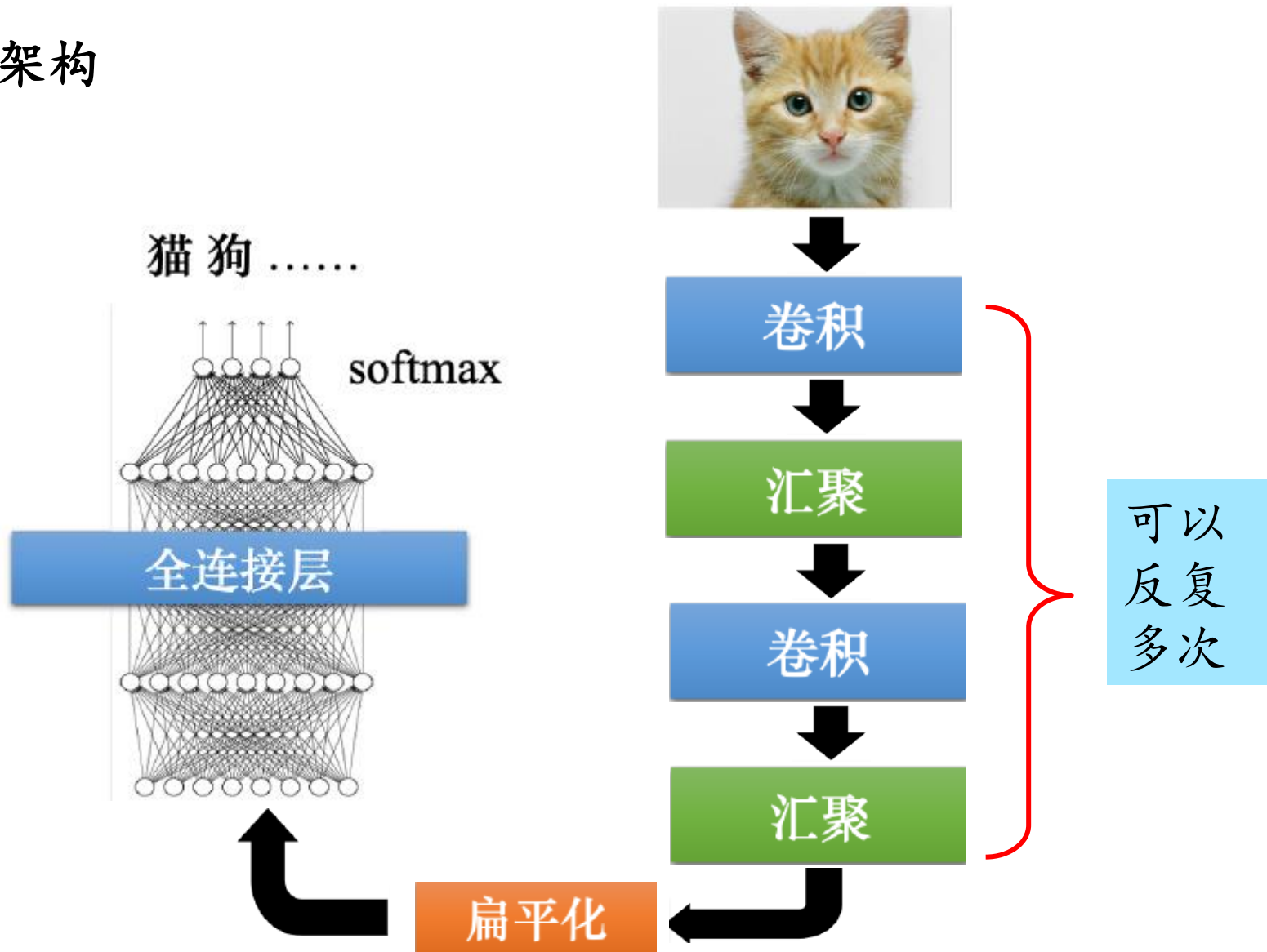
➤ 全连接神经网络



- 把图像展平成一个巨大的向量，作为神经网络的输入(全连接)

卷积神经网络

➤ CNN的架构



卷积神经网络

➤ CNN的架构 – 汇聚（最大池化，Max Pooling）

对卷积结果进行2x2无重叠最大池化操作

1	-1	-1
-1	1	-1
-1	-1	1

滤波器 1

-1	1	-1
-1	1	-1
-1	1	-1

滤波器 2

3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

-1	-1	-1	-1
-1	-1	-2	1
-1	-1	-2	1
-1	0	-4	3

卷积神经网络

➤ CNN的架构 – 汇聚（最大池化，Max Pooling）

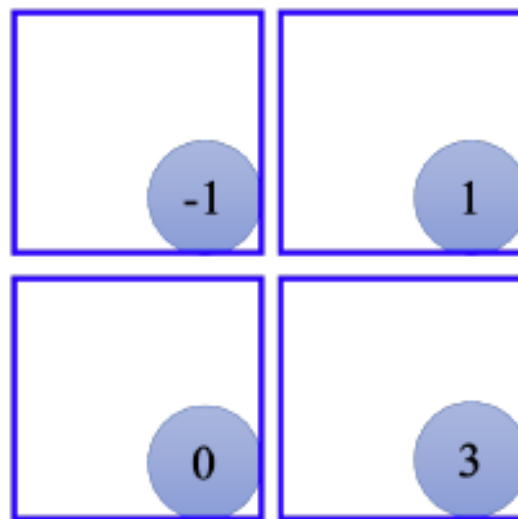
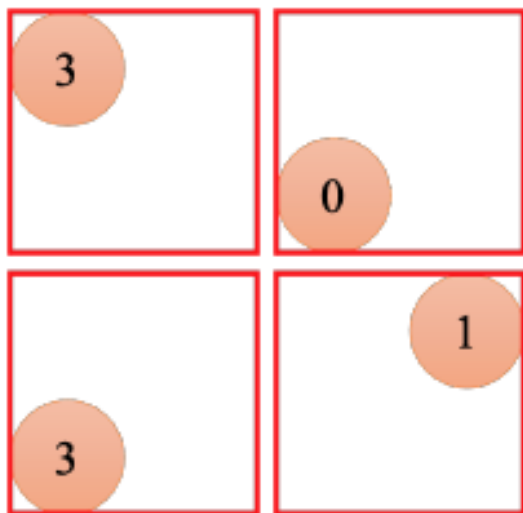
对卷积结果进行2x2无重叠最大池化操作

1	-1	-1
-1	1	-1
-1	-1	1

滤波器 1

-1	1	-1
-1	1	-1
-1	1	-1

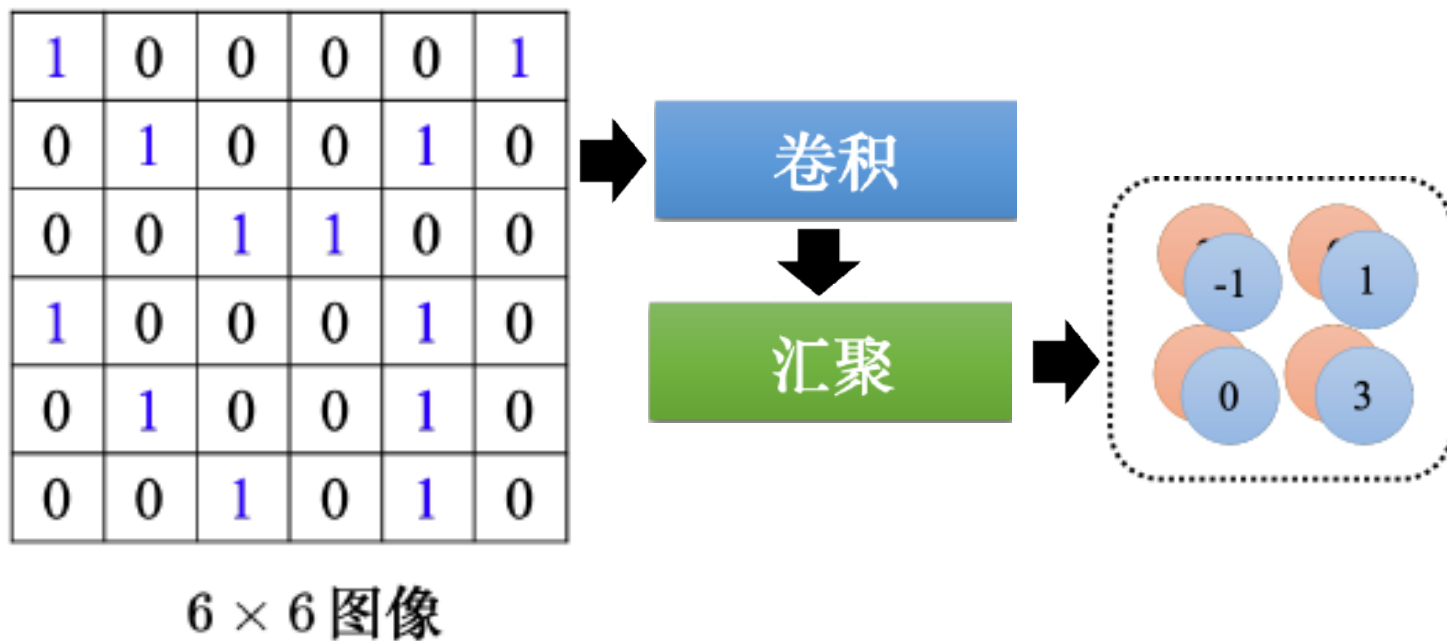
滤波器 2



观察3：下采样不影响模式检测

卷积神经网络

➤ CNN的架构 – 汇聚 (Max Pooling)



卷积神经网络

➤ CNN的架构 – 扁平化 (flatten)



卷积

汇聚

卷积

汇聚

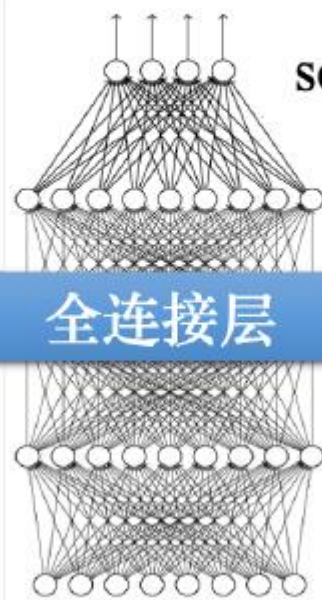
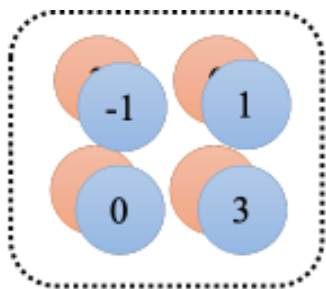
可以
反复
多次

扁平化

猫 狗

softmax

全连接层



卷积神经网络

➤ CNN的架构- 扁平化 (flatten)

